

腾讯云 CVM 实例购买及代理访问

（作者：尚硅谷研究院）

版本：V1.0

第 1 章 CVM 实例购买

1.1 腾讯云 CVM 介绍

1.1.1 概述

云服务器（Cloud Virtual Machine，CVM）是腾讯云提供的安全可靠的弹性计算服务。支持实时扩展或缩减计算资源，适应变化的业务需求，并只需按实际使用的资源计费。使用 CVM 可以极大降低软硬件采购成本，简化 IT 运维工作。

1.1.2 实例类型

目前，腾讯云提供三种 CVM 实例，如下。

- 包年包月：包年包月是购买云服务器实例的一种预付费模式，需要一次性支付所选择时间区间的费用，这种模式适用于提前预估设备需求量的场景，价格相较于按量计费模式更低廉。
- 按量计费：按量计费是购买云服务器实例的一种后付费模式，不需要提前支付费用，但需要冻结一定的费用，可以随时开通/销毁云服务器，按实例的实际使用量付费，计费时间粒度精确到秒，每小时整点进行一次结算，价格比包年包月计费模式高 3-4 倍。
- 竞价实例：竞价实例是类似按量付费模式的一种后付费模式（按秒计费，整点结算），与按量付费相比，竞价实例采用市场浮动计费，同时会因为资源库存减少、其他用户出价竞争而发生系统主动回收实例的情况。与稳定性下降所对应的，竞价实例相比按量付费会有较大幅度优惠，一般价格区间为后者的 10%~20%。

1.2 竞价实例购买流程

1) 注册腾讯云账户

注册链接如下：

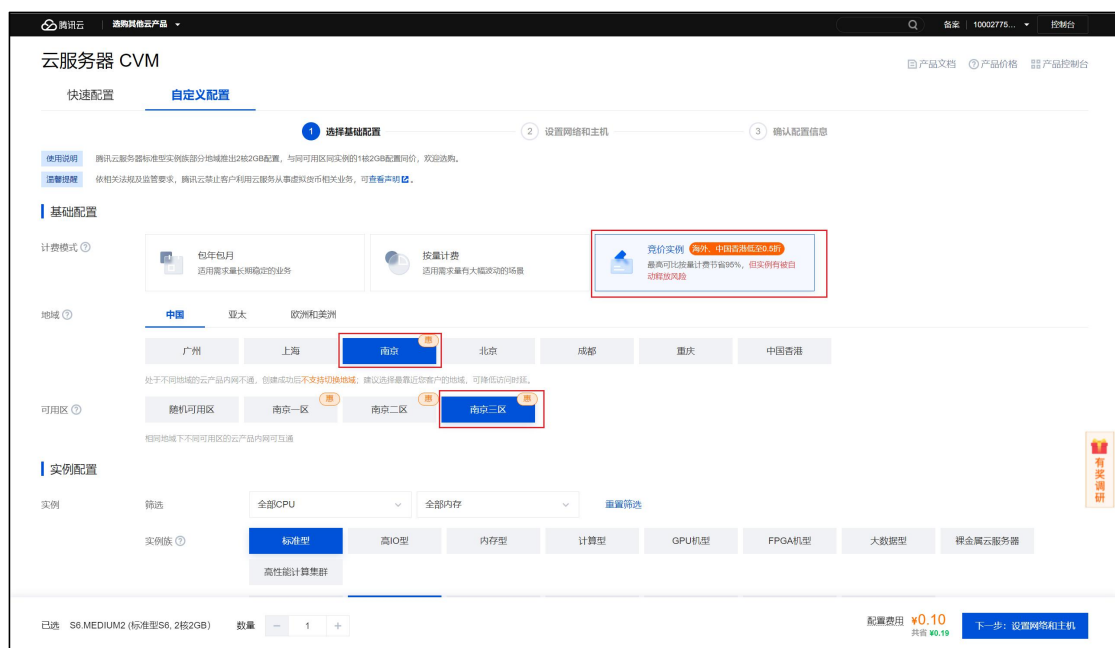
更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

https://buy.cloud.tencent.com/cvm?tab=custom&step=1&devPayMode=spotpaid®ionId=33&zoneId=330003&instanceType=S6.MEDIUM2&platform=TencentOS&systemDiskType=CL_OUD_BSSD&systemDiskSize=50&systemDiskBackupQuota=0&templateCreateMode=createLt&invitation=100031932357



微信扫码，按提示操作即可

2) 选择实例类型及可用区



“计费模式”选择“竞价实例”，“地域”及“可用区”随意选择

3) 实例配置

更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：尚硅谷官网

实例配置

实例 预选 4核 16GB 重置筛选

实例族 ① 标准型 高性能计算集群

类型 ② 全部实例类型 标准型S6 标准型SA3 标准型S5 标准型SA2 标准型S4 其他类型

已选实例 SA2.LARGE16 (标准型SA2, 4核 16GB)

实例 ①	规格	vCPU ①	内存	处理器型号	内网带宽	网络收发包 ①	支持可用区	参考费用 ①
☐ 标准型S6 (2.0RT)	S6.LARGE16	4核	16GB	Intel Ice Lake(2.7GHz/3.3GHz)	4Gbps	60万PPS	-	0.188元/小时 0.64元/小时
☐ 标准型SA3 (2.0RT)	SA3.LARGE16	4核	16GB	AMD EPYC™ Milan(2.55GHz/3.0GHz)	2Gbps	50万PPS	-	0.14元/小时 0.7元/小时
☐ 标准型S5 (2.0RT)	S5.LARGE16	4核	16GB	Intel Xeon Cascade Lake 8255C...	1.5Gbps	50万PPS	-	0.18元/小时 0.6元/小时
☒ 标准型SA2 (2.0RT)	SA2.LARGE16	4核	16GB	AMD EPYC™ Rome(2.6GHz/3.0GHz)	1.5Gbps	50万PPS	-	0.134元/小时 0.67元/小时

竞价出价模式 跟随市场价格

镜像 ③ 公共镜像 自定义镜像 共享镜像 镜像市场

已选 SA2.LARGE16 (标准型SA2, 4核 16GB) 数量 1

配置费用 ¥0.24 共收 ¥0.75 下一步: 设置网络和安全组

(1) “实例配置”根据需求选择，采集实战集群规模为 3*2 核 8G，离线及实时实战集群规模为 5*4 核 16G

(2) “实例族”选择“标准型”，“实例类型”选择“全部实例类型”

(3) “实例类型”随意选择

4) 镜像选择

标准型SA2 (2.0RT) SA2.LARGE16 4核 16GB AMD EPYC™ Rome(2.6GHz/3.0GHz) 1.5Gbps 50万PPS - 0.134元/小时
0.67元/小时

竞价出价模式 跟随市场价格

镜像 ③ 公共镜像 自定义镜像 共享镜像 镜像市场

TencentOS OpenCloudOS CentOS Windows Ubuntu Debian

CentOS 7.5 64位

CentOS 官方计划停止维护 CentOS Linux 项目，并于2022年01月01日停止对 CentOS 8的维护支持，CentOS 7于2024年06月30日也将停止维护。腾讯云为您提供替代方案，操作指南参见文档。

存储 ④

用途	类型	容量	数量	总性能
系统盘	通用型SSD云硬盘	40 GB	1	基准性能: 2400IOPS, 108MB/s 带宽

添加数据盘 您还可添加 20 块数据盘

定期快照

☐ 系统盘定期快照 default-policy(星期五...17:00)保留 15天后自动删除

☐ 数据盘定期快照 default-policy(星期五...17:00)保留 15天后自动删除

快照可恢复由用户创建，病毒感染等情况导致的数据异常，目前中国境内每个地域提供 80GB免费额度，详情可见 快照计费概述

已选 SA2.LARGE16 (标准型SA2, 4核 16GB) 数量 1

配置费用 ¥0.17 共收 ¥0.54 下一步: 设置网络和安全组

(1) “竞价出价模式”默认为“跟随市场价格”，无法更改

(2) “镜像”选择“CentOS 7.5 64 位”

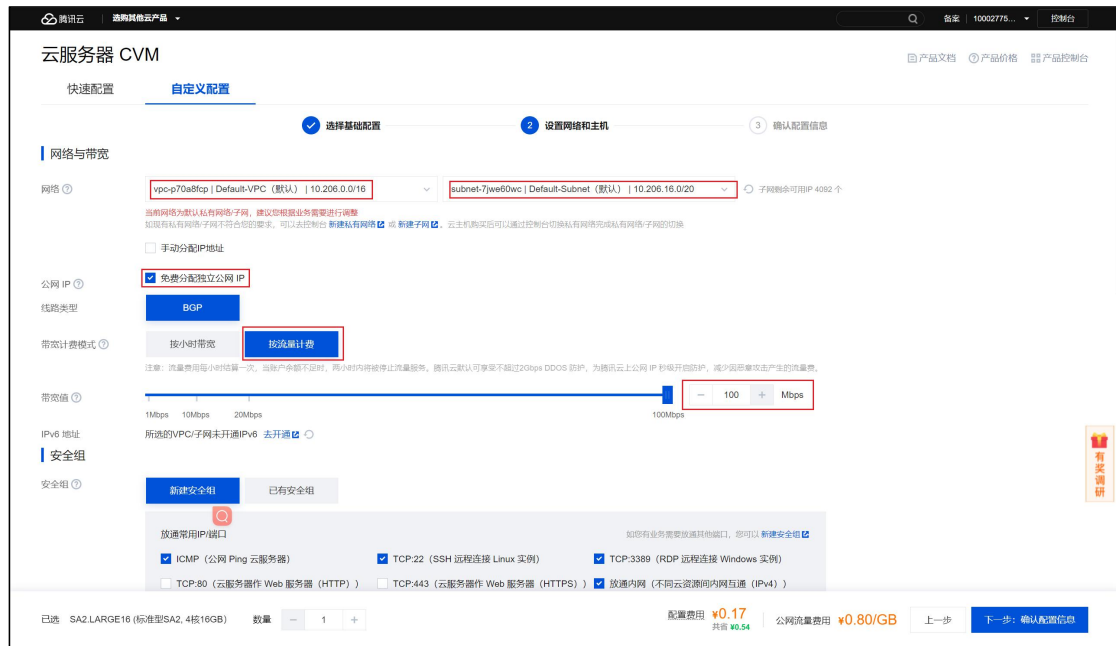
(3) “存储”选择“通用型 SSD 云硬盘”，“容量”为 40GB，此项可以根据实际情况调整

(4) “定期快照”可以不选择，也可以根据实际情况勾选

(5) “数量”为集群服务器的数量，根据实际情况调整。采集实战为 3 台，离线及实时实战为 5 台

(6) 上述内容设置完毕后，选择“下一步：设置网络和主机”

5) 网络及带宽



(1) “网络”保持默认即可

(2) “公网 IP”勾选“免费分配独立公网 IP”

(3) “带宽计费模式”选择“按流量计费”

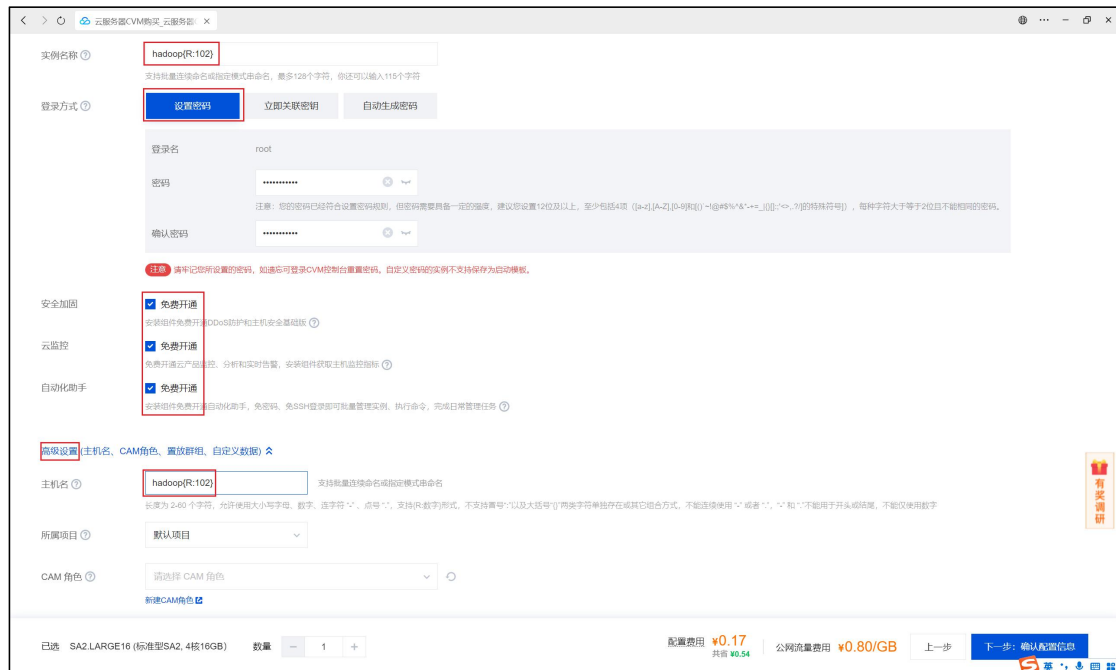
(4) “带宽值”拉满，设置为“100Mbps”

6) 安全组



选择“新建”安全组，按图示勾选即可

7) 实例名称及主机名



实例名称

登录方式 ☒ 设置密码 ☐ 立即关联密钥 ☐ 自动生成密码

登录名: root

密码:

确认密码:

安全加固 ☒ 免费加固

云监控 ☒ 免费开通过

自动化助手 ☒ 免费开通过

高级设置 主机名、CAM 角色、置放群组、自定义数据

主机名

所属项目

CAM 角色

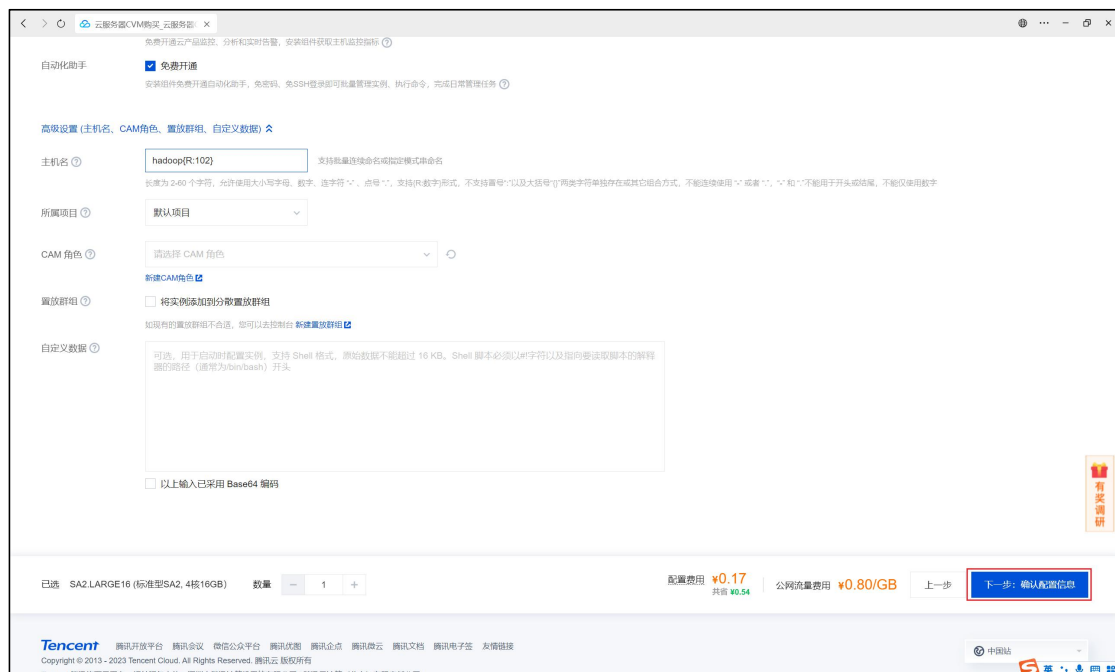
已选 SA2.LARGE16 (标准型SA2, 4核16GB) 数量 1

配置费用 ¥0.17 公网流量费用 ¥0.80/GB

下一步: 确认及支付

有序后缀: 可将批量购买的实例设置为复杂且指定序号的实例名称。实例名称支持指定单个或者多个模式串，命名语法：**{R:x}**，x 表示生成实例名称的初始序号。如上图 **hadoop{R:102}** 表示主机名前缀为 **hadoop**，有序后缀起始编号为 102，若一次性购买三台服务器，则名称依次为 **hadoop102**，**hadoop103**，**hadoop104**

- (1) 主机名称：按照上述方法定义有序后缀
- (2) 登录方式：选择密码登录，自定义密码
- (3) 安全加固、云监控、自动化助手：全部勾选
- (4) 主机名：域主机名称设置方式相同，主机名的设置默认折叠在“高级设置”中，鼠标左键单击“高级设置”即可打开折叠的菜单



自动化助手 ☒ 免费开通
安装组件免费开通自动化助手，免密码，免SSH登录即可批量管理实例，执行命令，完成日常运维任务

高级设置 (主机名、CAM角色、置放群组、自定义数据)

主机名 支持批量连续命名或指定模式串命名
长度为 2-60 个字符，允许使用大小写字母、数字、连字符“-”、点号“.”；支持负数形式；不支持符号“!”以及大括号“{}”两类字符单独存在或混合使用；不能连续使用“-”或者“.”；“!”和“{”不能用于开头或结尾，不能仅使用数字

所属项目

CAM 角色 [新建CAM角色](#)

置放群组 ☐ 将实例添加至分散置放群组
如现有的置放群组不合适，您可以去控制台的 [新建置放群组](#)

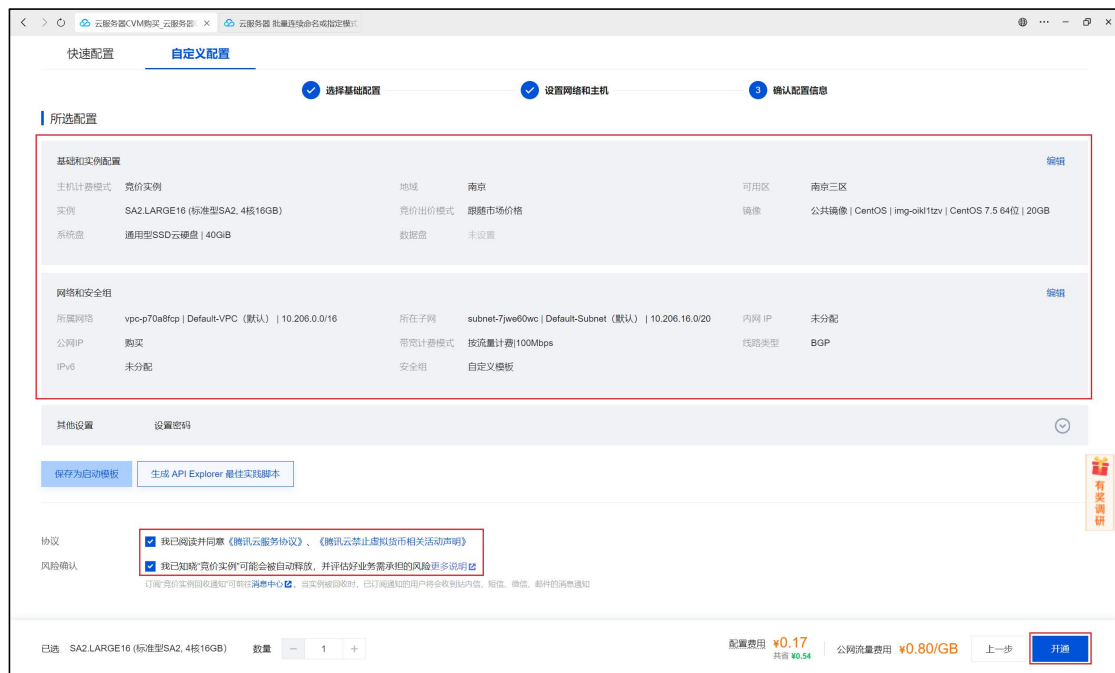
自定义数据
☐ 以上输入已采用 Base64 编码

已选 SA2.LARGE16 (标准型SA2, 4核16GB) 数量 配置费用 ¥0.17 共需 ¥0.54 公网流量费用 ¥0.80/GB 上一步 [下一步：确认配置信息](#)

Tencent 腾讯开放平台 腾讯会议 微信公众平台 腾讯优图 腾讯企点 腾讯文档 腾讯电子签 友情链接
Copyright © 2013 - 2023 Tencent Cloud. All Rights Reserved. 腾讯云 版权所有
Tencent 腾讯旗下云平台 腾讯云安全中心 深圳市腾讯计算机系统有限公司 腾讯云计算 (台湾) 有限公司

点击“下一步：确认配置信息”

8) 确认购买



快速配置 [自定义配置](#)

选择基础配置 ☒ 设置网络和主机 ☒ 确认配置信息 ☒

所选配置

基础和实例配置		地域	可用区	网络和安全
主机计费模式	竞价实例	南京	南京三区	网络
实例	SA2.LARGE16 (标准型SA2, 4核16GB)	竞价出的模式	跟随市场价格	镜像
系统盘	通用型SSD云硬盘 40GB	数据盘	未设置	公共镜像 CentOS img-aki1tzu CentOS 7.5 64位 20GB
网络和安全组		所在子网	子网	内网 IP
所属网络	vpc-p70a8fcp Default-VPC (默认) 10.206.0.0/16	subnet-7jwe60wc Default-Subnet (默认) 10.206.16.0/20	未分配	未分配
公网IP	购买	带宽计费模式	按流量计费100Mbps	线路类型
IPv6	未分配	安全组	自定义模板	BGP

其他设置 设置密码

[保存为自动模板](#) [生成 API Explorer 最佳实践脚本](#)

协议 ☒ 我已阅读并同意《腾讯云服务协议》、《腾讯云禁止虚拟货币相关活动声明》
风险确认 ☒ 我已知晓“竞价实例”可能会被自动释放，并评估好业务需求承担的风险[更多说明](#)
订购“竞价实例”前请通知“竞价中心”，购买时请仔细阅读《竞价实例购买须知》，已订购通知的用户将会收到内部、微信、邮件的购买通知

已选 SA2.LARGE16 (标准型SA2, 4核16GB) 数量 配置费用 ¥0.17 共需 ¥0.54 公网流量费用 ¥0.80/GB 上一步 [开通](#)

核对配置信息，勾选协议，点击“开通”即可完成购买。随后会自动跳转至控制台。

第 2 章 离线实战代理访问方式

2.1 知识储备

更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：尚硅谷官网

2.1.1 腾讯云安全组

安全组是一种虚拟防火墙，用于控制安全组内 CVM 实例的入流量和出流量，从而提高 CVM 实例的安全性。安全组具备状态检测和数据包过滤能力，用户可以基于安全组的特性和安全组规则的配置在云端划分安全域。

安全组规则的配置分为入方向和出方向，入方向规定允许 IP 处于哪些网段的机器访问本机，并指定允许访问的端口。出方向规定腾讯云机器可以访问哪些机器的哪些端口，安全组出方向默认允许所有访问，即从安全组内节点访问外部都是放行的。用户可以通过配置安全组，指定网段和端口，只有 IP 处于该网段的节点可以访问本机，且仅能访问指定端口。如：要通过本地浏览器访问 HDFS WebUI 界面（访问端口为 9870），就需要在入方向中添加规则，如下。



➤ 协议类型：通过浏览器访问 HDFS WebUI，会发出一次 HTTP 请求，而后通过 TCP 与服务器建立通道。HTTP 协议是应用层协议，主要是解决如何包装数据。而 TCP 协议是传输层协议，主要解决数据如何在网络中传输。HTTP 协议通常是建立在 TCP 连接之上的，此处协议类型选择 TCP 即可。

➤ 端口范围：允许访问的本地端口，此处为 9870。

➤ 授权对象：允许访问的节点 IP 范围，0.0.0.0/0 表示不做限制，即任意节点均可访问。

2.1.2 SSH 代理

2.1.2.1 SSH

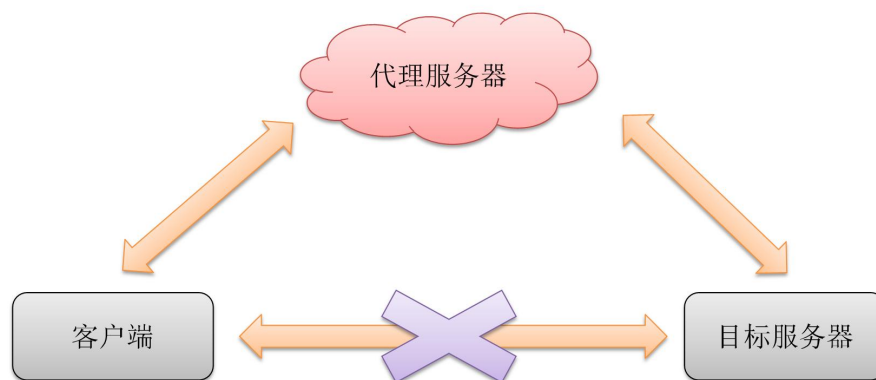
SSH 为 Secure Shell 的缩写，由 IETF 的网络小组（Network Working Group）所制定；SSH 为建立在应用层基础上的安全协议。SSH 是较可靠，专为远程登录会话和其他网络服务提供安全性的协议。利用 SSH 协议可以有效防止远程管理过程中的信息泄露问题。SSH 最初是 UNIX 系统上的一个程序，后来又迅速扩展到其他操作平台。SSH 在正确使用时可弥补网络中的漏洞。SSH 客户端适用于多种平台。几乎所有 UNIX 平台—包括 HP-UX、Linux、AIX、Solaris、Digital UNIX、Irix，以及其他平台，都可运行 SSH。

通过 Xshell、MobaXterm 等工具远程连接服务器就是基于 SSH 实现的。

2.1.2.2 代理

通常，使用浏览器访问某站点，是直接连接该站点的服务器，服务器根据我们的请求返回信息。但在某些情况下客户端无法直连 Web 服务器，此时可以在客户端和 Web 服务器之间架设一台代理服务器，客户端的请求会被发送到代理服务器，后者帮助将请求转发给 Web 服务器，并将 Web 服务器返回的信息转发给客户端。

代理架构图



让天下没有难学的技术

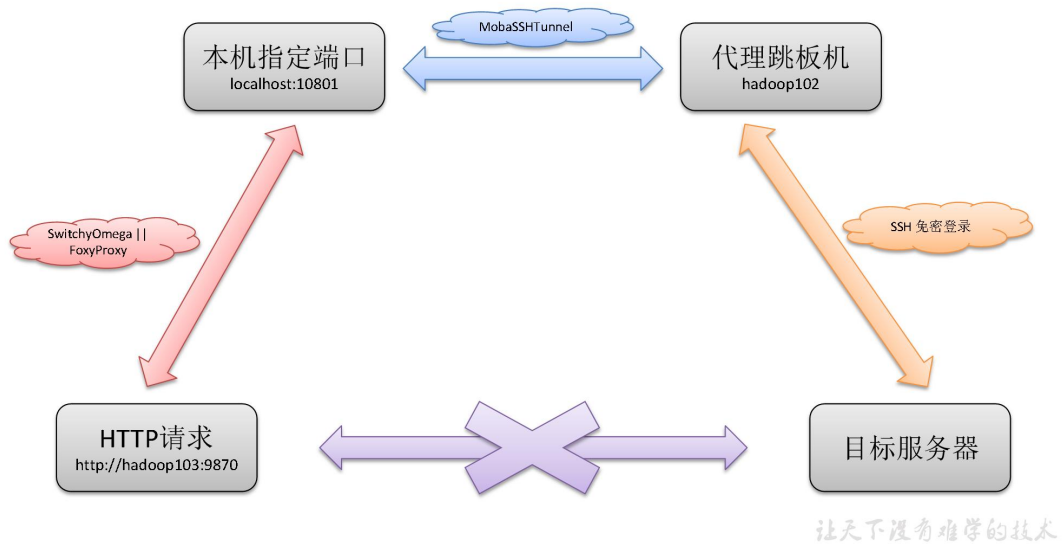
代理服务器要能同时与客户端所在机器和 Web 服务器建立连接。

通常代理是基于 SSH 协议的。

2.1.3 代理访问腾讯云节点架构

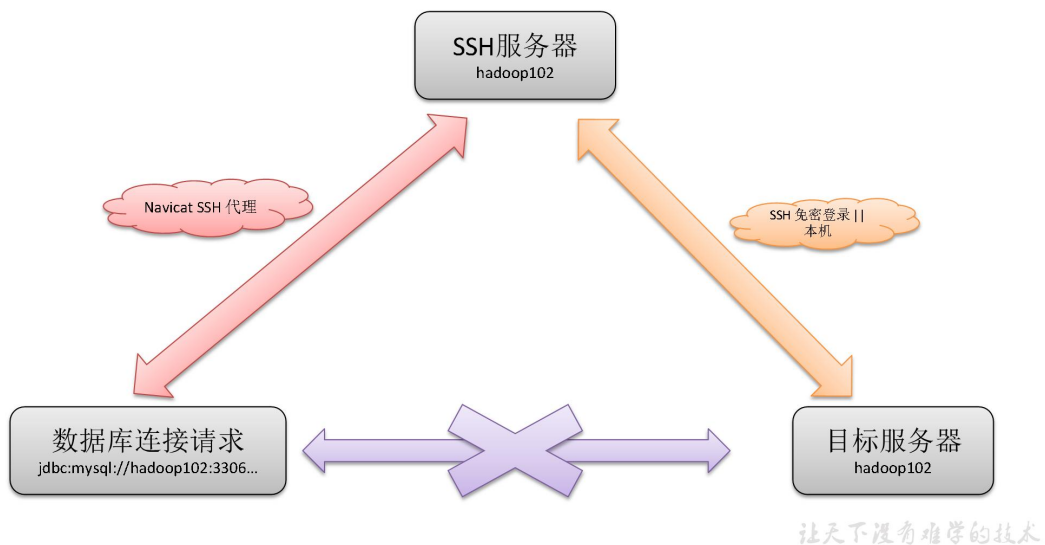
2.1.3.1 浏览器访问 WebUI

浏览器访问WebUI代理架构图



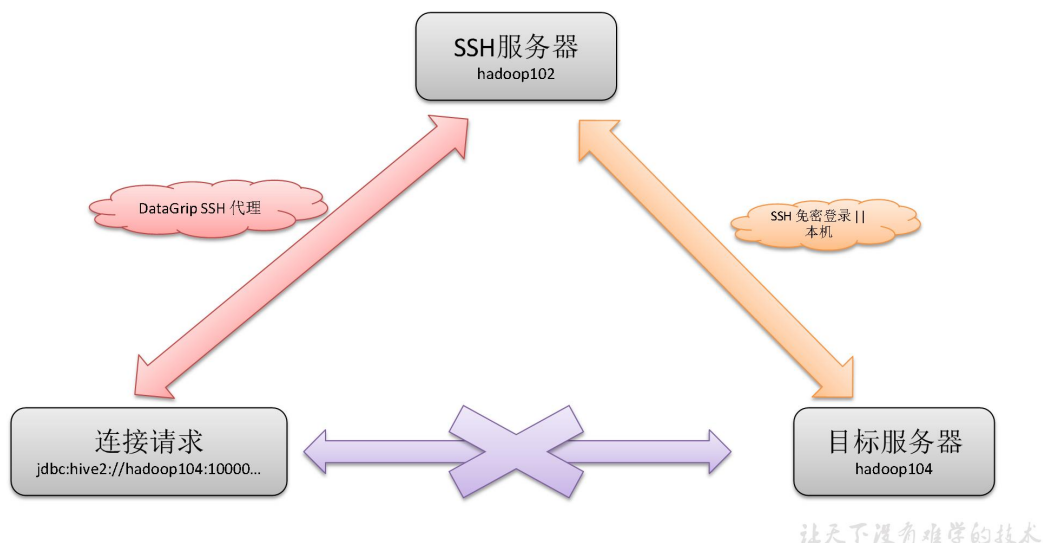
2.1.3.2 Navicat 连接 Mysql

Navicat连接Mysql代理架构图



2.1.3.3 DataGrip 连接 Hiveserver2

DataGrip连接Hiveserver2代理架构图



2.2 代理配置

2.2.1 HTTP 请求代理配置

2.2.1.1 浏览器配置

部分浏览器提供了代理插件，此处仅对 SwitchyOmega 和 FoxyProxy 进行介绍。

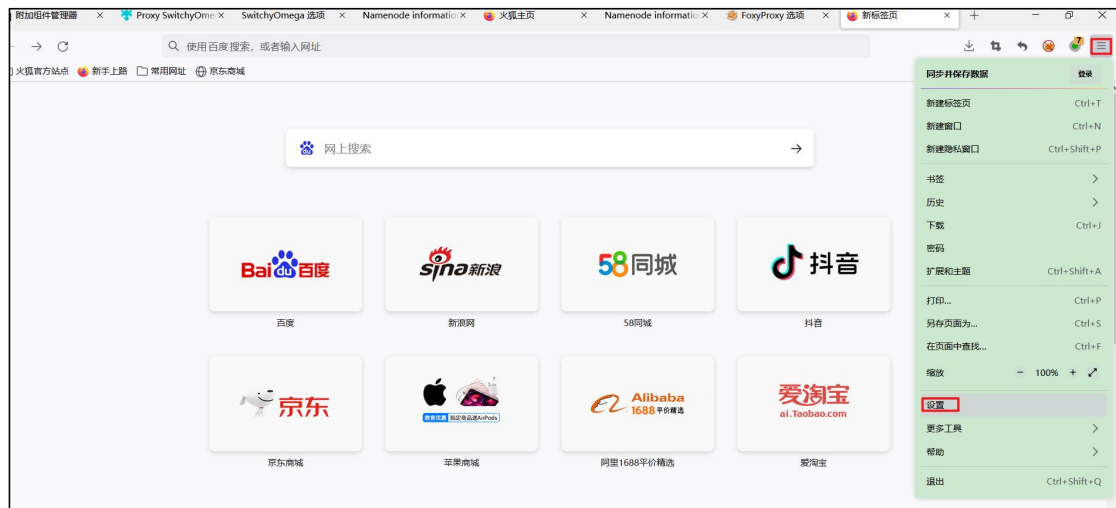
1) SwitchyOmega

SwitchyOmega 可以在基于 Chromium 或 Mozilla 内核的浏览器中安装，可以将指定格式的 HTTP 请求发送到转发到本机的指定端口。常见的浏览器 Chrome、Edge 基于 Chromium 内核，Firefox 基于 Mozilla 内核。受限于网络状况，Chrome 浏览器安装 SwitchyOmega 可能失败，此处仅对 Firefox 浏览器配置进行介绍，Chrome 及 Edge 配置同理。

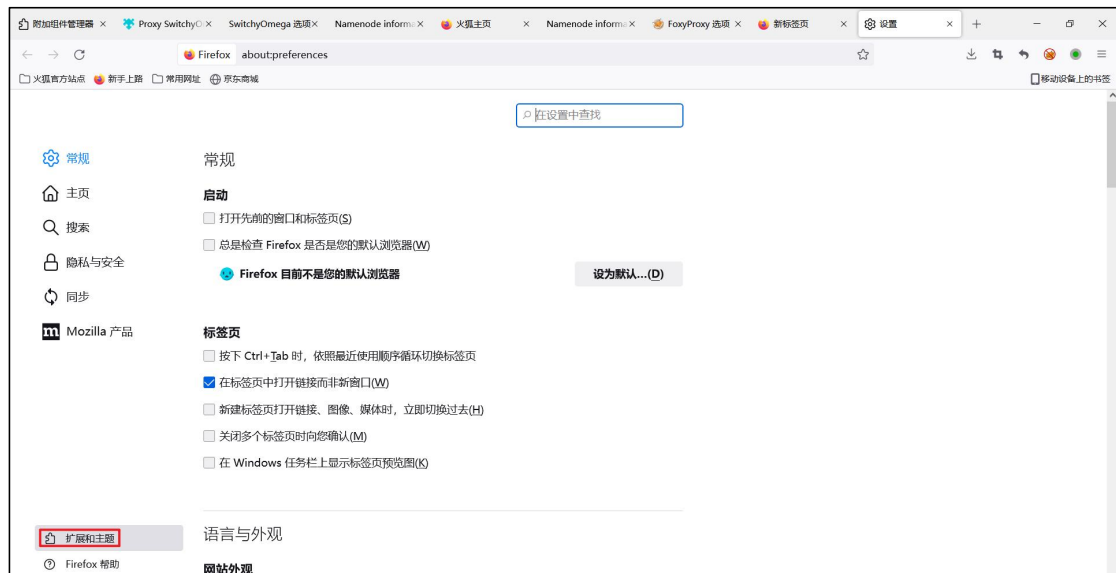
官网地址：<https://proxy-switchyomega.com/>

(1) 安装部署

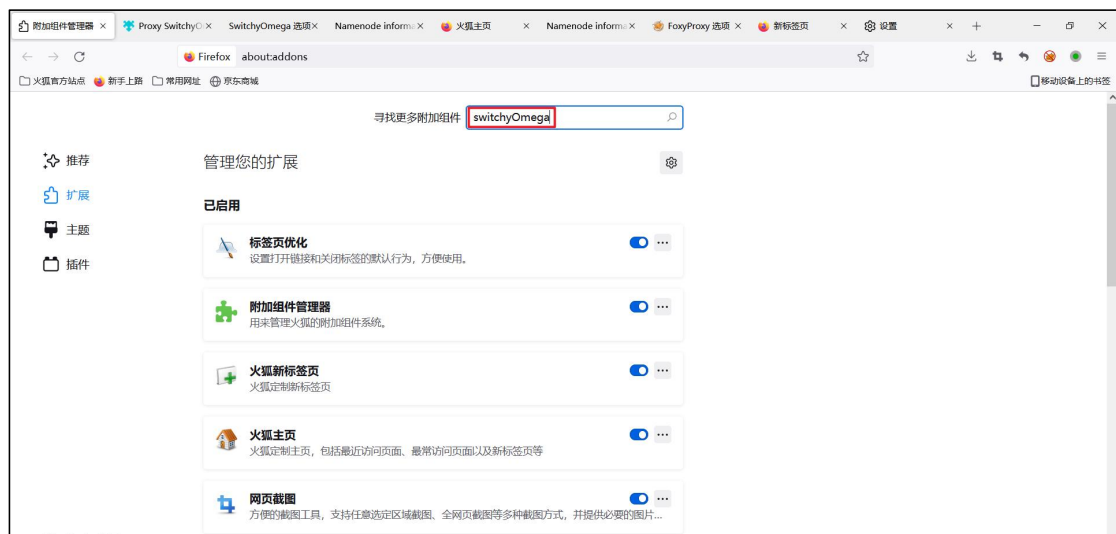
① 选择“设置”



② 选择“扩展和主题”



③ 搜索“SwitchyOmega”，安装即可



(2) 代理配置

① 配置代理服务器



➤ 代理协议：SOCKS5 是一个代理协议，它在使用 TCP/IP 协议通讯的前端机器和服务端机器之间扮演一个中介角色，使得内部网中的前端机器变得能够访问 Internet 网中的服务器，或者使通讯更加安全。SOCKS5 服务器通过将前端发来的请求转发给真正的目标服务器，模拟了一个前端的行为。在这里，前端和 SOCKS5 之间也是通过 TCP/IP 协议进行通讯，前端将原本要发送给真正服务器的请求发送给 SOCKS5 服务器，然后 SOCKS5 服务器将请求转发给真正的服务器。

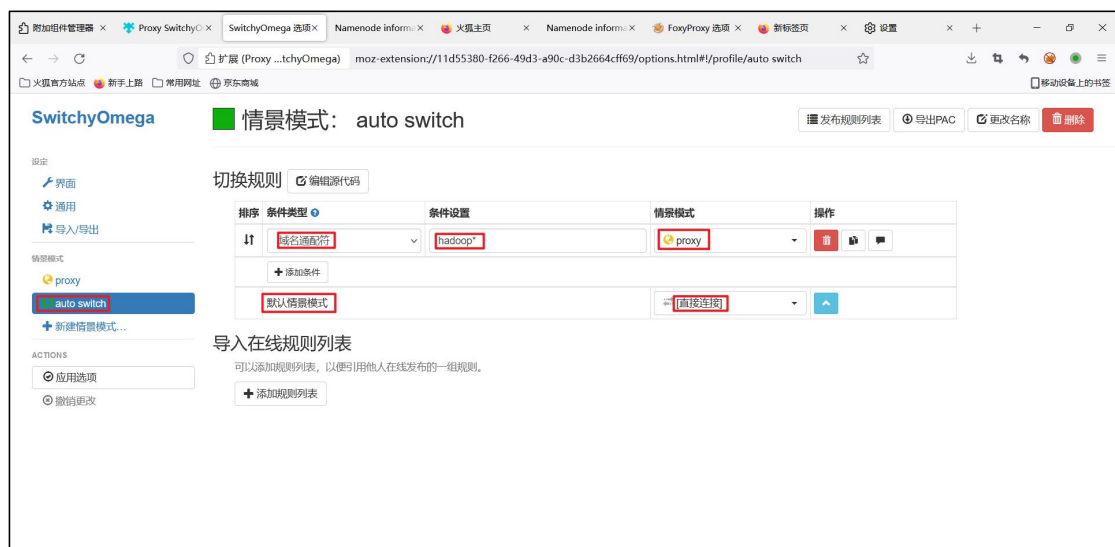
➤ 代理服务器：代理服务器 IP 或主机名。

➤ 代理端口：代理服务器端口。

➤ 不代理的地址列表：顾名思义，发送给以下主机的请求不会被转发给代理服务器。

此处表示将满足条件的所有 HTTP 请求转发到本机的 10801 端口。

② 代理规则配置



此处表示对于以 `hadoop` 开头的所有请求使用（1）中指定的代理规则处理，其余请求直接连接对应的 Web 服务器，不会被转发给代理服务器的指定端口（此处为 `localhost:10801`）。

2) FoxyProxy

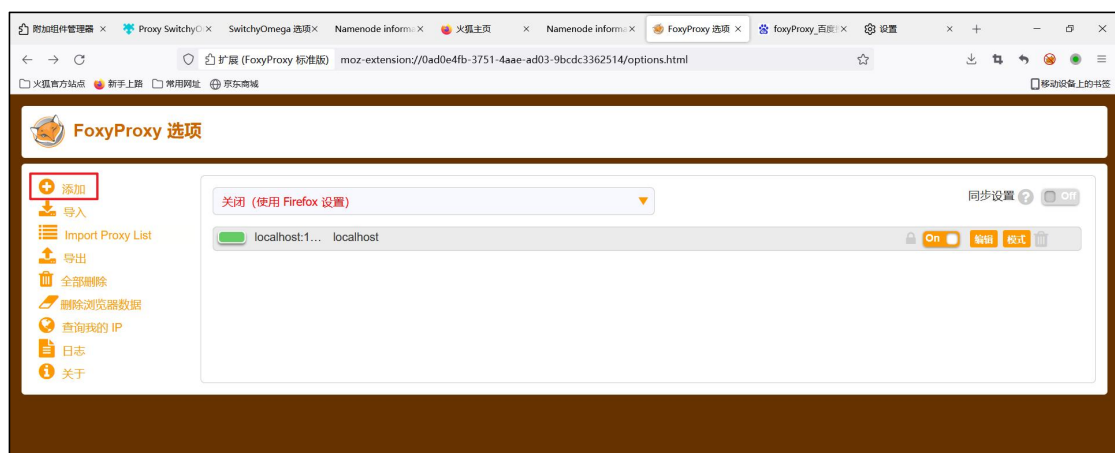
FoxyProxy 是一款火狐浏览器插件，不能用于 Chrome 浏览器。功能与 SwitchyOmega 相似，但不能设置代理规则，需要手动在代理和直连模式间切换。

（1）安装部署

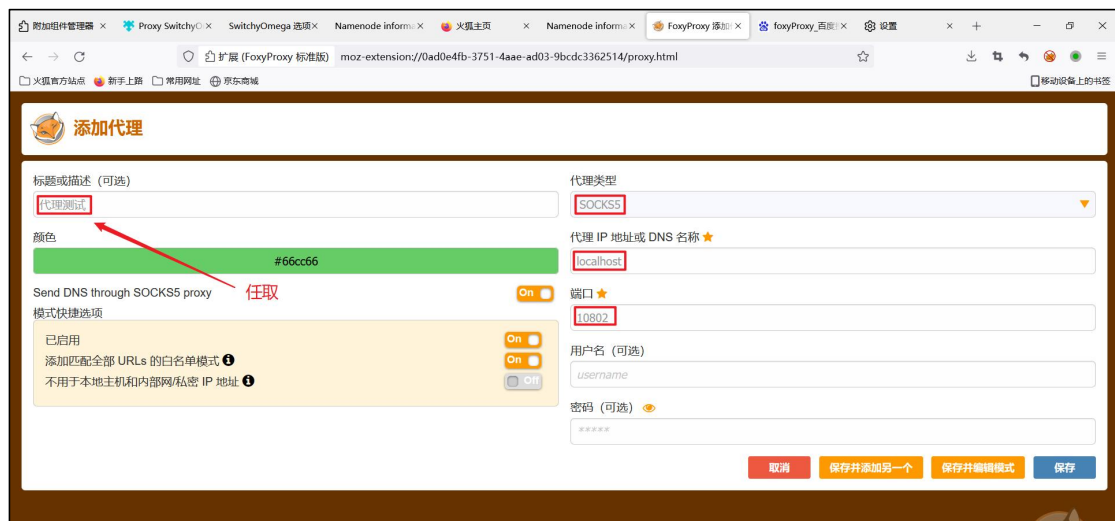
与 SwitchyOmega 相似，在插件市场搜索 FoxyProxy 即可。

（2）配置

① 点击“添加”添加规则

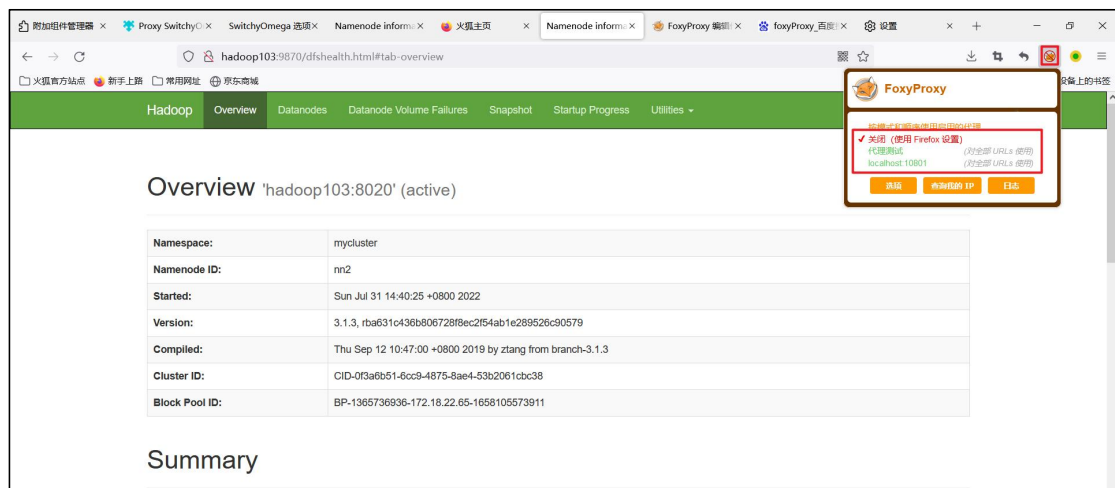


② 在弹出的对话框中自定义代理信息



标题任取，代理类型、代理 IP 和端口同 SwitchyOmega。

③ 配置完成后即可在浏览器右上角选择代理或直连模式。



2.2.1.2 隧道配置

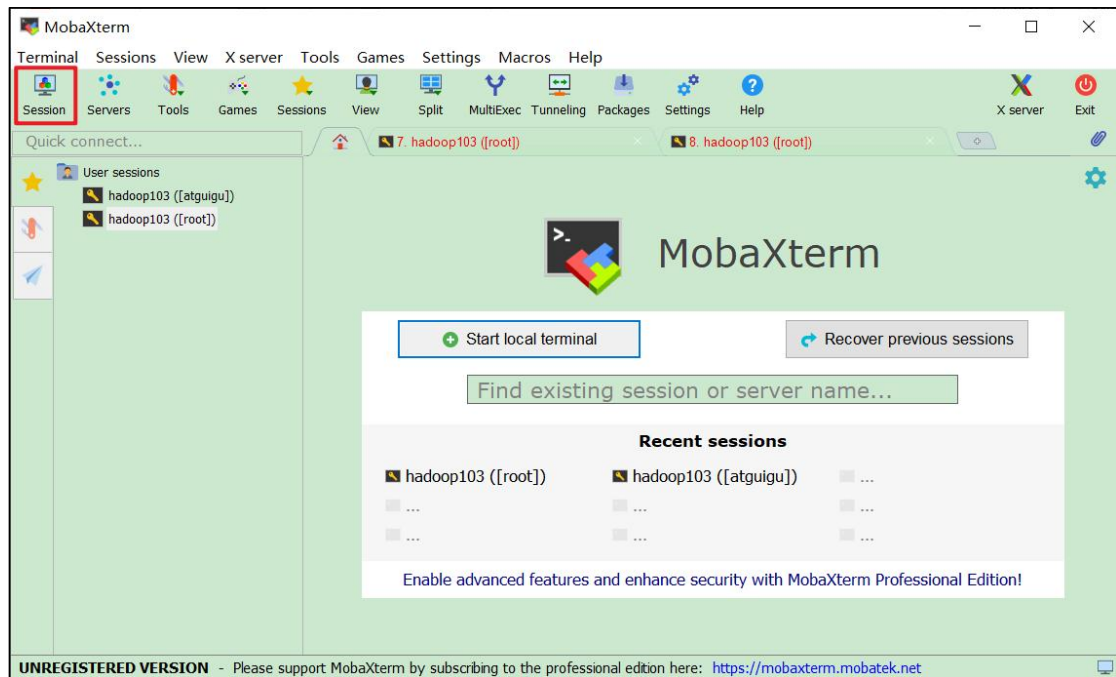
1) MobaXterm 配置

(1) 安装部署

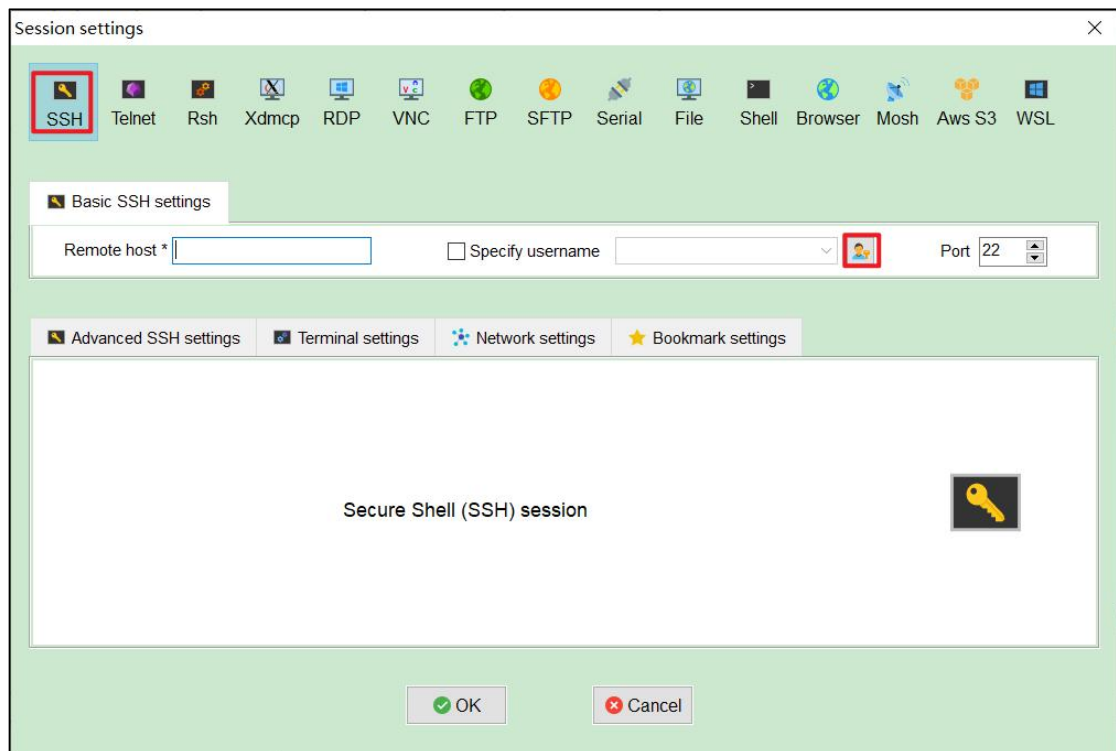
下载安装包，按提示操作即可。

(2) 配置用户证书

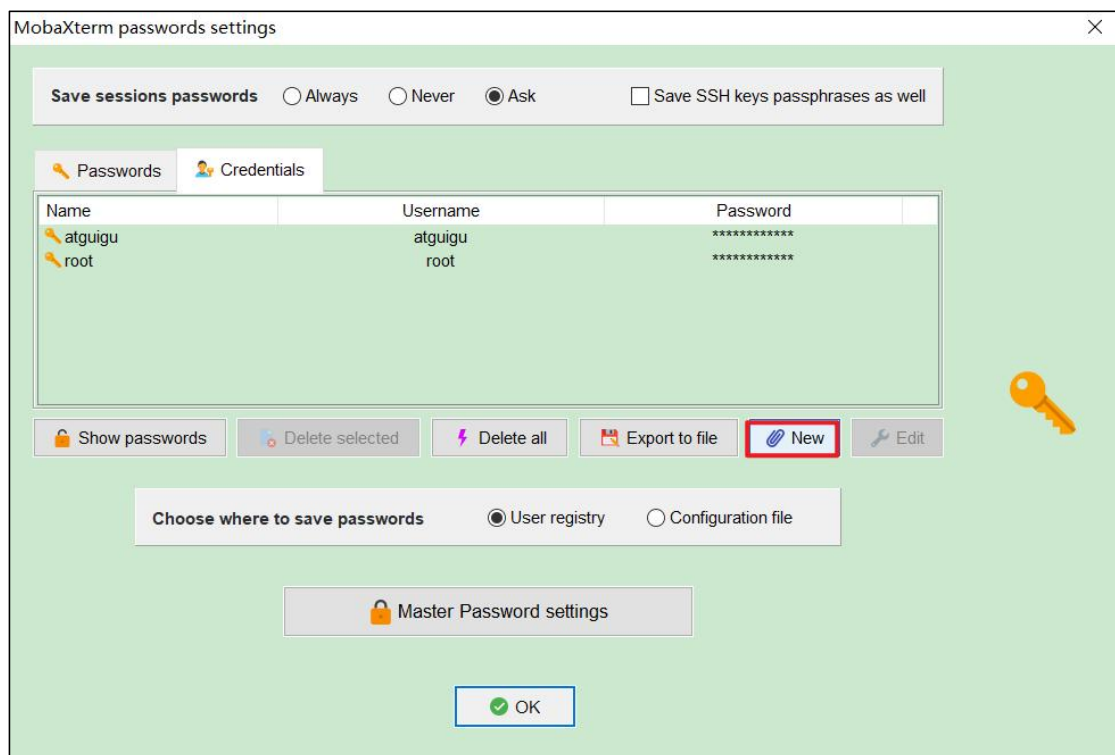
① 点击“Session”创建新会话



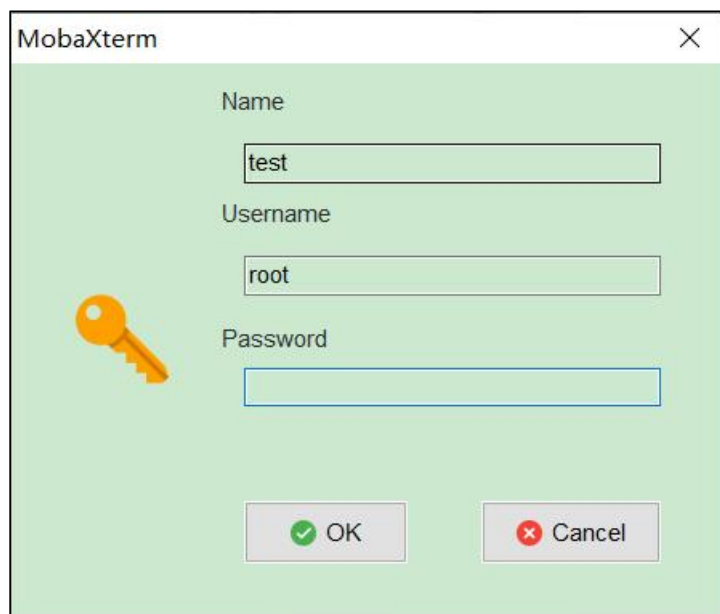
② 点击“SSH”，在下方弹出的对话框中点击“带钥匙的人像图标”



③ 点击“New”创建新的证书



④ 在弹出的对话框中填入信息

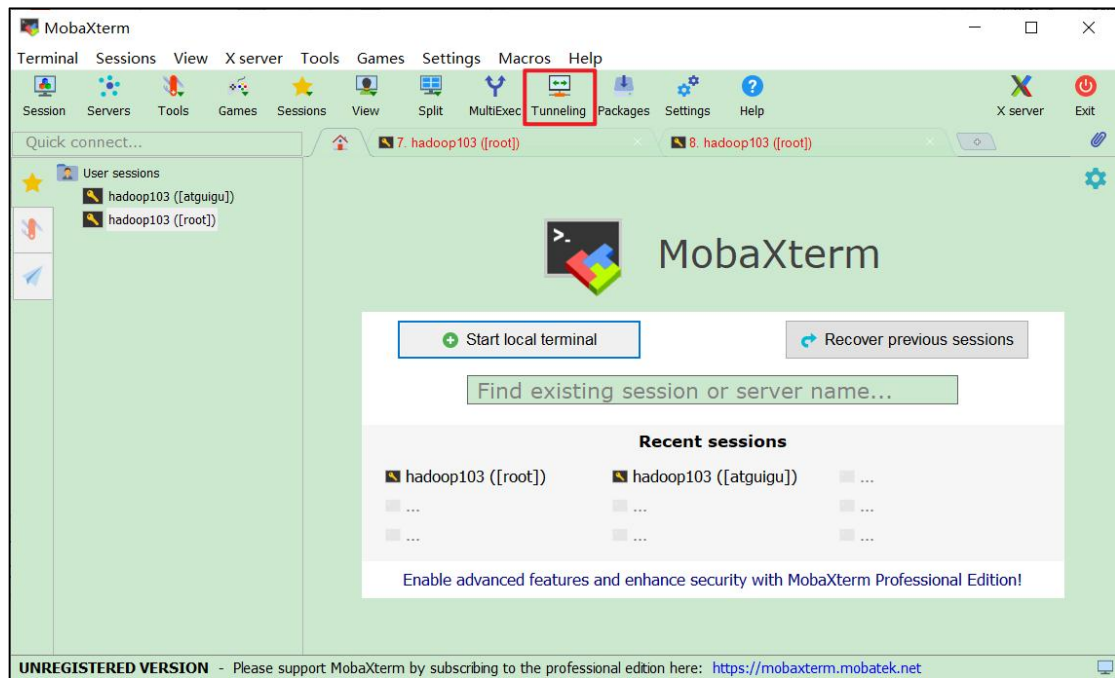


此处创建的用户证书用于连接 SSH 服务器，所以要输入 SSH 服务器的用户名密码。

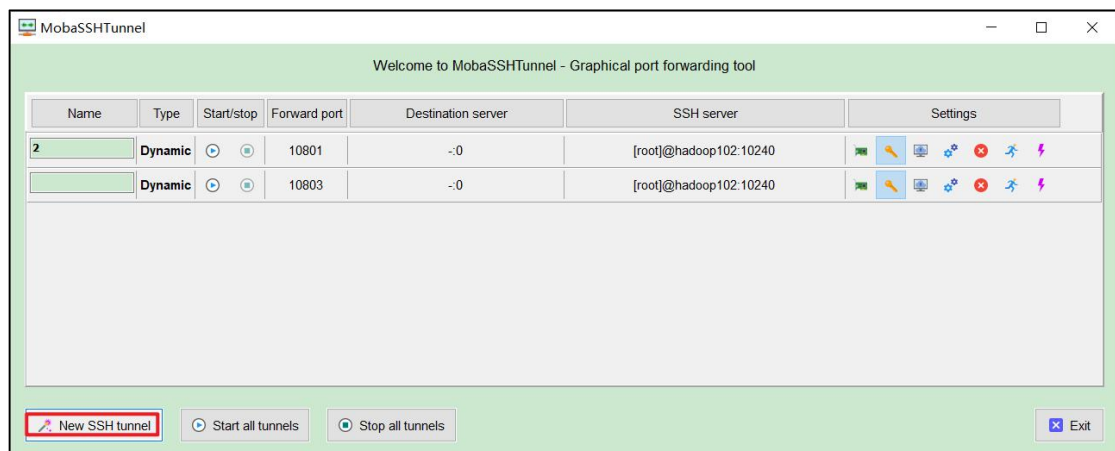
- **Name:** 用户证书名称。
- **Username:** SSH 服务器登陆用户名。
- **Password:** SSH 服务器登陆密码。

(3) 配置 SSH 服务器

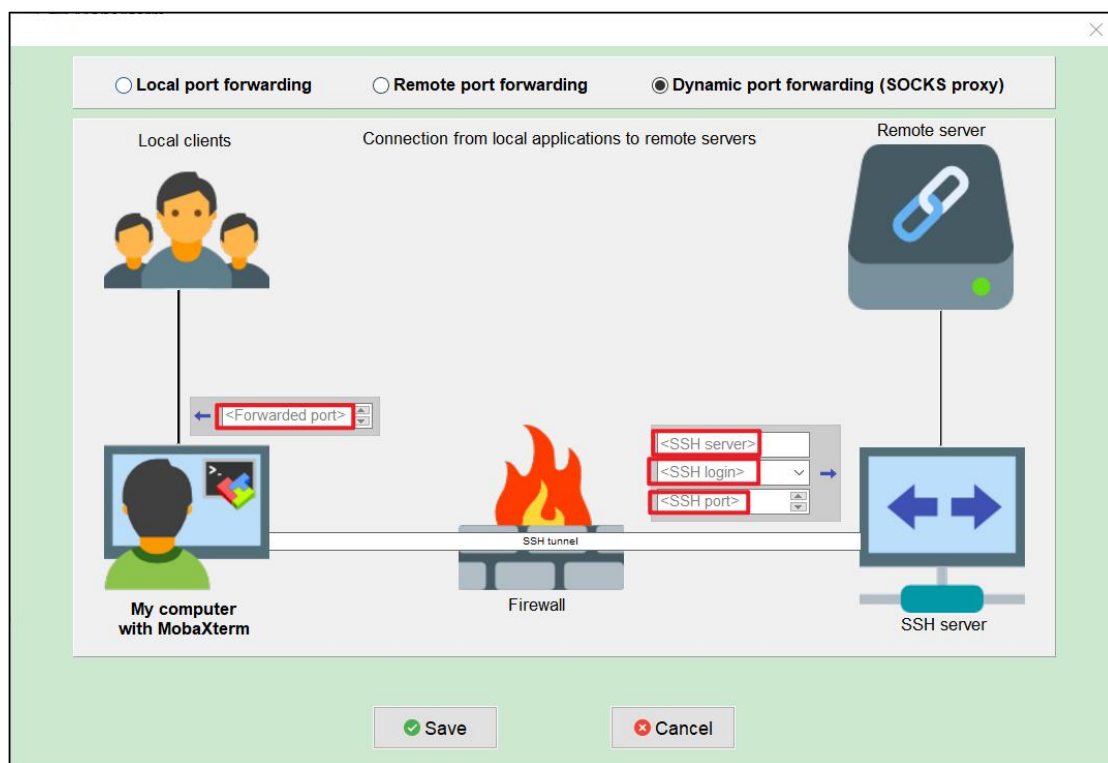
① 点击 “Tunneling”



② 点击 “New SSH Tunnel” 创建新的 SSH 连接通道

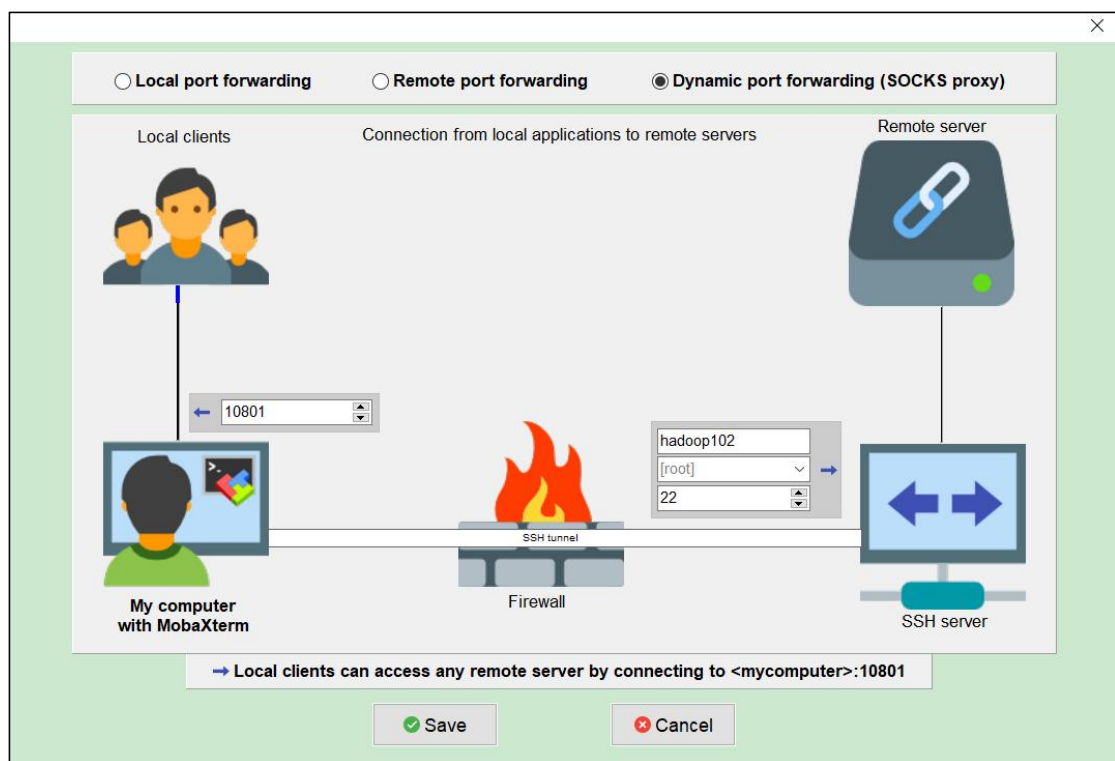


③ 在弹出的对话框中填入代理信息。



- Forwarded port: 需要代理的本地端口，此处应填入浏览器中配置的本地代理端口。
- SSH server: SSH 服务器 IP 或主机名。
- SSH login: SSH 服务器登陆证书，选择 2) 中配置的用户证书即可。
- SSH port: SSH 服务器请求端口（此处通过 SSH 方式连接代理服务器，默认连接端口为 22）。

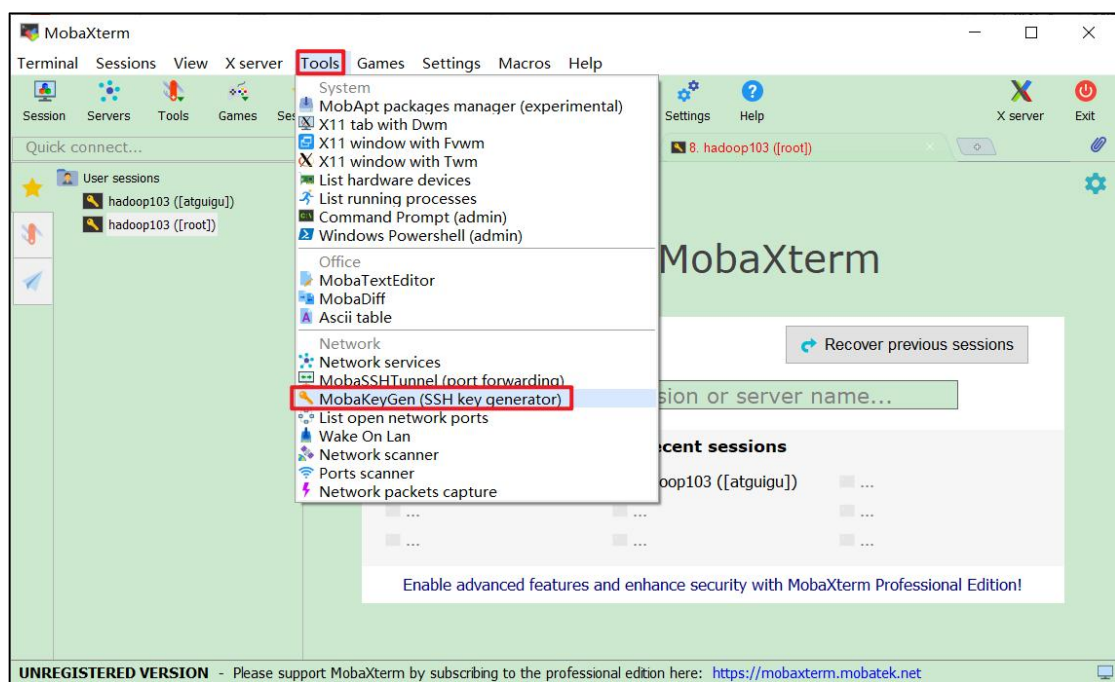
如下。



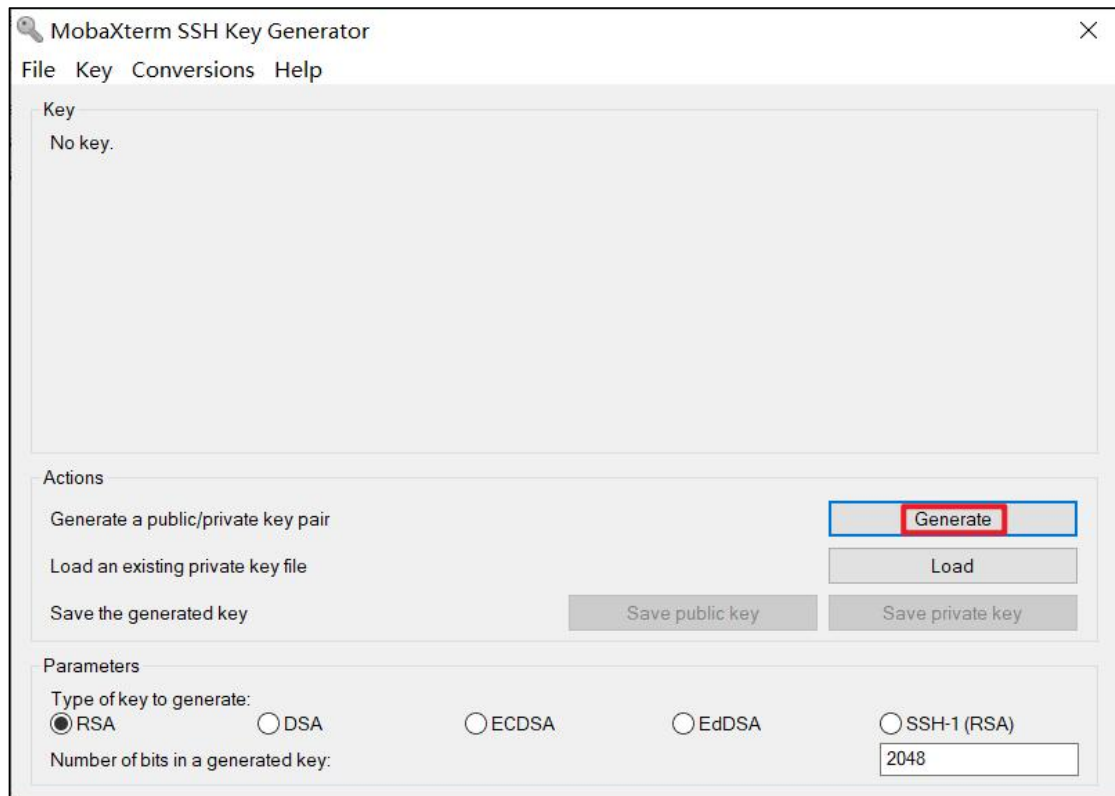
(4) 通过密钥连接代理服务器。

可以通过密码登陆 SSH 服务器，也可以通过密钥对的方式登陆服务器，若使用密钥登录则在创建用户证书时不需要填入密码。

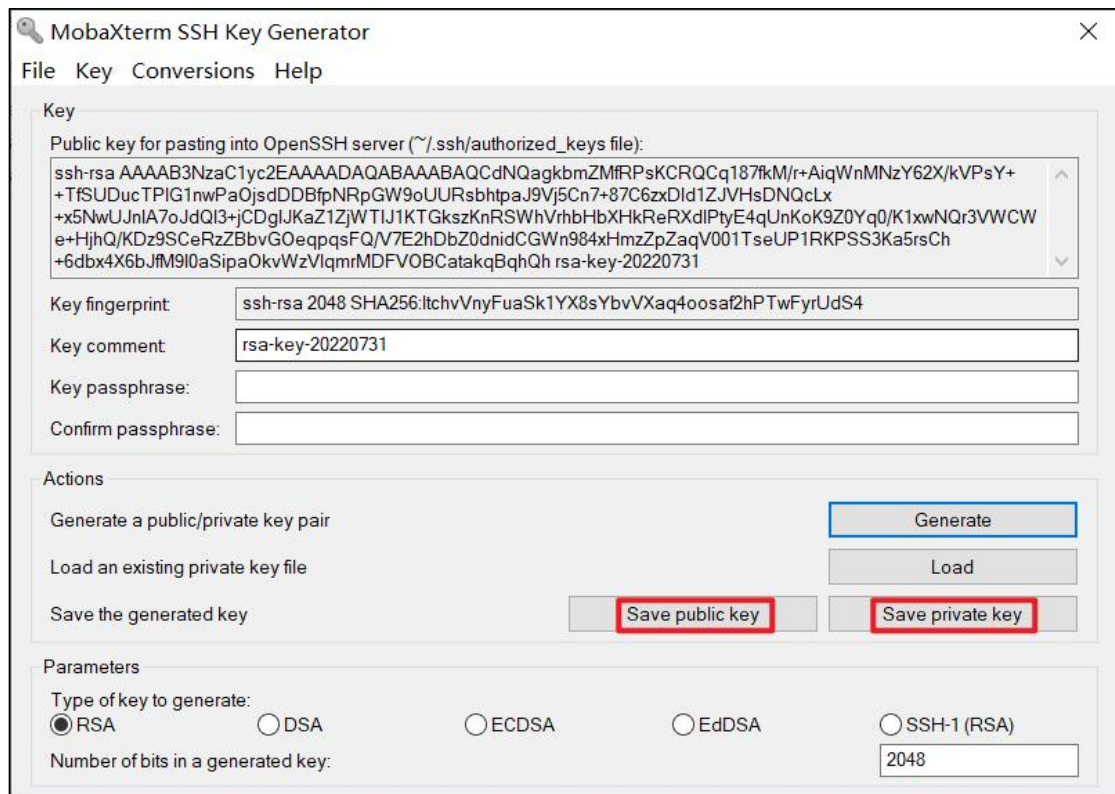
① 点击“Tools”下的“MobaKeyGen (SSH key generator)”。



② 点击“Generate”生成密钥对。



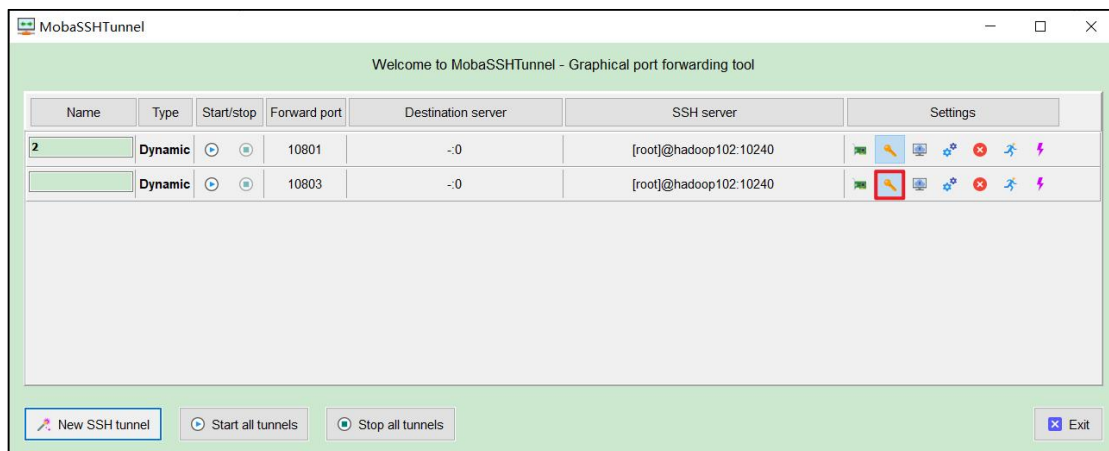
③ 保存公钥和私钥。



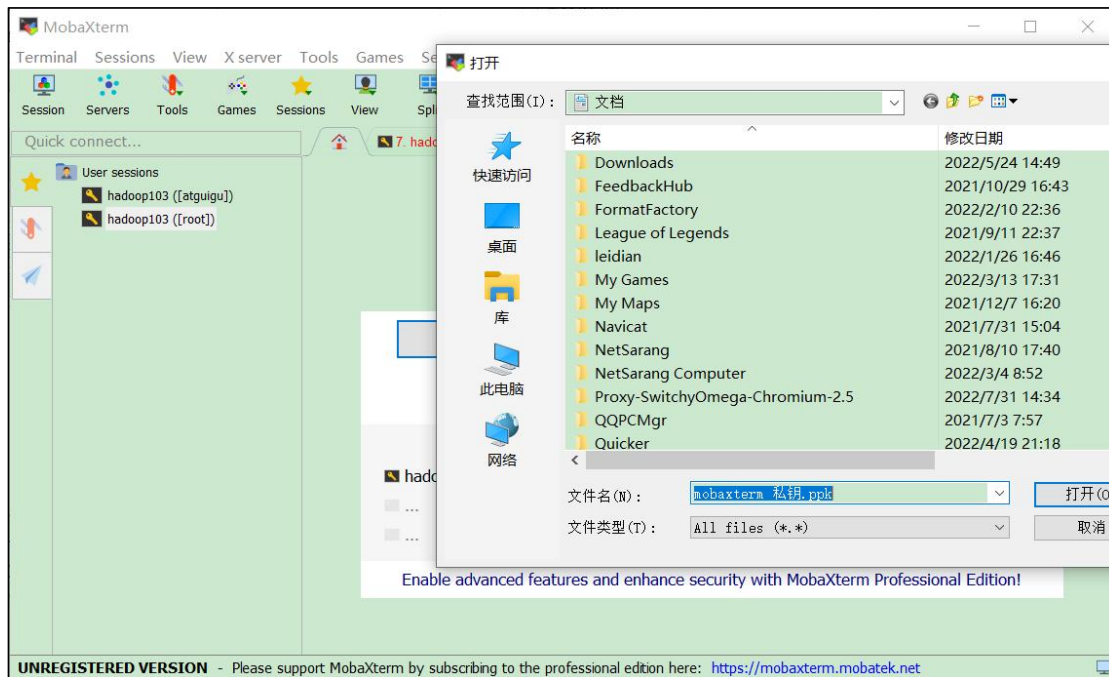
④ 将公钥追加到 SSH 代理服务器存储已认证公钥的文件中。

当前用户已认证的公钥默认存储在用户家目录下的.ssh目录下的authorized_keys文件中。

⑤ 点击“钥匙”图标



⑥ 在弹出的对话框中选择上文生成的私钥文件

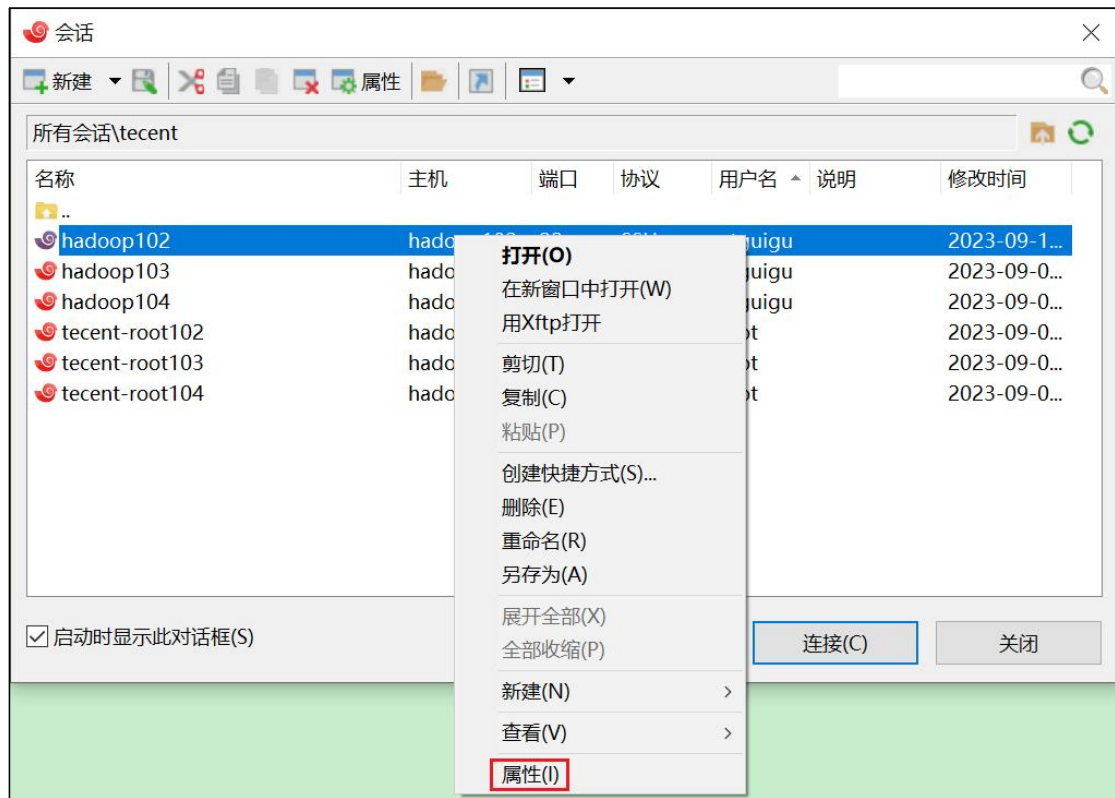


配置完成，点击 Start 图标即可启用隧道。

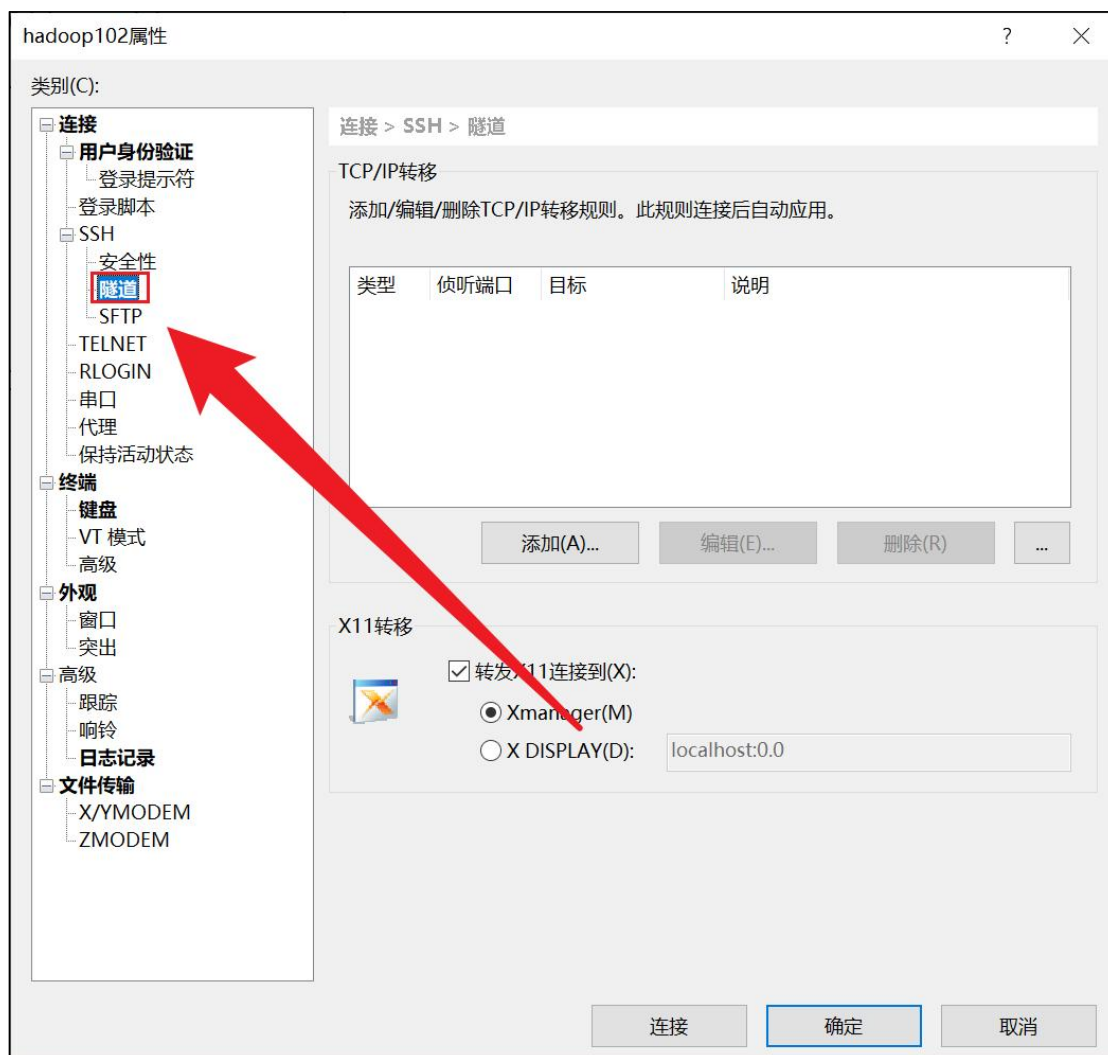
2) Xshell 配置

Xshell 同样提供了 SSH 隧道功能。

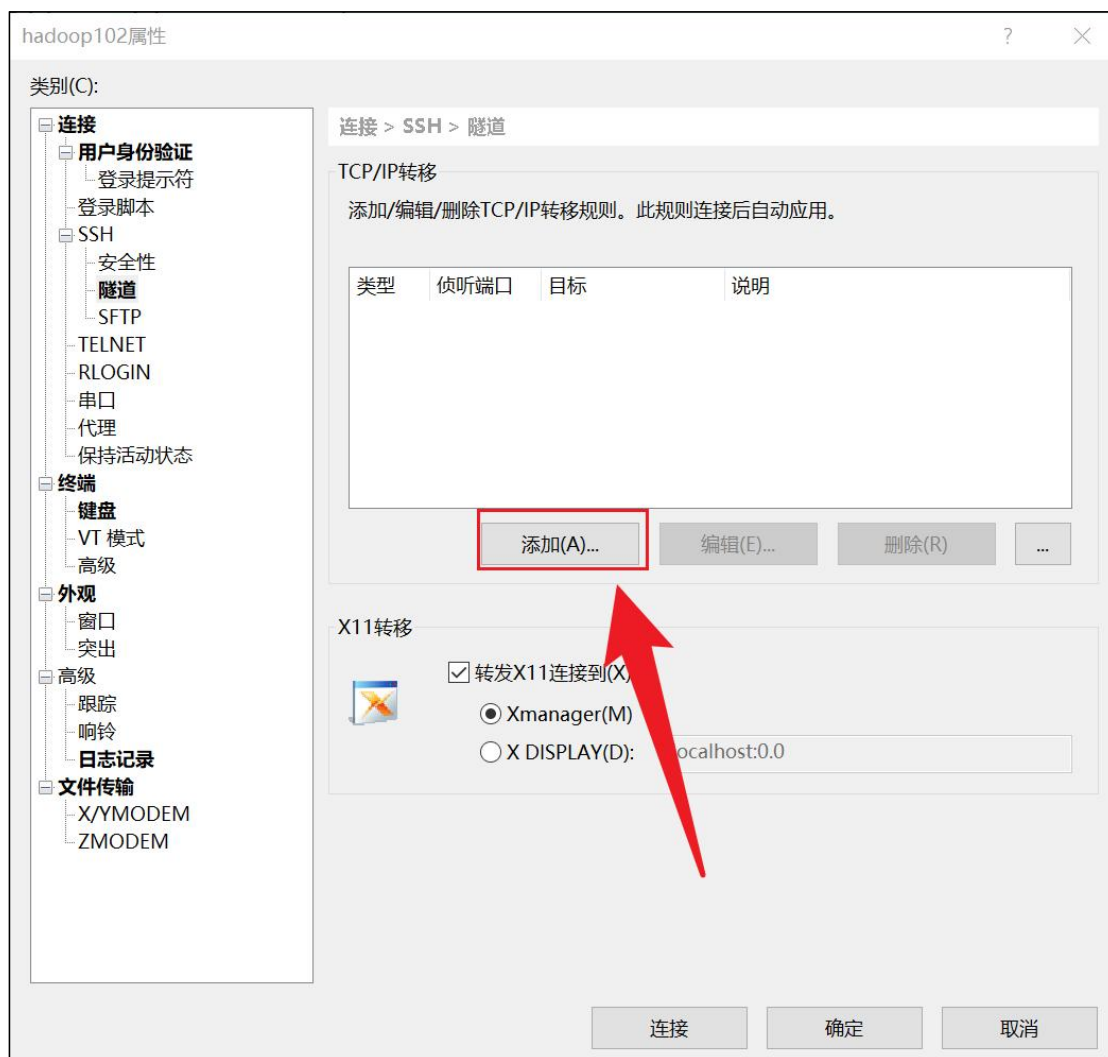
(1) 右键选项卡，点击“属性”



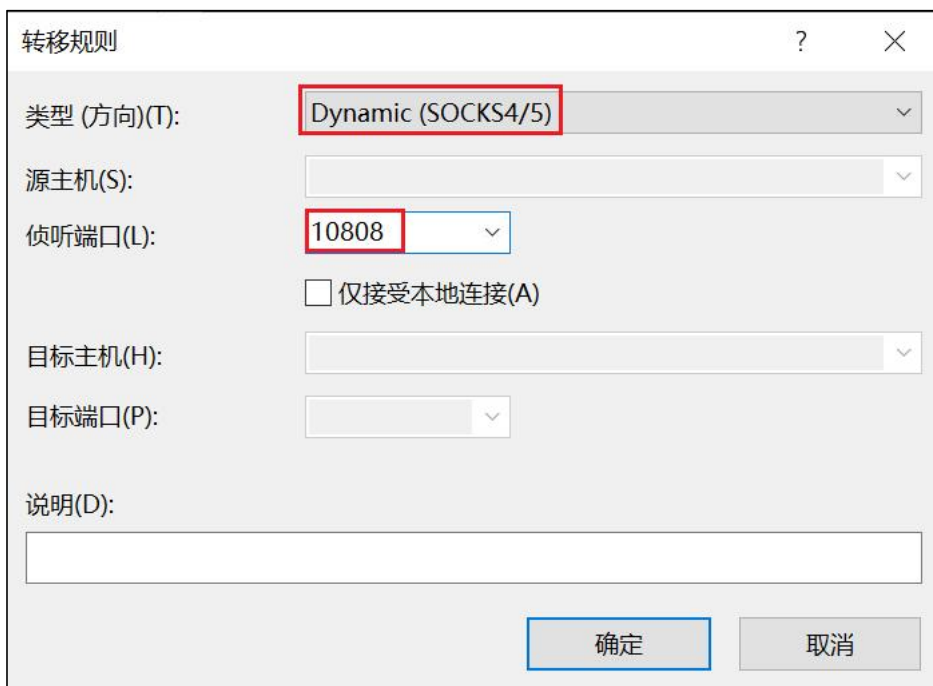
(2) 选中“隧道”



(3) 点击“添加”



(4) 在弹出的窗口中选择“类型”及“侦听端口”



转移规则

类型 (方向)(T): **Dynamic (SOCKS4/5)**

源主机(S):

侦听端口(L): **10808**

☐ 仅接受本地连接(A)

目标主机(H):

目标端口(P):

说明(D):

确定 **取消**

- 类型（方向）：隧道类型，Xshell 提供了三种选项，如下。
 - **Dynamic (SOCKS4/5)**：动态端口转发，会创建一个本地 SOCKS 代理服务器。这种方式不需要在客户端预先定义特定的端口转发规则。当不确定要访问的远程网络服务，或需要访问多个服务时，可以配置应用程序（例如浏览器或邮件客户端）使用本地 SOCKS 代理，通过动态隧道转发请求。所有的网络流量都会被安全地隧道化并通过 SSH 连接传输。
 - **本地（拨出）**：将本地机器某个端口的数据转发到远程服务器的特定端口。
 - **远程（传入）**：将远程服务器某个端口的数据转发到本地机器的特定端口。
- 侦听端口：根据不同类型填写源机器的侦听端口。

此处的场景是将浏览器的所有请求转发至本地的某个端口，然后再通过隧道将这些请求抓发到目标集群，显然，应选择 **Dynamic**，侦听端口与浏览器中 SwitchyOmega 或 FoxyProxy 转发的目标端口保持一致即可。

（5）“确定”即可

转移规则

?

×

类型 (方向)(T):

Dynamic (SOCKS4/5)

源主机(S):

侦听端口(L):

10808

☐ 仅接受本地连接(A)

目标主机(H):

目标端口(P):

说明(D):

确定

取消

hadoop102属性

?

×

类别(C):

连接

用户身份验证

登录提示符

登录脚本

SSH

安全性

隧道

SFTP

TELNET

RLOGIN

串口

代理

保持活动状态

终端

键盘

VT 模式

高级

外观

窗口

突出

高级

跟踪

响铃

日志记录

文件传输

X/YMODEM

ZMODEM

连接 > SSH > 隧道

TCP/IP 转移

添加/编辑/删除TCP/IP转移规则。此规则连接后自动应用。

类型	侦听端口	目标	说明
Dyna...	10808		

添加(A)...

编辑(E)...

删除(R)

...

X11 转移

☒ 转发X11连接到(X):

☒ Xmanager(M)

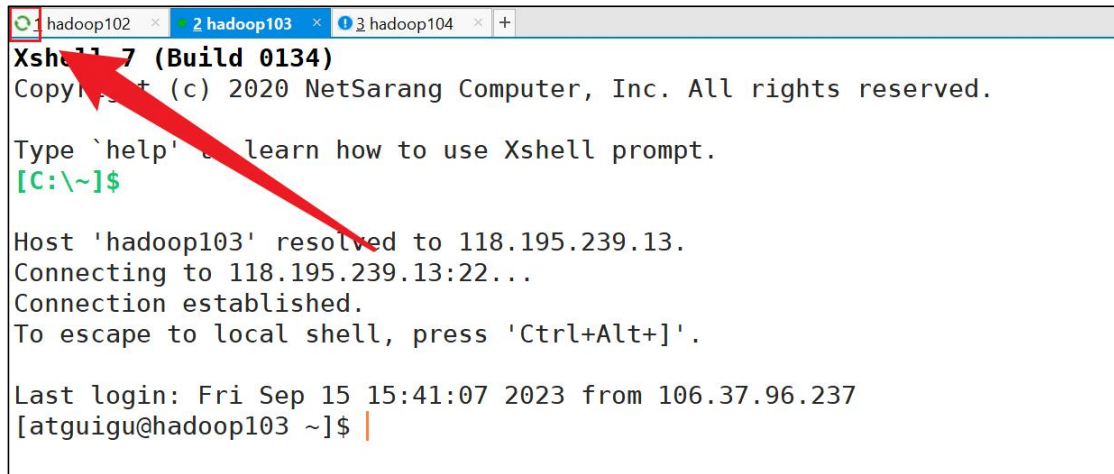
☐ X DISPLAY(D): localhost:0.0

连接

确定

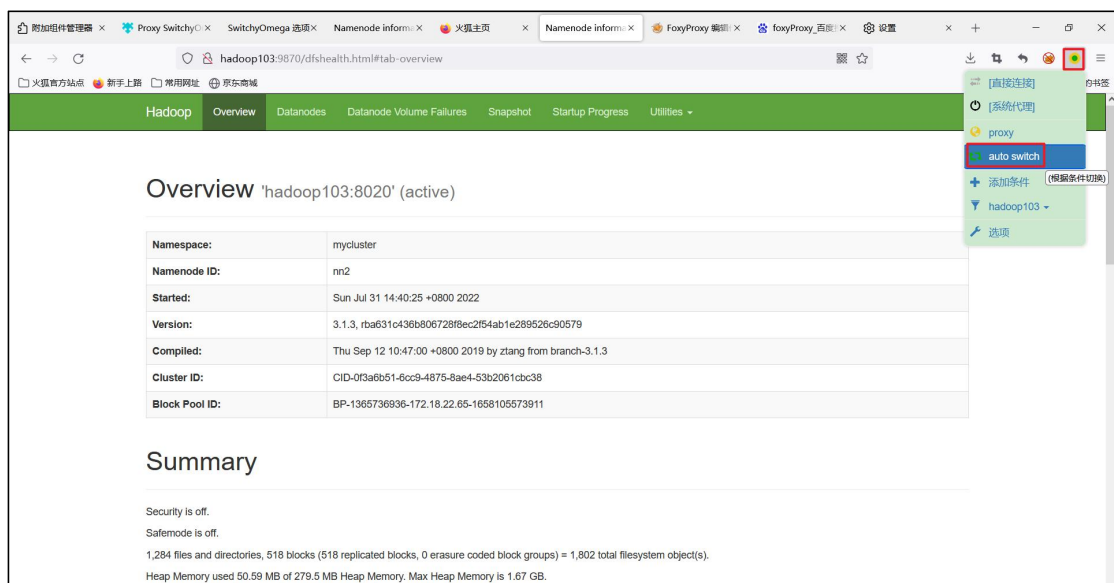
取消

(6) 当隧道生效时，终端标签会出现一个首尾相接的箭头

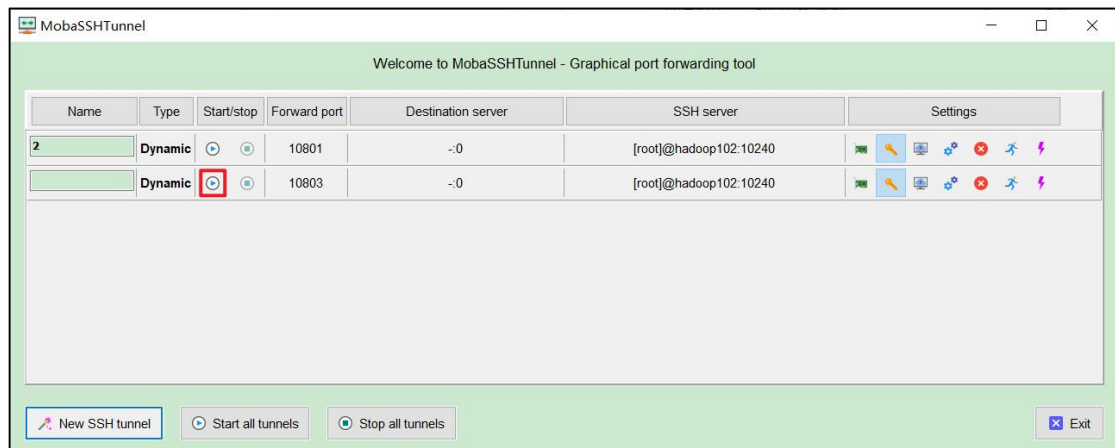


2.2.1.3 通道测试

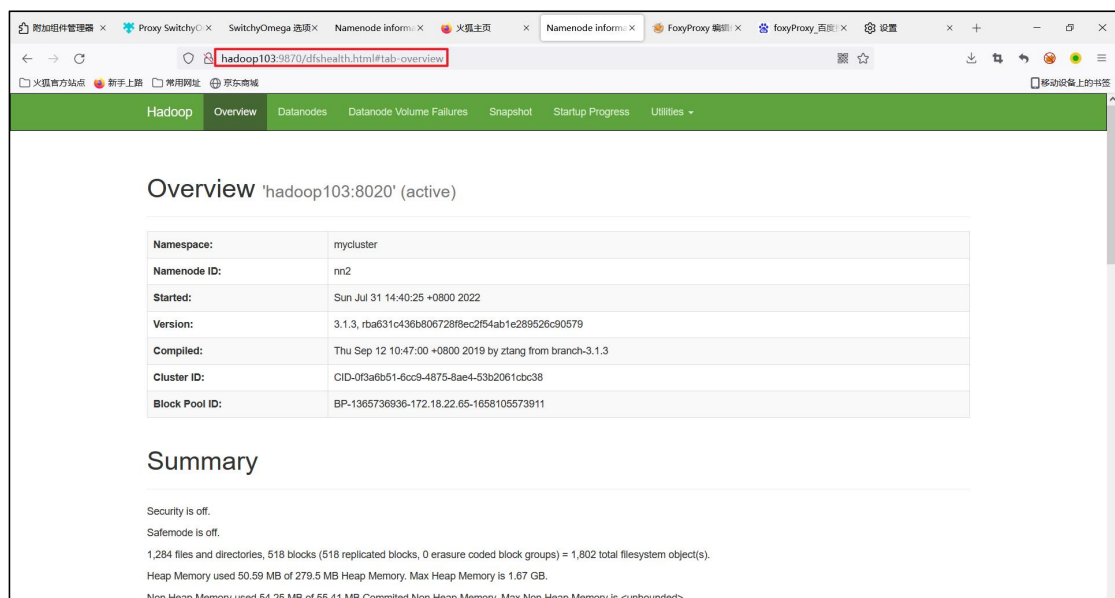
(1) 在浏览器中选择上文配置好的“auto switch”模式。



(2) 开启 MobaXterm 中的代理通道。

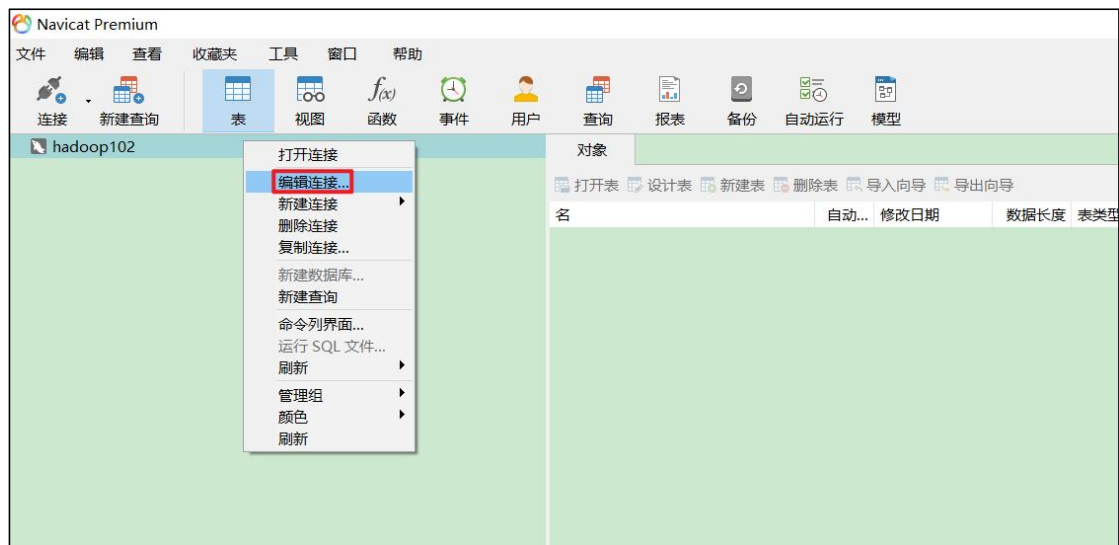


(3) 仅在腾讯云安全组出方向中保留开放 22 端口的规则，在服务器启动 HDFS，访问 HDFS WebUI，出现如下界面则通道已打通。



2.3 Navicat 连接 Mysql 代理配置

(1) 右键已有连接，选择“编辑连接”。



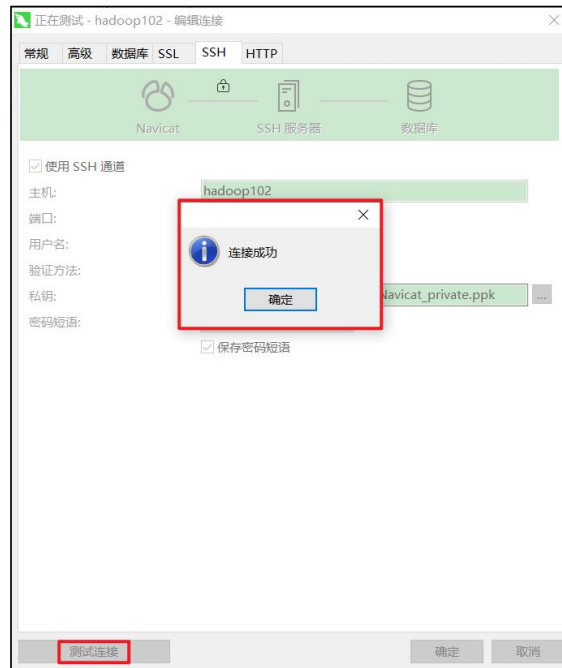
(2) 点击对话框上方的“SSH”，配置 SSH 代理服务器相关信息。



- 主机：Mysql 数据库所在节点主机名或 IP。
- 端口：代理服务器 SSH 服务端口，默认为 22。
- 用户名：代理服务器用户名。
- 验证方法：登录到 SSH 服务器的方式，可以选择密码，此处选择公钥。
- 私钥：用于认证登陆代理服务器的私钥文件。

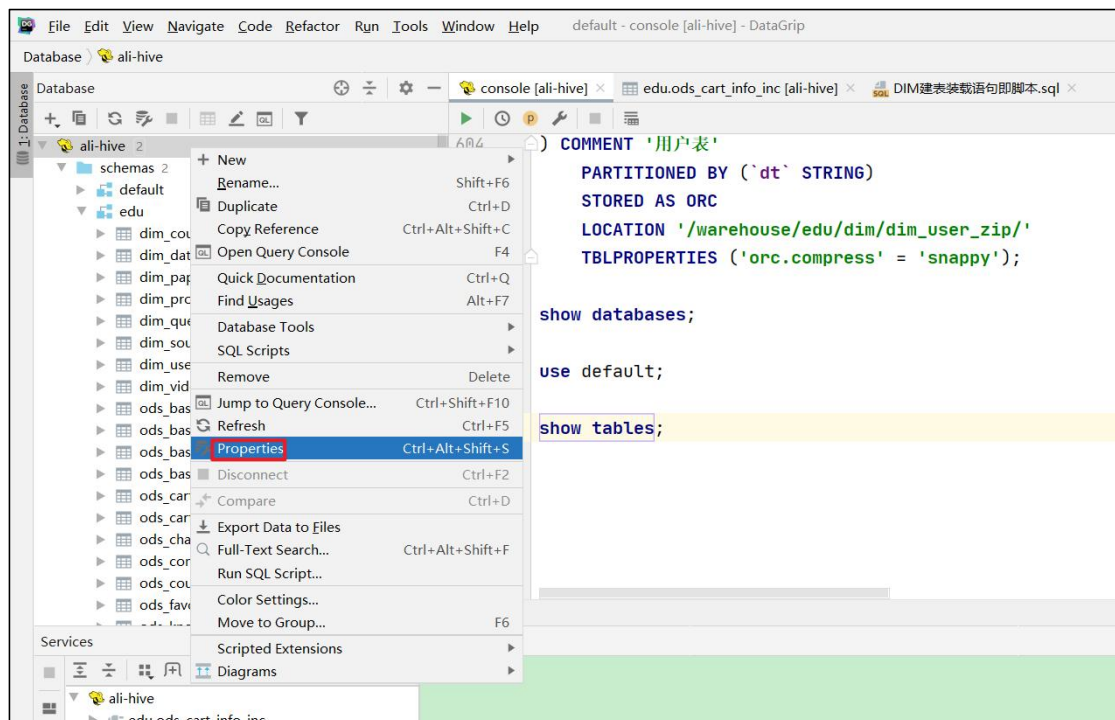
- 密码短语：SSH 服务器登陆密码。

(3) 点击“测试连接”，弹出如下对话框则通道已打通。

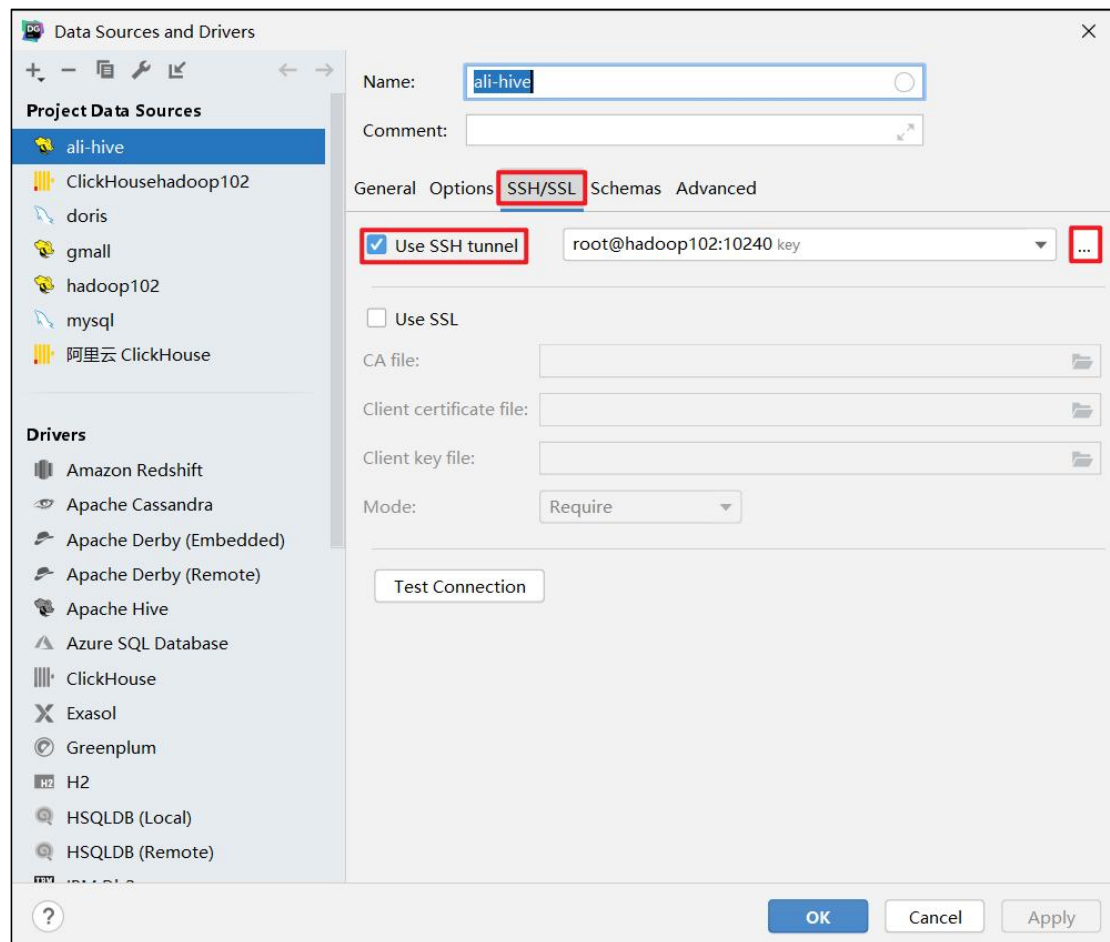


2.4 DataGrip 连接 Hiveserver2 代理配置

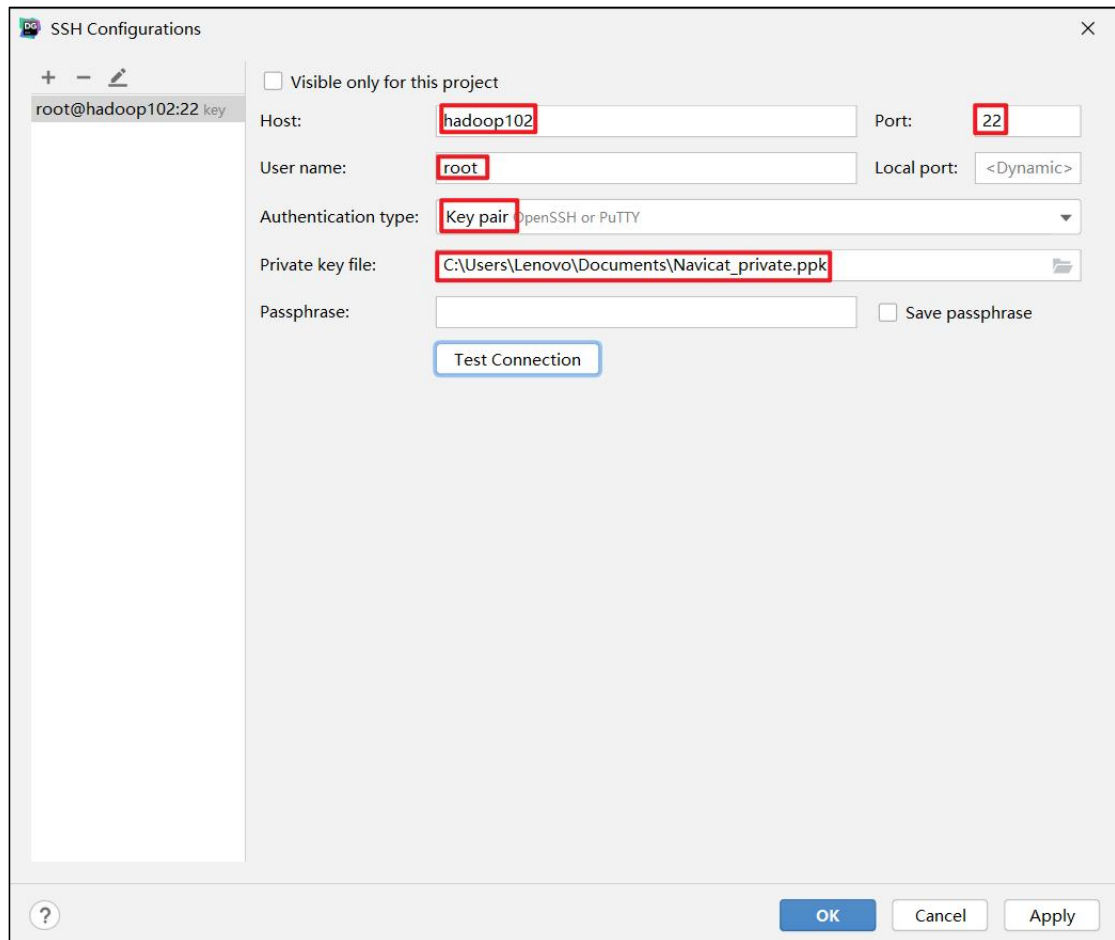
(1) 打开已有连接属性或新建连接



(2) 点击“SSH/SSL”。

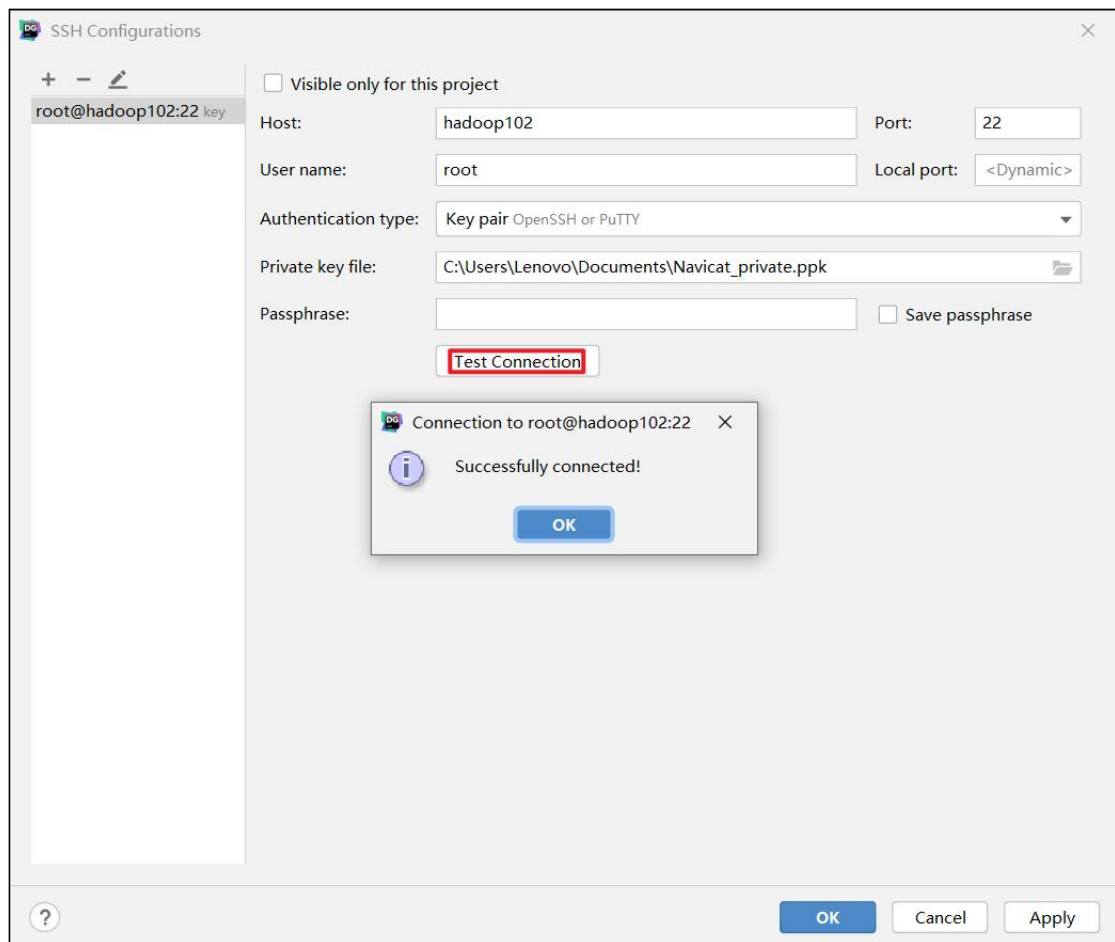


(3) 配置代理服务器。



- Host: 代理服务器主机名或 IP 地址。
- Port: 代理服务器 SSH 服务端口号，默认 22。
- User name: 代理服务器用户名。
- Authentication type: 认证方式，可以用密码，此处选择密钥对。
- Private key file: 私钥文件。

(4) 测试连接，出现如下对话框则通道已打通。



2.5 进阶配置

经过了上述配置，目前仍需开放 22 端口，被攻击的可能性仍然存在，我们可以通过某些方式进一步提升安全等级。

2.5.1 修改代理服务器 SSH 服务的默认端口

默认的 SSH 服务端口为 22，若将其修改为其它值，将会增大黑客攻击的难度。步骤如下。

1) 打开 SSH 服务配置文件 sshd_config

```
[root@hadoop102 .ssh]# vim /etc/ssh/sshd_config
```

2) 打开“Port 22”的注释，修改端口号

注意：端口号的理论值在 0 到 65535 之间，但 0~1023 通常被系统占用，49152~65535 通常为动态端口，我们选择的端口号尽量位于 1024 到 49152 之间，还要避免与 Mysql(3306) 等服务的端口冲突。

```
# 修改前
#Port 22
# 修改后
```


Port 10240

保存退出。

3) 重启 SSH 服务

```
[root@hadoop102 .ssh]# systemctl restart sshd
```

4) 测试

```
[root@hadoop103 ~]# ssh hadoop102
ssh: connect to host hadoop102 port 22: Connection refused
```

指定端口号登录。

```
[root@hadoop103 ~]# ssh hadoop102 -p 10240
Last login: Sun Jul 31 19:09:54 2022 from 172.17.156.165

Welcome to Alibaba Cloud Elastic Compute Service !

[root@hadoop102 ~]#
```

通过集群内的其它节点远程登录 hadoop102，如果答疑上述日志则配置成功。

2.5.2 通过密钥对远程连接并禁用密码登录

2.5.2.1 配置通过密钥对远程登录

以 XShell 为例，MobaXterm 配置同理。

1) 点击“生成密钥对”



2) 指定密钥类型和长度



3) 输入秘钥名称和密码



新建用户密钥生成向导

用户密钥信息
请输入生成用户密钥的名称和密码。

请输入用户密钥的名称。
密钥名称(K):

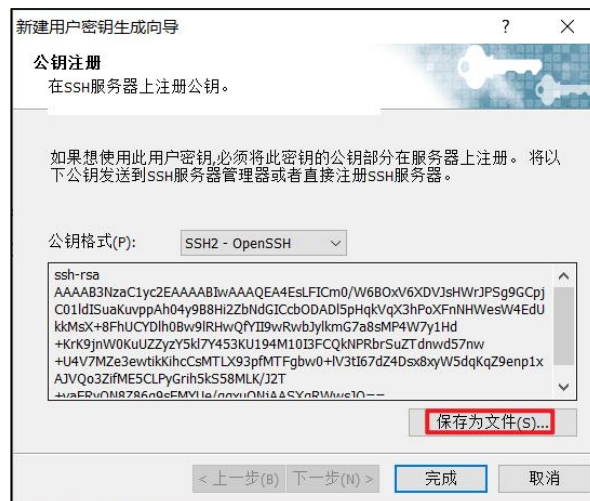
请输入给用户密钥加密的密码。
密码(P):
确认(C): (重新键入密码)

请单击"下一步"在SSH服务器上注册公钥。

< 上一步(B) 下一步(N) > 完成 取消

此处的名称和密码都是提供给 XShell 用于管理密钥对的。

4) 保存公钥



新建用户密钥生成向导

公钥注册
在SSH服务器上注册公钥。

如果想使用此用户密钥,必须将此密钥的公钥部分在服务器上注册。将以下公钥发送到SSH服务器管理器或者直接注册SSH服务器。

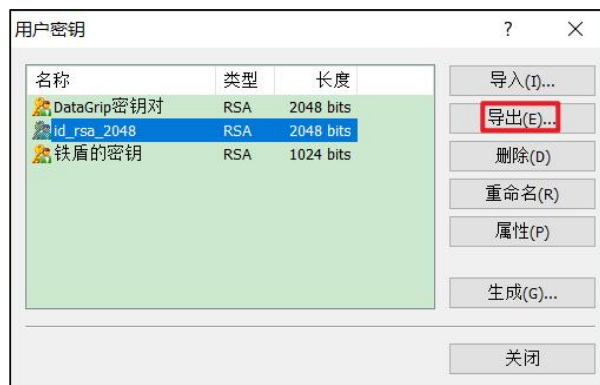
公钥格式(P):

```
ssh-rsa
AAAAB3NzaC1yc2EAAAABIwAAAQEA4EsLFICm0/W6BOxV6XDVJsHWrJPSg9GCpj
C01ldISuaKuvppAh04y988Hi2ZbNdGICbODADl5pHqVqX3hPoXFnNHWesW4EdU
kkMsX+8FhUCYDlh08w9IRHwQfYII9wRwbJyIkmg7a8sMP4W7y1Hd
+Krk9JnW0KuUZZyZy5kl7Y453KU194M10I3FCQkNPRBrSuZT dnwd57mw
+U4V7MZe3ewtikKihcCsMTLX93pMTFgbw0+IV3tI67DZ4Ds8xyW5dqKqZ9enp1x
AJVQo3ZifME5CLPyGnih5K58MLK/J2T
+raEPvQNI8786n9cEMVlIe/nnyuQNI8&CYnRMueTQ--
```

保存为文件(S)...

< 上一步(B) 下一步(N) > 完成 取消

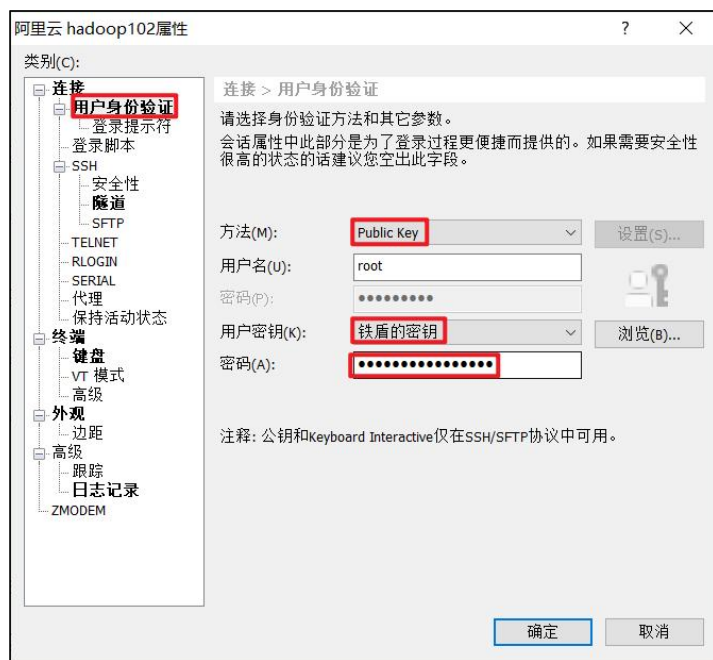
5) 保存私钥



名称	类型	长度
DataGrip密钥对	RSA	2048 bits
id_rsa_2048	RSA	2048 bits
铁盾的密钥	RSA	1024 bits

导入(I)...
导出(E)...
删除(D)
重命名(R)
属性(P)
生成(G)...
关闭

6) 修改连接属性



用户身份验证方法更改为 Public Key，此处选择的用户密钥名称、密码在前文生成密钥时指定。

2.5.2.2 禁用密码登录

1) 打开 sshd_config 文件

```
[root@hadoop104 ~]# vim /etc/ssh/sshd_config
```

2) 修改如下配置

```
# 公钥认证文件存储位置，保持默认即可
AuthorizedKeysFile .ssh/authorized_keys
# 是否可以通过密钥对登录，默认即开启，无需修改
PubkeyAuthentication yes
# 是否允许使用密码登录，修改为 no
PasswordAuthentication no
```

3) 重启服务

```
[root@hadoop104 ~]# systemctl restart sshd
```

如此，除非本机私钥文件泄露或黑客破解 SSH 协议（几乎不可能），集群就是安全的。

第 3 章 实时实战代理访问方式

与离线不同，实时项目会在代码中向远端集群发送请求，因此代理方式也有所不同。本章将通过 Xshell 提供的本地播出隧道实现代理访问，为每种请求绑定一个本地端口。

3.1.1 访问 MySQL

(1) 隧道配置如下

转移规则

?

×

类型 (方向)(T):

本地 (拨出)

源主机(S):

localhost

侦听端口(L):

33060

☐ 仅接受本地连接(A)

目标主机(H):

hadoop102

目标端口(P):

3306

说明(D):

确定

取消

(2) 访问 MySQL 实例代码如下

```
package com.atguigu.source.app;

import java.sql.*;

public class MySQLTest {
    public static void main(String[] args) throws
        ClassNotFoundException, SQLException, InterruptedException {
        // 1. 注册驱动
        Class.forName("com.mysql.cj.jdbc.Driver");

        // 2. 获取连接对象
        String url = "jdbc:mysql://localhost:33060/gmall";
        String user = "root";
        String passwd = "000000";
        Connection connection = DriverManager.getConnection(url, user,
passwd);

        // 3. 获取数据库操作对象 (预编译)
        PreparedStatement preparedStatement =
connection.prepareStatement("show databases");

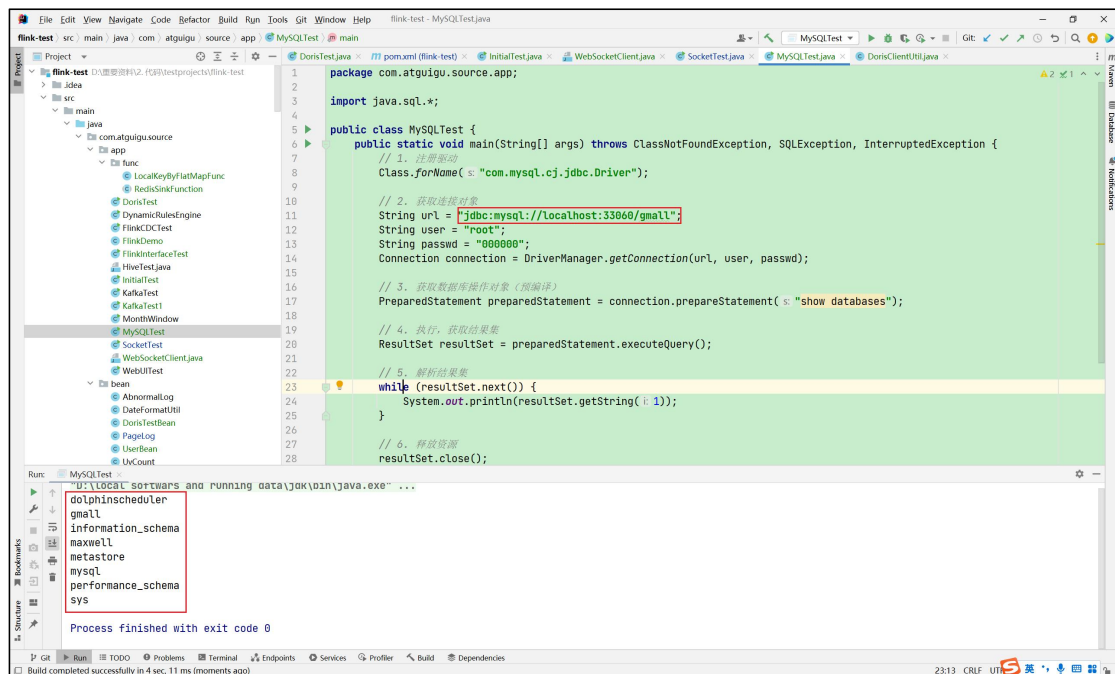
        // 4. 执行, 获取结果集
        ResultSet resultSet = preparedStatement.executeQuery();

        // 5. 解析结果集
        while (resultSet.next()) {
            System.out.println(resultSet.getString(1));
        }

        // 6. 释放资源
        resultSet.close();
        preparedStatement.close();
    }
}
```

```
        connection.close();
    }
}
```

(3) 执行结果如下



3.1.2 访问 Kafka

通过代理的方式连接 Kafka 集群完成读写操作, 首先要理解 Kafka 的读写机制及该过程中涉及的通信交互行为。

3.1.2.1 Kafka 消费流程

Kafka-3.3.1 传统集群 (非 KRaft 集群) 的大致消费流程如下。

1) 发现 Broker

客户端使用预先配置的 Broker 列表 (bootstrap.servers 配置项指定) 来发现 Kafka 集群, 与任意一台正常工作的 Broker 建立连接。

2) 获取集群元数据

客户端连接到 Broker 后, 该 Broker 会从 ZooKeeper 中获取 Kafka 集群的元数据, 如当前正常工作的 brokers、topics、partitions 及其相应的 leader。

3) 加入消费者组并获取分区分配

消费者与 Kafka 中的组协调器交互来加入消费者组。组协调器负责选出消费者的 Leader, 并将当前消费者列表和所有的 topic-partition 发送给 Leader, 后者执行分区分配策略, 决定

哪个消费者消费哪个分区，再把分配结果发送给组协调器。最后由组协调器向组内的所有消费者发送 SyncGroup 响应，告诉它们分区分配情况。

4) 建立连接到分区的 Leader

消费者从 Zookeeper 获取元数据来确定每个分区的 Leader 所在的 Broker，并直接与这些 Leader Brokers 建立连接。

5) 开始消费

消费者从分区 Leader 拉取数据。

6) 心跳和维持连接

消费者定期向 Kafka 的组协调器发送心跳来维持其在消费者组中的成员资格。

7) 提交偏移量

消费者向 Kafka 的 __consumer_offsets 主题提交偏移量。

3.1.2.2 Kafka 生产流程

Kafka-3.3.1 传统集群（非 KRaft 集群）的大致生产流程如下。

1) 发现 Broker

生产者使用预先配置的 bootstrap broker 列表来发现 Kafka 集群。

2) 获取集群元数据

生产者首次连接到 bootstrap broker 后，该 broker 会从 ZooKeeper 中获取关于 Kafka 集群的元数据，如当前存在的 brokers、topics、partitions 及其相应的 leader。

3) 确定分区

基于分区策略，生产者确定将消息发送到哪个分区。

4) 建立连接到分区的 Leader

生产者使用从 ZooKeeper 通过 broker 获取的元数据来确定目标分区的 leader broker，并直接与该 leader broker 建立连接。

5) 发送消息

生产者直接将消息发送到目标分区的 leader broker。

6) 接收确认

根据生产者配置的 acks，生产者可能会等待 broker 的确认。

3.1.2.3 Kafka 集群内外网分流

某些场景下，Kafka 集群的所有节点处于同一内网环境，而客户端不在该局域网下。此时，客户端请求集群只能通过外网 IP 实现，而集群内部节点间通信应通过内网，此时就需要内外网分流。

1) 知识储备

在开始之前，我们先要了解 Kafka 的几个相关参数。

(1) listener.security.protocol.map

Kafka 监听器名称和安全协议的映射。此处的安全协议用于 Broker 间通信，Kafka 支持的协议有四种：PLAINTEXT, SSL, SASL_PLAINTEXT, SASL_SSL。默认值：

PLAINTEXT:PLAINTEXT,SSL:SSL,SASL_PLAINTEXT:SASL_PLAINTEXT,SASL_SSL:SASL_SSL。

(2) listeners

Kafka 集群监听的 URI 及监听器名称，多个监听器之间通过逗号分割。如果监听器名称不是 Kafka 支持的安全协议，就需要通过 listener.security.protocol.map 指定监听器名称。如果没有配置，则使用 PLAINTEXT 协议，主机名为 `java.net.InetAddress.getCanonicalHostName()` 的返回值，端口为 9092。

(3) advertised.listeners

发布到 Zookeeper 用于客户端通信的监听器。如果没有配置，则会取 listeners 的值。

(4) inter.broker.listener.name

Broker 之间的通信所使用的监听器名称，若此项未设置，则 Broker 之间通信所用的监听器名称将由 `security.inter.broker.protocol` 结合其它配置项确定。默认值为 null，如果同时指定此项与 `security.inter.broker.protocol` 将会导致错误。

(5) security.inter.broker.protocol

Broker 之间通信所用的安全协议，支持的协议列表见上文。默认值为 PLAINTEXT。

2) 实现腾讯云服务器 Kafka 集群内外网分流

(1) Kafka 集群概况

集群共有三个节点：hadoop102, hadoop103, hadoop104。

三台机器的内网 IP 如下

```
10.206.16.6 hadoop102
10.206.16.17 hadoop103
```



```
10.206.0.16 hadoop104
```

三台机器的外网 IP 如下

```
175.27.230.177 hadoop102
43.137.5.10 hadoop103
146.56.230.220 hadoop104
```

三台机器的外网 IP 都是通过转发定义的，服务器并没有实际的外网网卡，在三台机器分别执行 ifconfig，结果如下。

```
[atguigu@hadoop102 zookeeper-3.7.1]$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.206.16.6 netmask 255.255.240.0 broadcast
10.206.31.255
    inet6 fe80::5054:ff:fe78:39d0 prefixlen 64 scopeid
0x20<link>
    ether 52:54:00:78:39:d0 txqueuelen 1000 (Ethernet)
    RX packets 161263 bytes 67831277 (64.6 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 113259 bytes 15462359 (14.7 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 5765 bytes 599692 (585.6 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 5765 bytes 599692 (585.6 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

[atguigu@hadoop103 config]$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.206.16.17 netmask 255.255.240.0 broadcast
10.206.31.255
    inet6 fe80::5054:ff:feae:2fd6 prefixlen 64 scopeid
0x20<link>
    ether 52:54:00:ae:2f:d6 txqueuelen 1000 (Ethernet)
    RX packets 270192 bytes 220947615 (210.7 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 185535 bytes 22287548 (21.2 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 1133 bytes 193868 (189.3 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1133 bytes 193868 (189.3 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

[atguigu@hadoop104 config]$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.206.0.16 netmask 255.255.240.0 broadcast
10.206.15.255
```

```

        inet6    fe80::5054:ff:fef5:2e63    prefixlen    64    scopeid
0x20<link>
        ether 52:54:00:f5:2e:63    txqueuelen 1000    (Ethernet)
        RX packets 271721    bytes 253955014    (242.1 MiB)
        RX errors 0    dropped 0    overruns 0    frame 0
        TX packets 124610    bytes 15140712    (14.4 MiB)
        TX errors 0    dropped 0    overruns 0    carrier 0    collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>    mtu 65536
    inet 127.0.0.1    netmask 255.0.0.0
    inet6 ::1    prefixlen 128    scopeid 0x10<host>
    loop txqueuelen 1000    (Local Loopback)
    RX packets 192    bytes 27854    (27.2 KiB)
    RX errors 0    dropped 0    overruns 0    frame 0
    TX packets 192    bytes 27854    (27.2 KiB)
    TX errors 0    dropped 0    overruns 0    carrier 0    collisions 0

```

eth 开头的表示网卡，lo 表示回环网卡（完全由软件实现，是一种特殊的网络接口，不与任何实际设备连接）。三台服务器都只有一张网卡 eth0。

（2）配置

① hadoop102

```

[atguigu@hadoop102 kafka-3.3.1]$ cd $KAFKA_HOME
[atguigu@hadoop102 kafka-3.3.1]$ vim config/server.properties

```

添加以下内容

```

# 定义监听器和协议的映射关系
listener.security.protocol.map=INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT

# 定义监听器，INTERNAL 监听本机的 9092 端口，EXTERNAL 监听本机的 19092 端口。
只有一张网卡，所以此处都是的 IP 均为内网 IP
listeners=INTERNAL://10.206.16.6:9092,EXTERNAL://10.206.16.6:19092

# 定义 Broker 发布到 Zookeeper 的 URI，用于客户端通信，INTERNAL 为内网 IP:9092，EXTERNAL 为外网 IP:19092。INTERNAL 用于内部通信，使用内网 IP，EXTERNAL 用于外网客户端访问，使用外网 IP
advertised.listeners=INTERNAL://10.206.16.6:9092,EXTERNAL://175.27.230.177:19092

# 声明 Broker 通信使用 INTERNAL 监听器，即集群内部通信使用内网 IP，走内网流量
inter.broker.listener.name=INTERNAL

```

② hadoop103

```

[atguigu@hadoop103 kafka-3.3.1]$ cd $KAFKA_HOME
[atguigu@hadoop103 kafka-3.3.1]$ vim config/server.properties

```

添加以下内容

```

listener.security.protocol.map=INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT
listeners=INTERNAL://10.206.16.17:9092,EXTERNAL://10.206.16.17:19092
advertised.listeners=INTERNAL://10.206.16.17:9092,EXTERNAL://43.137.5.10:19092
inter.broker.listener.name=INTERNAL

```

③ hadoop104

更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：[尚硅谷官网](http://www.shang硅谷.com)

```
[atguigu@hadoop104 kafka-3.3.1]$ cd $KAFKA_HOME
[atguigu@hadoop104 kafka-3.3.1]$ vim config/server.properties
```

添加以下内容

```
listener.security.protocol.map=INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT
listeners=INTERNAL://10.206.0.16:9092,EXTERNAL://10.206.0.16:19092
advertised.listeners=INTERNAL://10.206.0.16:9092,EXTERNAL://146.56.230.220:19092
inter.broker.listener.name=INTERNAL
```

如此，内部通信走内网流量，外部通信走外网流量，实现了内外网分流。

3.1.2.4 通过本地拨出避免开放端口

通过上一节的配置，我们实现了内外网分流，节省了外网流量，提升了内部通信的效率。但是，由于安全组的存在，外网客户端要连接到集群，需要开放 19092 端口，会增加被攻击的可能性，这是我们所不愿看到的。如果将 Zookeeper 中记录的 EXTERNAL URI 替换为本地的某个端口，再结合 XShell 隧道的拨出功能，就可以在不放通 19092 的前提下建立连接。

1) 集群配置

修改 advertised.listeners

(1) hadoop102

```
[atguigu@hadoop102 kafka-3.3.1]$ cd $KAFKA_HOME
[atguigu@hadoop102 kafka-3.3.1]$ vim config/server.properties
```

修改如下

```
advertised.listeners=INTERNAL://10.206.16.6:9092,EXTERNAL://localhost:9092
```

(2) hadoop103

```
[atguigu@hadoop103 kafka-3.3.1]$ cd $KAFKA_HOME
[atguigu@hadoop103 kafka-3.3.1]$ vim config/server.properties
```

修改如下

```
advertised.listeners=INTERNAL://10.206.16.17:9092,EXTERNAL://localhost:9093
```

(3) hadoop104

```
[atguigu@hadoop104 kafka-3.3.1]$ cd $KAFKA_HOME
[atguigu@hadoop104 kafka-3.3.1]$ vim config/server.properties
```

修改如下

```
advertised.listeners=INTERNAL://10.206.0.16:9092,EXTERNAL://localhost:9094
```

2) 隧道配置

如下。


```
import
org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;
import org.apache.kafka.clients.consumer.OffsetResetStrategy;

import java.io.IOException;

public class KafkaTest1 {
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        conf.setInteger(RestOptions.PORT, 8081);
        StreamExecutionEnvironment env =
StreamExecutionEnvironment.getExecutionEnvironment(conf);

        KafkaSource<String> kafkaSource =
KafkaSource.<String>builder()
            .setBootstrapServers("localhost:9092, localhost:9093,
localhost:9094")
            .setTopics("test")
            .setGroupId("test")
            .setStartingOffsets(OffsetsInitializer.committedOffsets(OffsetResetStrategy.LATEST))
            .setValueOnlyDeserializer(new
DeserializationSchema<String>() {
                @Override
                public String deserialize(byte[] bytes) throws
IOException {
                    if (bytes != null && bytes.length != 0) {
                        return new String(bytes);
                    }
                    return null;
                }

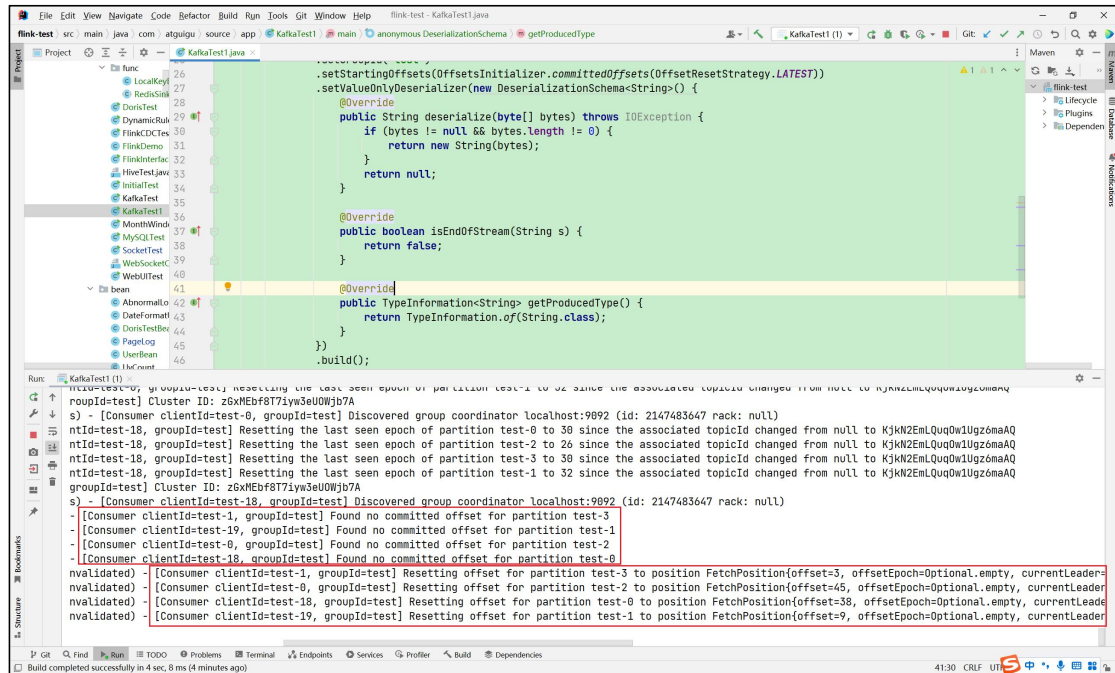
                @Override
                public boolean isEndOfStream(String s) {
                    return false;
                }

                @Override
                public TypeInformation<String> getProducedType() {
                    return TypeInformation.of(String.class);
                }
            })
            .build();
        DataStreamSource<String> source =
env.fromSource(kafkaSource,
            WatermarkStrategy.noWatermarks(), "kafka_source");

        source.print();

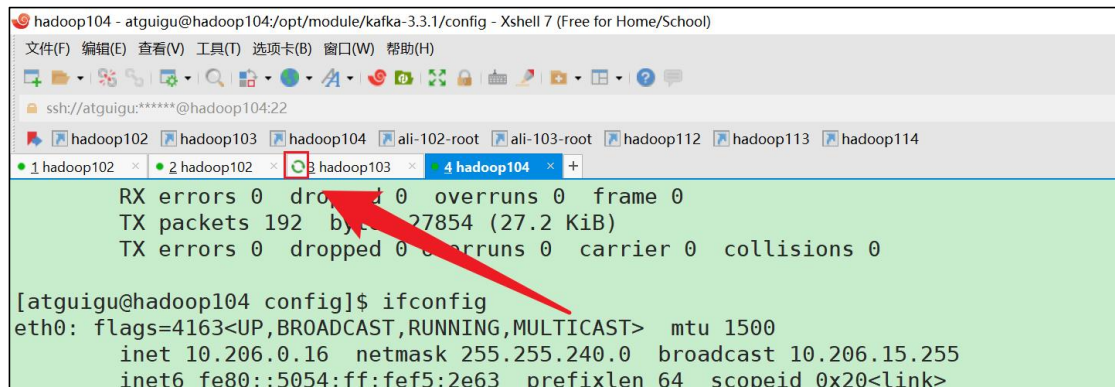
        env.execute();
    }
}
```

(2) 执行结果如下



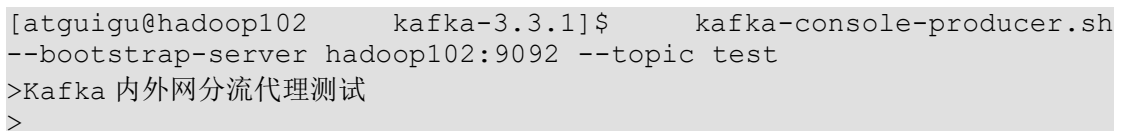
打印如上日志，说明连接已经建立。

(3) 查看 XShell 窗口图标

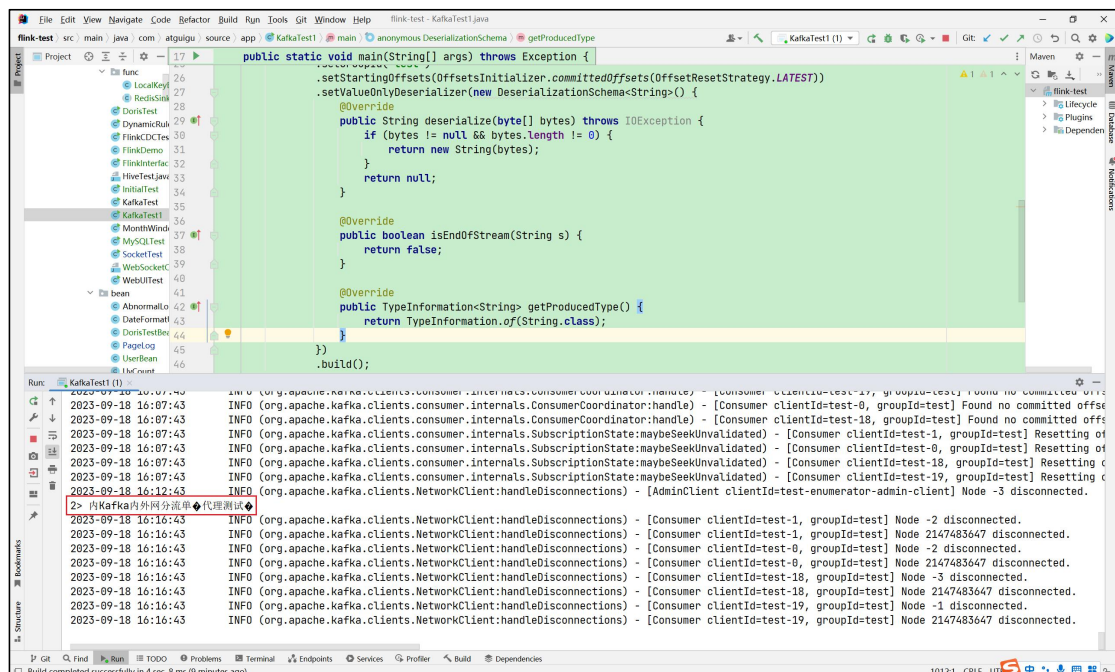


转发已生效。

(4) 启动命令行生产者并发送数据



(5) 查看 IDEA 程序



可以消费数据。测试成功。

3.1.3 访问 HBase

与 Kafka 类似，HBase 客户端的读写也需要 Zookeeper 的参与，连接的建立大致分为两部分。

- (1) 首先要连接到 Zookeeper 获取集群的元数据信息，这一步可以通过拨出隧道转发。
- (2) 然后再连接到 HMaster 或 HRegionServer，与 Kafka 不同，Zookeeper 中记录的 HMaster 和 HRegionServer URI 是 HBase 集群根据系统环境和一些配置项自动生成的，我们无法干预，因此，建立连接只能通过开放端口的方式实现。

1) 将 Zookeeper Quorum 设置为本地端口

测试代码如下。

(1) HBaseUtil

```
package com.atguigu.source.util;

import com.alibaba.fastjson.JSONObject;
import org.apache.hadoop.hbase.Cell;
import org.apache.hadoop.hbase.CellUtil;
import org.apache.hadoop.hbase.TableName;
import org.apache.hadoop.hbase.client.*;
import org.apache.hadoop.hbase.util.Bytes;

import java.io.IOException;
import java.util.List;

public class HbaseUtil {
```



```
public static JSONObject readDim(Connection conn,
                                String nameSpace,
                                String tableName,
                                String rowKey) {

    try {
        Table table = conn.getTable(TableName.valueOf(nameSpace,
        tableName));
        Get get = new Get(Bytes.toBytes(rowKey));
        Result result = table.get(get);
        List<Cell> cells = result.listCells(); // 得到所有的列
        JSONObject obj = new JSONObject();

        for (Cell cell : cells) {
            String name =
            Bytes.toString(CellUtil.cloneQualifier(cell));
            String value =
            Bytes.toString(CellUtil.cloneValue(cell));
            obj.put(name, value);
        }
        table.close();
        return obj;
    } catch (IOException e) {
        e.printStackTrace();
    }
    return null;
}

public static void closeHbaseConnection(Connection conn) {
    if (conn != null && !conn.isClosed()) {
        try {
            conn.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

/**
 * 在 hbase 中建表
 *
 * @param hbaseConn 到 hbase 的连接对象
 * @param nameSpace 命名空间
 * @param tableName 表名
 * @param families 列族
 */
public static void createHbaseTable(Connection hbaseConn,
                                    String nameSpace,
                                    String tableName,
                                    String... families) {
    if (families.length < 1) {
        throw new IllegalArgumentException("请至少需要一个列族名...");
    }
}
```

```
        try (Admin admin = hbaseConn.getAdmin()) {
            //判断表是否存在
            if (admin.tableExists(TableNames.valueOf(nameSpace,
tableNames))) {
                System.out.println(nameSpace + ":" + tableName + "已存
在");
                return;
            }
            TableDescriptorBuilder builder =
TableDescriptorBuilder.newBuilder(TableNames.valueOf(nameSpace,
tableName));

            //指定列族
            for (String family : families) {
                ColumnFamilyDescriptor familyDescriptor =
ColumnFamilyDescriptorBuilder
                    .newBuilder(Bytes.toBytes(family)).build();
                builder.setColumnFamily(familyDescriptor);
            }
            admin.createTable(builder.build());
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    public static void putRow(Connection conn,
        String nameSpace,
        String tableName,
        String rowKey,
        String family,
        String[] columns,
        String[] values) throws IOException {
        // 获取 table 对象
        Table table = conn.getTable(TableNames.valueOf(nameSpace,
tableName));
        // 根据传入的参数创建 put 对象
        Put put = new Put(Bytes.toBytes(rowKey));
        for (int i = 0; i < columns.length; i++) {
            String column = columns[i];
            String value = values[i];
            if (value != null) { // 有部分列的值是 null, 不用写出
                put.addColumn(Bytes.toBytes(family),
Bytes.toBytes(column), Bytes.toBytes(value));
            }
        }
        table.put(put);
        table.close();
    }
}
```

(2) 测试程序

```
package com.atguigu.source.app;

import com.alibaba.fastjson.JSONObject;
```

```
import com.atguigu.source.util.HbaseUtil;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.hbase.HBaseConfiguration;
import org.apache.hadoop.hbase.client.Connection;
import org.apache.hadoop.hbase.client.ConnectionFactory;

import java.io.IOException;
import java.util.concurrent.ExecutionException;

public class HBaseTest {
    public static void main(String[] args) throws ExecutionException,
        InterruptedException, IOException {
        Configuration conf = HBaseConfiguration.create();

        // 配置 Zookeeper
        conf.set("hbase.zookeeper.quorum",
            "localhost:2181,localhost:2182,localhost:2183");

        Connection connection =
            ConnectionFactory.createConnection(conf);

        HbaseUtil.createHbaseTable(connection, "test_namespace",
            "test_table", "test_family");

        String[] columns = {"test_column"};
        String[] values = {"test_value"};

        HbaseUtil.putRow(connection, "test_namespace", "test_table",
            "1", "test_family", columns, values);

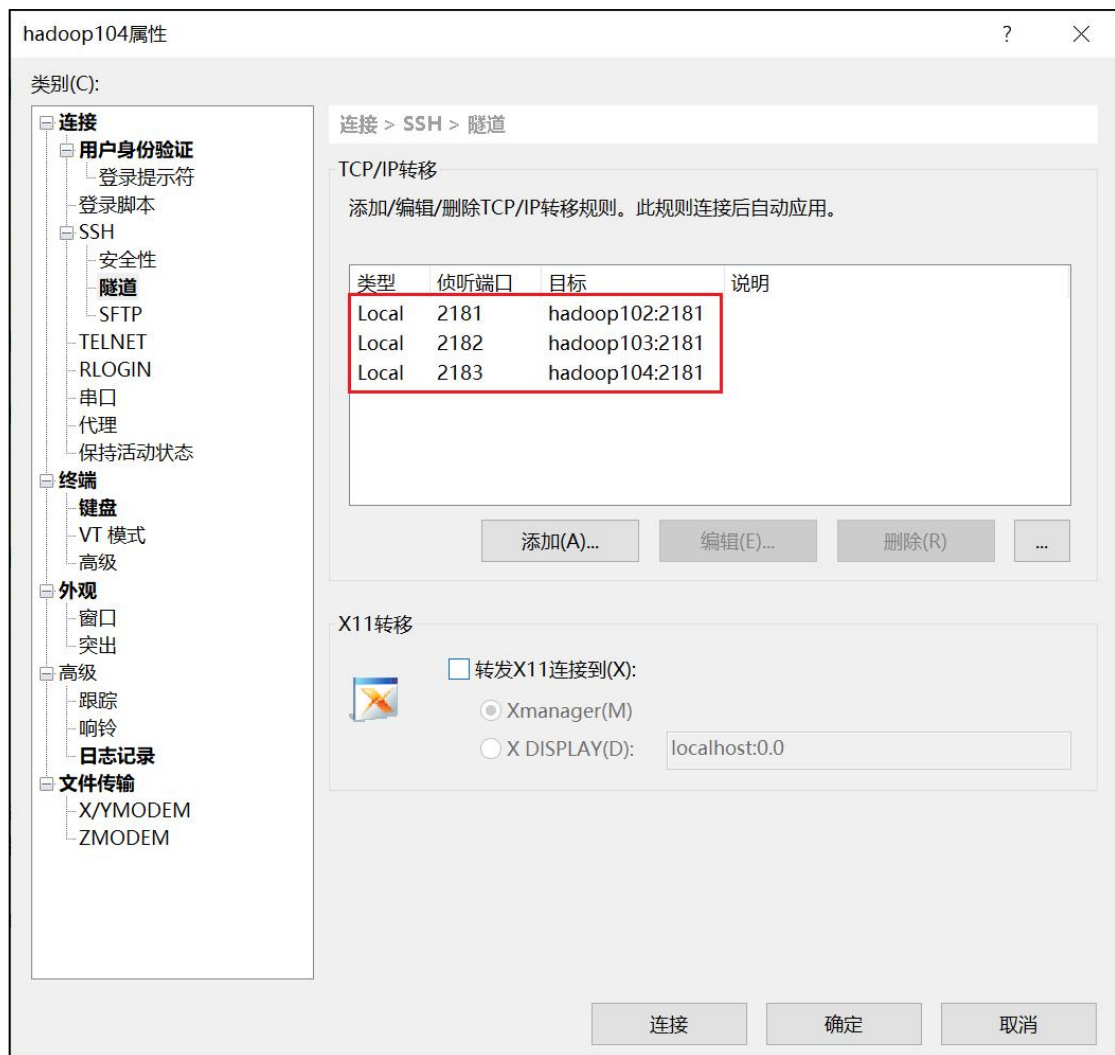
        JSONObject value = HbaseUtil.readDim(connection,
            "test_namespace", "test_table", "1");

        System.out.println("value = " + value);

        connection.close();
    }
}
```

2) 配置隧道

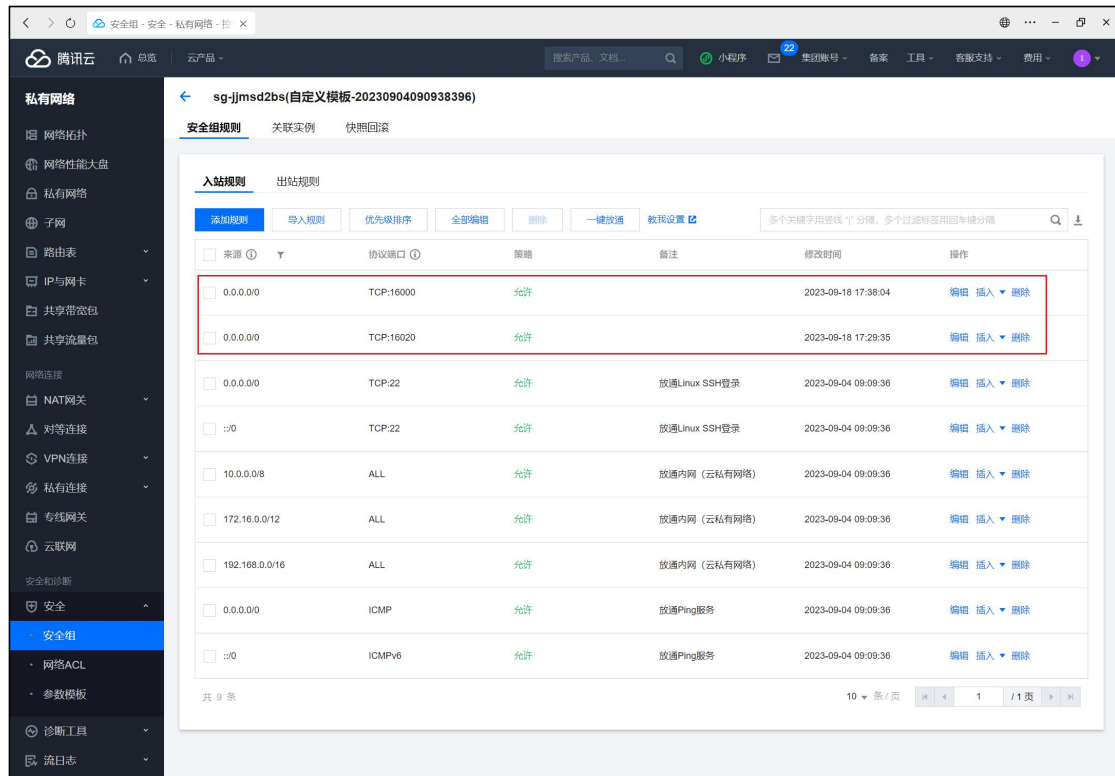
将本地的 2181 端口和 hadoop102 的 2181 绑定, 本地的 2182 端口和 hadoop103 的 2181 绑定, 本地的 2183 端口和 hadoop104 的 2181 绑定, 如下。



3) 放通 HBase 相关端口

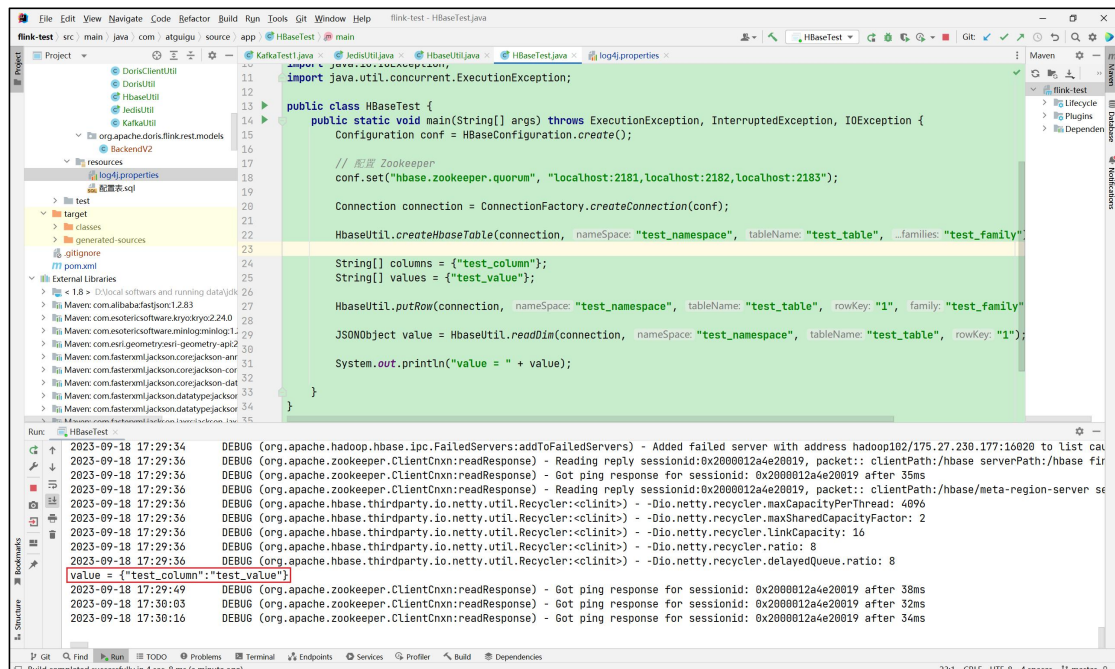
如果只是读写数据，只需要开放客户端连接 HRegionServer 的通信端口 16020，如果涉及到元数据修改，如创建删除表结构，则要放通客户端连接 HMaster 的通信端口 16000 和 16020，之所以要放通 16020 是因为建表和删除表结构都需要在 HBase meta 表中读取表的信息，需要连接到 meta 表所在的 HRegionServer 读取数据。

腾讯云安全组配置如下。



4) 执行代码，查看效果

结果如下。



3.1.4 访问 Redis

1) 代码中请求本地端口

获取连接时主机名指定为 localhost，端口为 6379。

更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
package com.atguigu.source.util;

import redis.clients.jedis.Jedis;
import redis.clients.jedis.JedisPool;
import redis.clients.jedis.JedisPoolConfig;

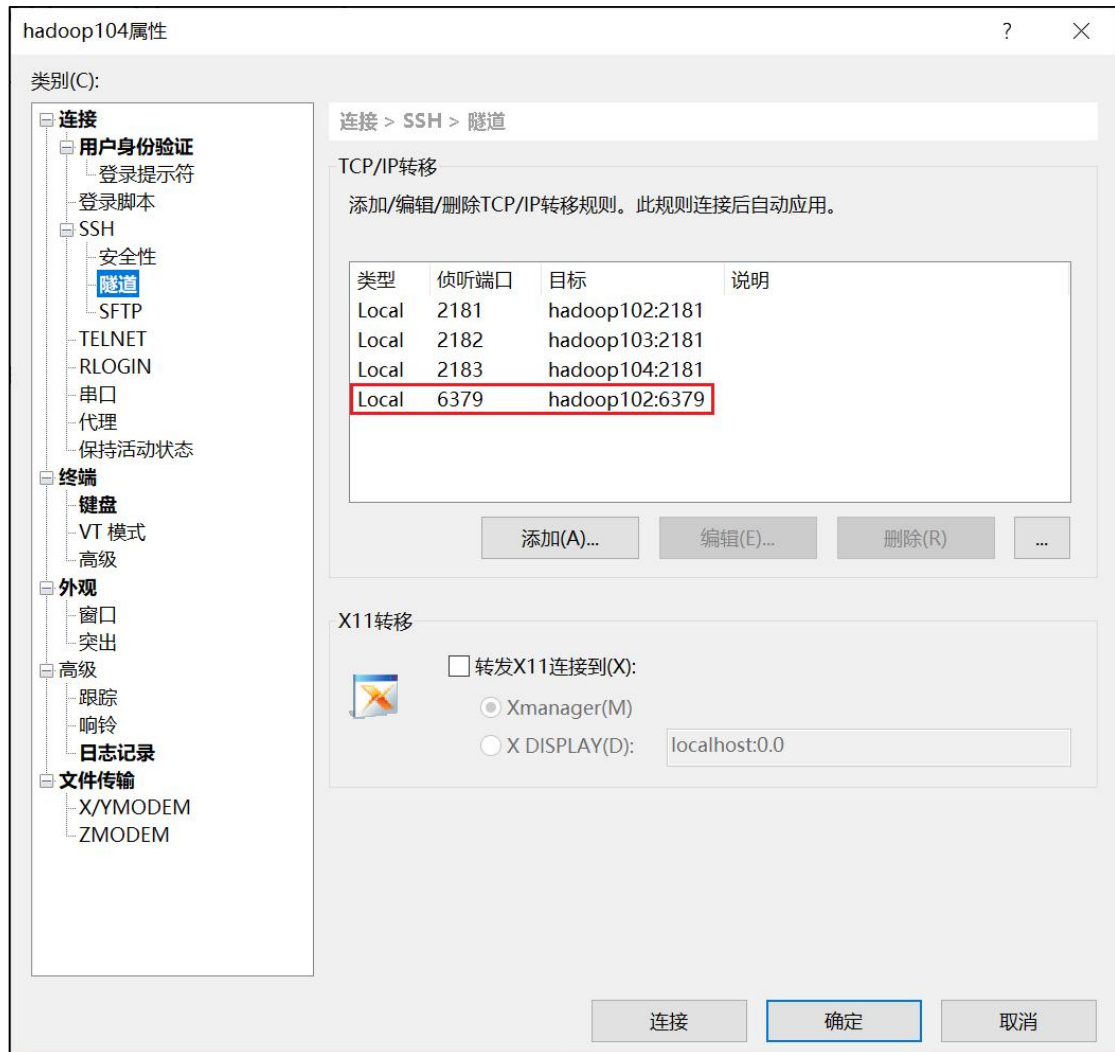
public class JedisUtil {
    private static JedisPool jedisPool;

    static {
        JedisPoolConfig poolConfig = new JedisPoolConfig();
        poolConfig.setMaxTotal(100);
        poolConfig.setMaxIdle(5);
        poolConfig.setMinIdle(5);
        poolConfig.setBlockWhenExhausted(true);
        poolConfig.setMaxWaitMillis(2000);
        poolConfig.setTestOnBorrow(true);
        jedisPool = new JedisPool(poolConfig, "localhost", 6379,
10000);
    }

    public static Jedis getJedis() {
        // 获取 Jedis 客户端
        Jedis jedis = jedisPool.getResource();
        return jedis;
    }

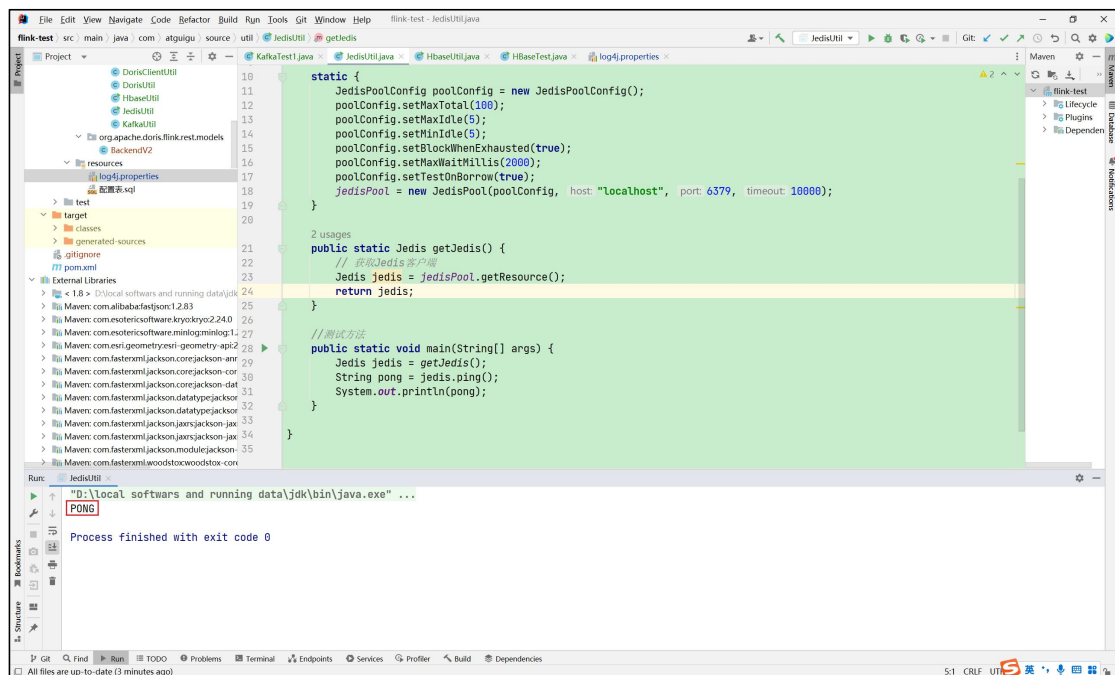
    //测试方法
    public static void main(String[] args) {
        Jedis jedis = getJedis();
        String pong = jedis.ping();
        System.out.println(pong);
    }
}
```

2) 配置隧道



3) 测试代码，查看结果

结果如下。



3.1.5 访问 HDFS

实时项目需要将检查点保存在 HDFS，涉及到 HDFS 的读写。

HDFS 的读写操作都需要先请求 NameNode，然后获取 DataNode 的地址建立连接，然后开始数据传输。需要注意的是，默认情况下返回的 DataNode URI 中节点地址是通过 IP 标识的，而 HDFS 集群内节点是通过内网 IP 通信的，外网客户端直接通过内网 IP 去连接是不可能成功的。好在 HDFS 提供了一个配置项，可以将 DataNode URI 中的 IP 替换为主机名，该配置项为：dfs.client.use.datanode.hostname，默认值为 false，如果要想实现上述效果，在 hdfs-site.xml 中添加内容，将该配置项的值置为 true 即可。注意，如果只是在集群中修改了该配置项，IDEA 测试环境无法生效，需要在项目的 resources 目录下添加 hdfs-site.xml 方可生效。

通过上述分析可知，客户端要和 NameNode、DataNode 建立连接，端口分别为 8020 和 9866。结合上文的讲解，与 NameNode 建立连接时可以通过本地拨出的方式转发请求。但 DataNode URI 并不能自由配置，因此只能放通 9866 端口。

不过，IDEA 运行程序是为了测试代码，对安全性没有那么高的要求，而生产环境应用会被打包部署到服务器，程序和集群节点处于同一内网环境，不会被安全组拦截。因此，测试环境可以直接调用 enableCheckpointing() 方法启用检查点，使用默认的

HashMapStateBackend()作为状态后端，使用默认的 JobManager 堆内存存储检查点。这样就不必放通端口，减小被攻击的可能性。

3.1.6 访问 Doris

Doris 为客户端提供了两种连接远端服务器执行读写操作的方式，如下。

3.1.6.1 通过 MySQL 客户端连接远端服务器

可以通过 JDBC 方式读写数据，与访问 MySQL 服务器类似，只需要将 3306 端口替换为 9030（默认，由 fe.conf 中的 query_port 指定）即可。可以效仿 3.1.1 节，通过本地拨出的方式映射到远端的 9030 端口。

(1) doris 建库建表

```
create database test;
drop table if exists test.stu;
CREATE TABLE if not exists test.stu
(
    name VARCHAR(100)
)
AGGREGATE KEY(name)
DISTRIBUTED BY HASH(name) BUCKETS 10;
```

(2) JDBC 工具类

```
package com.atguigu.source.util;

import org.apache.flink.connector.jdbc.JdbcConnectionOptions;
import org.apache.flink.connector.jdbc.JdbcExecutionOptions;
import org.apache.flink.connector.jdbc.JdbcSink;
import org.apache.flink.connector.jdbc.JdbcStatementBuilder;
import org.apache.flink.streaming.api.functions.sink.SinkFunction;

import java.lang.reflect.Field;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class DorisClientUtil {
    public static <T> SinkFunction<T> getJdbcSink(String sql) {

        return JdbcSink.<T>sink(
            sql,
            new JdbcStatementBuilder<T>() {
                @Override
                public void accept(PreparedStatement
preparedStatement, T obj) throws SQLException {
                    Field[] declaredFields =
obj.getClass().getDeclaredFields();
                    int skipNum = 0;
                    for (int i = 0; i < declaredFields.length; i++)
                    {
                        Field declaredField = declaredFields[i];
                        declaredField.setAccessible(true);
                        try {
```

```

        Object value = declaredField.get(obj);
        preparedStatement.setObject(i + 1 -
skipNum, value);
    } catch (IllegalAccessException e) {
        System.out.println("Doris 数据插入 SQL 占
位符传参异常 ~");
        e.printStackTrace();
    }
}
},
JdbcExecutionOptions.builder()
    .withBatchIntervalMs(100L)
    .withBatchSize(1)
    .build(),
new
JdbcConnectionOptions.JdbcConnectionOptionsBuilder()
    .withDriverName("com.mysql.cj.jdbc.Driver")
    .withUrl("jdbc:mysql://localhost:9030/test")
    .withUsername("root")
    .withPassword("")
    .build()
);
}
}

```

(3) 工具类 KafkaUtil

```

package com.atguigu.source.util;

import
org.apache.flink.api.common.serialization.DeserializationSchema;
import org.apache.flink.api.common.typeinfo.TypeInformation;
import org.apache.flink.connector.kafka.source.KafkaSource;
import
org.apache.flink.connector.kafka.source.enumerator.initializer.OffsetsInitializer;
import org.apache.kafka.clients.consumer.OffsetResetStrategy;

import java.io.IOException;

public class KafkaUtil {
    /**
     * 指定主题和消费者组获取 FlinkKafkaConsumer 对象
     *
     * @param topic 主题
     * @param groupId 消费者组
     * @return FlinkKafkaConsumer 实例
     */
    public static KafkaSource<String> getKafkaConsumer(String topic,
String groupId, OffsetResetStrategy offsetReset) {

        return KafkaSource.<String>builder()
            .setBootstrapServers("localhost:9092, localhost:9093,
localhost:9094")
            .setTopics(topic)
            .setGroupId(groupId)

```

```
        .setStartingOffsets(OffsetsInitializer.committedOffsets(offsetReset))
        .setValueOnlyDeserializer(new
DeserializationSchema<String>() {
            @Override
            public String deserialize(byte[] bytes) throws
IOException {
                if (bytes != null && bytes.length != 0) {
                    return new String(bytes);
                }
                return null;
            }

            @Override
            public boolean isEndOfStream(String s) {
                return false;
            }

            @Override
            public TypeInformation<String> getProducedType() {
                return TypeInformation.of(String.class);
            }
        })
        .build();
    }
}
```

(4) 实体类 DorisTestBean

```
package com.atguigu.source.bean;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor
@Builder
public class DorisTestBean {
    // 学生姓名
    String name;
}
```

(5) 测试程序

```
package com.atguigu.source.app;

import com.atguigu.source.bean.DorisTestBean;
import com.atguigu.source.util.DorisClientUtil;
import com.atguigu.source.util.KafkaUtil;
import org.apache.flink.api.common.eventtime.WatermarkStrategy;
import org.apache.flink.configuration.Configuration;
import org.apache.flink.configuration.RestOptions;
import org.apache.flink.streaming.api.datastream.DataStreamSource;
import org.apache.flink.streaming.api.datastream.SingleOutputStreamOperator;
```

```
import
org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;
import org.apache.kafka.clients.consumer.OffsetResetStrategy;

public class DorisTest {
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        conf.setInteger(RestOptions.PORT, 8081);
        StreamExecutionEnvironment env =
        StreamExecutionEnvironment.getExecutionEnvironment(conf);
        env.disableOperatorChaining();

        env.setParallelism(1);
        env.enableCheckpointing(5000L);

        DataStreamSource<String> source =
        env.fromSource(KafkaUtil.getKafkaConsumer("test", "test",
        OffsetResetStrategy.LATEST),
        WatermarkStrategy.noWatermarks(), "test");

        SingleOutputStreamOperator<DorisTestBean> mappedStream =
        source.map(DorisTestBean::new);
        mappedStream.print();

        source.print();

        mappedStream.addSink(DorisClientUtil.getJdbcSink("insert
        into test.stu values(?)"));

        env.execute();
    }
}
```

(6) 隧道配置

转移规则

?

×

类型(方向)(T):

本地(拨出)

▼

源主机(S):

localhost

▼

侦听端口(L):

9030

▼

☐

仅接受本地连接(A)

目标主机(H):

hadoop102

▼

目标端口(P):

9030

▼

说明(D):

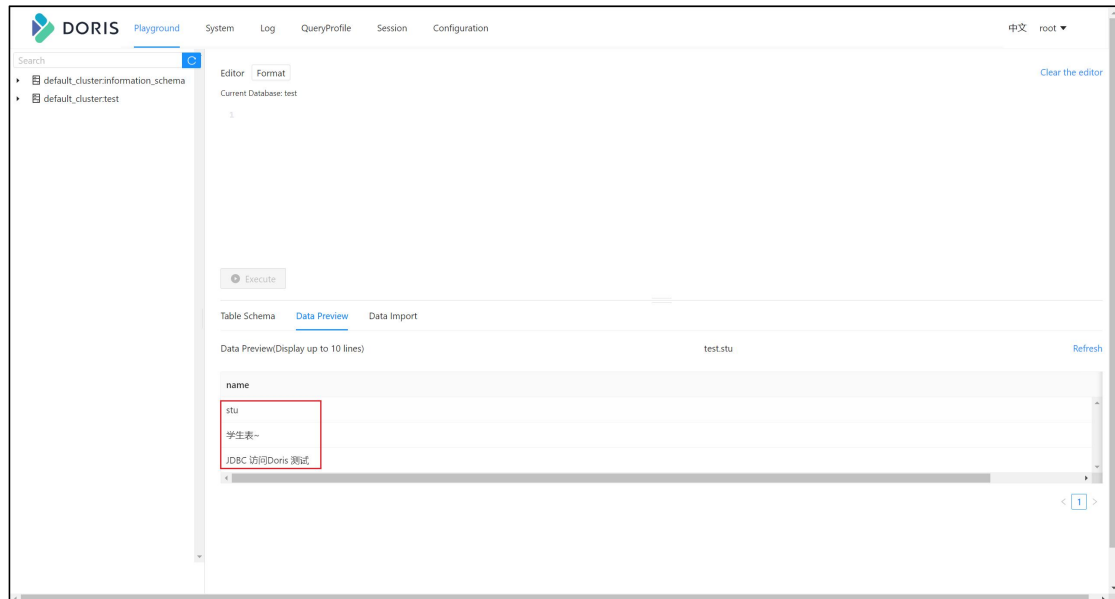
确定

取消

(7) 启动 Kafka 命令行生产者，发送数据如下

```
[atguigu@hadoop102 be-1.2.4.1]$ kafka-console-producer.sh
--bootstrap-server hadoop102:9092 --topic test
>JDBC 访问 Doris 测试
>学生表~
>stu
>
```

(8) 查询



3.1.6.2 基于 HTTP 协议连接远端服务器

基于 HTTP 访问 Frontend 的默认端口为 8030, 由 fe.conf 中的 http_port 指定, 基于 HTTP 访问 Backend 的默认端口为 8040, 由 be.conf 中的 webserver_port 指定。此处使用默认端口。

为了排除其它因素的干扰，首先开放 8030 和 8040 端口。

1) 修改代码，切换访问方式

(1) 定义 DorisUtil 工具类

```
package com.atguigu.source.util;

import org.apache.doris.flink.cfg.DorisExecutionOptions;
import org.apache.doris.flink.cfg.DorisOptions;
import org.apache.doris.flink.cfg.DorisReadOptions;
import org.apache.doris.flink.sink.DorisSink;
import org.apache.doris.flink.sink.writer.SimpleStringSerializer;

import java.util.Properties;

public class DorisUtil {

    public static DorisSink<String> getDorisSink(String table, String
dorisLabel) {
        Properties props = new Properties();
```

```

        props.setProperty("format", "json");
        props.setProperty("read_json_by_line", "true"); // 每行一条
json 数据

        return DorisSink.<String>builder()
            .setDorisReadOptions(DorisReadOptions.builder().build())
            .setDorisOptions(DorisOptions.builder() // 设置 doris
的连接参数
                .setFenodes("hadoop102:8030")
                .setTableIdentifier(table)
                .setUsername("root")
                .setPassword("")
                .build())
            .setDorisExecutionOptions(DorisExecutionOptions.builder() // 执行参数
                .setLabelPrefix(dorisLabel) // stream-load 导
入的时候的 label 前缀
                .disable2PC() // 开启两阶段提交后,labelPrefix 需要
全局唯一,为了测试方便禁用两阶段提交
                .setDeletable(false)
                .setBufferCount(3) // 批次条数: 默认 3
                .setBufferSize(8 * 1024) // 批次大小: 默认 1M
                .setCheckInterval(3000) // 批次输出间隔 三个对批
次的限制是或的关系
                .setMaxRetries(3)
                .setStreamLoadProp(props) // 设置 stream load 的
数据格式 默认是 csv,需要改成 json
                .build())
            .setSerializer(new SimpleStringSerializer())
            .build();
    }
}

```

(2) 更改 DorisTest 测试类

```

package com.atguigu.source.app;

import com.atguigu.source.util.DorisUtil;
import com.atguigu.source.util.KafkaUtil;
import org.apache.flink.api.common.eventtime.WatermarkStrategy;
import org.apache.flink.configuration.Configuration;
import org.apache.flink.configuration.RestOptions;
import org.apache.flink.streaming.api.datastream.DataStreamSource;
import
org.apache.flink.streaming.api.environment.StreamExecutionEnviron
ment;
import org.apache.kafka.clients.consumer.OffsetResetStrategy;

public class DorisTest {
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        conf.setInteger(RestOptions.PORT, 8081);
        StreamExecutionEnvironment env =
StreamExecutionEnvironment.getExecutionEnvironment(conf);
    }
}

```



```
env.disableOperatorChaining();

env.setParallelism(1);
env.enableCheckpointing(5000L);

DataStreamSource<String> source =
env.fromSource(KafkaUtil.getKafkaConsumer("test", "test",
OffsetResetStrategy.LATEST),
WatermarkStrategy.noWatermarks(), "test");

source.map(str -> "{\"name\": \"" + str + "\"}")
.sinkTo(DorisUtil.getDorisSink("test.stu", "test"));

env.execute();
}
```

2) 连接建立流程分析

(1) 执行修改后的 DorisTest 测试程序，将日志级别降到 info 及以下，可以观察到如下信息

```
INFO (org.apache.flink.runtime.checkpoint.CheckpointFailureManager:handleCheckpointException) - Failed to trigger checkpoint for job
INFO (org.apache.flink.runtime.checkpoint.CheckpointFailureManager:handleCheckpointException) - Failed to trigger checkpoint for job
INFO (org.apache.flink.runtime.checkpoint.CheckpointFailureManager:handleCheckpointException) - Failed to trigger checkpoint for job
INFO (org.apache.flink.runtime.checkpoint.CheckpointFailureManager:handleCheckpointException) - Failed to trigger checkpoint for job
WARN (org.apache.doris.flink.sink.writer.DorisWriter:tryHttpConnection) - Failed to connect to backend:http://10.206.16.6:8040
Create breakpoint: Connection timed out: connect
PlainSocketImpl.waitForConnect(Native Method)
PlainSocketImpl.socketConnect(DualStackPlainSocketImpl.java:85)
PlainSocketImpl.doConnect(AbstractPlainSocketImpl.java:350)
```

报错提示无法连接到 Backend，URI 为：<http://10.206.16.6:8040>，可以发现，10.206.16.6 为 hadoop102 的内网 IP，此处通过内网 IP 连接 8040 端口，必然无法成功。

(2) 查找日志中的其它信息

```
askmanager.Task:transitionState) - Sink: Writer (1/1)#0 (723b81e63d48961ab6dd747cdb5bcb5f_ea632d67b7d595e5b851708ae9ad79d6_0_0) switched from
askmanager.Task:transitionState) - Sink: Committer (1/1)#0 (723b81e63d48961ab6dd747cdb5bcb5f_6d2677a0ecc3fd8df0b72ec675edf8f4_0_0) switched fr
xecutionGraph.ExecutionGraph:transitionState) - Source: test (1/1) (723b81e63d48961ab6dd747cdb5bcb5f_bc764cd8ddf7a0cff126f51c16239658_0_0) swi
xecutionGraph.ExecutionGraph:transitionState) - Map (1/1) (723b81e63d48961ab6dd747cdb5bcb5f_0a448493b4782967b150582570326227_0_0) switched frd
xecutionGraph.ExecutionGraph:transitionState) - Sink: Writer (1/1) (723b81e63d48961ab6dd747cdb5bcb5f_ea632d67b7d595e5b851708ae9ad79d6_0_0) swi
xecutionGraph.ExecutionGraph:transitionState) - Sink: Committer (1/1) (723b81e63d48961ab6dd747cdb5bcb5f_6d2677a0ecc3fd8df0b72ec675edf8f4_0_0) swi
askmanager.Task:transitionState) - Map (1/1)#0 (723b81e63d48961ab6dd747cdb5bcb5f_0a448493b4782967b150582570326227_0_0) switched from INITIALI
xecutionGraph.ExecutionGraph:transitionState) - Map (1/1) (723b81e63d48961ab6dd747cdb5bcb5f_0a448493b4782967b150582570326227_0_0) switched frd
askmanager.Task:transitionState) - Sink: Committer (1/1)#0 (723b81e63d48961ab6dd747cdb5bcb5f_6d2677a0ecc3fd8df0b72ec675edf8f4_0_0) switched fr
k.writer.DorisWriter:<init>) - restore checkpointId 0
k.writer.DorisWriter:<init>) - labelPrefix test
xecutionGraph.ExecutionGraph:transitionState) - Sink: Committer (1/1) (723b81e63d48961ab6dd747cdb5bcb5f_6d2677a0ecc3fd8df0b72ec675edf8f4_0_0)
askmanager.Task:transitionState) - Source: test (1/1)#0 (723b81e63d48961ab6dd747cdb5bcb5f_bc764cd8ddf7a0cff126f51c16239658_0_0) switched from
ource.coordinator.SourceCoordinator:handleReaderRegistrationEvent) - Source Source: test registering reader for parallel task 0 (#0) @
xecutionGraph.ExecutionGraph:transitionState) - Source: test (1/1) (723b81e63d48961ab6dd747cdb5bcb5f_bc764cd8ddf7a0cff126f51c16239658_0_0) swi
k.writer.DorisWriter:send) - Send request to Doris FE 'http://localhost:8030/api/backends?is\_alive=true' with user 'root'.
k.writer.DorisWriter:getBackendsV2) - Backend Info:{backends":[{"ip":"10.206.16.17","http_port":8040,"is_alive":true},{"ip":"10.206.0.16","h
.kafka.source.enumerator.KafkaSourceEnumerator:getPartitionChange) - Discovered new partitions: [test-1, test-0, test-3, test-2]
.kafka.source.enumerator.KafkaSourceEnumerator:assignPendingPartitionSplits) - Assigning splits to readers {0}=[{Partition: test-0, StartingOff
.base.source.reader.SourceReaderBase:addSplits) - Adding split(s) to reader: [{Partition: test-0, StartingOffset: -3, StoppingOffset: -9223372
onsumer.ConsumerConfig:logAll) - ConsumerConfig values:
```

发现 Backend 信息最早是在 getBackendV2 方法中获取的。

(3) 在 DorisWriter 类中搜索 send 和 getBackendsV2 方法

没有找到 send，但是可以找到 getBackendsV2，如下。

```
1 usage
public void initializeLoad(List<DorisWriterState> state) throws IOException {
    //cache backend
    this.backends = RestService.getBackendsV2(dorisOptions, dorisReadOptions, LOG);
    String backend = getAvailableBackend();
    try {
        this.dorisStreamLoad = new DorisStreamLoad(
            backend,
            dorisOptions,
            executionOptions,
            labelGenerator, new HttpUtil().getHttpClient());
        // TODO: we need check and abort all pending transaction.
        // Discard transactions that may cause the job to fail.
        if(executionOptions.enabled2PC()) {
            dorisStreamLoad.abortPreCommit(labelPrefix, chkID: lastCheckpointId + 1);
        }
    } catch (Exception e) {
        throw new DorisRuntimeException(e);
    }
}
```

(4) 进入 getBackendsV2

```
2 usages
public static List<BackendV2.BackendRowV2> getBackendsV2(DorisOptions options, DorisReadOptions readOptions,
String feNodes = options.getFeNodes();
List<String> feNodeList = allEndpoints(feNodes, logger);
for (String feNode: feNodeList) {
    try {
        String beUrl = "http://" + feNode + BACKENDS_V2;
        HttpGet httpGet = new HttpGet(beUrl);
        String response = send(options, readOptions, httpGet, logger);
        logger.info("Backend Info:{", response);
        List<BackendV2.BackendRowV2> backends = parseBackendV2(response, logger);
        return backends;
    } catch (ConnectedFailedException e) {
        logger.info("Doris FE node {} is unavailable: {}, Request the next Doris FE node", feNode, e.getM
    }
}
String errMsg = "No Doris FE is available, please check configuration";
logger.error(errMsg);
throw new DorisRuntimeException(errMsg);
}
```

找到 send 方法。

(5) 进入 send 方法

```
4 usages
private static String send(DorisOptions options, DorisReadOptions readOptions, HttpRequestBase request, Logger logger) throws
    ConnectedFailedException {
    int connectTimeout = readOptions.getRequestConnectTimeoutMs() == null ? ConfigurationOptions.DORIS_REQUEST_CONNECT_T
    int socketTimeout = readOptions.getRequestReadTimeoutMs() == null ? ConfigurationOptions.DORIS_REQUEST_READ_TIMEOUT_M
    int retries = readOptions.getRequestRetries() == null ? ConfigurationOptions.DORIS_REQUEST_RETRIES_DEFAULT : readOpti
    logger.trace("connect timeout set to '{}'. socket timeout set to '{}'. retries set to '{}'.",
        connectTimeout, socketTimeout, retries);

    RequestConfig requestConfig = RequestConfig.custom()
        .setConnectTimeout(connectTimeout)
        .setSocketTimeout(socketTimeout)
        .build();

    request.setConfig(requestConfig);
    logger.info("Send request to Doris FE '{}' with user '{}'.", request.getURI(), options.getUsername());
    IOException ex = null;
    int statusCode = -1;

    for (int attempt = 0; attempt < retries; attempt++) {
        logger.debug("Attempt {} to request {}.", attempt, request.getURI());
        try {
            String response;
            if (request instanceof HttpGet) {
```

看到日志信息，与上文观察到的 "... DorisWriter:send) - Send request to Doris FE 'http://localhost:8030/api/backends?is_alive=true' with user 'root'." 信息吻合，这里的 send 方法在 RestService 类下，之所以前缀为 DorisWriter 类，是因为打印日志所用的 logger 对象是基于 DorisWriter.class 定义的 Logger 对象。

```
5 usages
public class DorisWriter<IN> implements SinkWriter<IN, DorisCommittable, DorisWriterState> {
    7 usages
    private static final Logger LOG = LoggerFactory.getLogger(DorisWriter.class);
    1 usage
```

(6) 回到 getBackendsV2 方法

```
2 usages
public static List<BackendV2.BackendRowV2> getBackendsV2(DorisOptions options, DorisReadOptions readOptions, Logger logger) {
    String feNodes = options.getFeNodes();
    List<String> feNodeList = allEndpoints(feNodes, logger);
    for (String feNode: feNodeList) {
        try {
            String beUrl = "http://" + feNode + BACKENDS_V2;
            HttpGet httpGet = new HttpGet(beUrl);
            String response = send(options, readOptions, httpGet, logger);
            logger.info("Backend Info:{}", response);
            List<BackendV2.BackendRowV2> backends = parseBackendV2(response, logger);
            return backends;
        } catch (ConnectedFailedException e) {
            logger.info("Doris FE node {} is unavailable: {}, Request the next Doris FE node", feNode, e.getMessage());
        }
    }
    String errMsg = "No Doris FE is available, please check configuration";
    logger.error(errMsg);
    throw new DorisRuntimeException(errMsg);
}
```

打印 Backends 信息

解析 Backends 信息

向 Frontends 发送请求后，获取了 Backends 信息并打印，即上文观察到的信息：

"Backend Info:{"backends":[{"ip":"10.206.16.17","http_port":8040,"is_alive":true},{xxx}."。

然后通过 parseBackendV2 将 response 做了解析，将每个 Backend 的 ip 和端口封装为一个 BackendV2.BackendRowV2 对象，然后放在了 List 集合中。

(7) 回到 initializeLoad 方法

```
1 usage
public void initializeLoad(List<DorisWriterState> state) throws IOException {
    //cache backend
    this.backends = RestService.getBackendsV2(dorisOptions, dorisReadOptions, LOG);
    String backend = getAvailableBackend();
    try {
        this.dorisStreamLoad = new DorisStreamLoad(
            backend,
            dorisOptions,
            executionOptions,
            labelGenerator, new HttpUtil().getHttpClient());
        // TODO: we need check and abort all pending transaction.
        // Discard transactions that may cause the job to fail.
        if(executionOptions.enabled2PC()) {
            dorisStreamLoad.abortPreCommit(labelPrefix, chkID: lastCheckpointId + 1);
        }
    } catch (Exception e) {
        throw new DorisRuntimeException(e);
    }
}
```


返回的 Backends 信息被赋值给了 DorisWriter 的 backends 对象。

然后执行 getAvailableBackend 方法，将其返回值赋给了 backend 对象。

最后将 backend 对象传入 DorisStreamLoad 类的构造器，用于数据装载。

(8) 进入 getAvailableBackend 方法

```
2 usages
@VisibleForTesting
public String getAvailableBackend() {
    long tmp = pos + backends.size();
    while (pos < tmp) {
        BackendV2.BackendRowV2 backend = backends.get((int) (pos % backends.size()));
        String res = backend.toBackendString();
        if (tryHttpConnection(res)){
            pos++;
            return res;
        }
    }
    throw new DorisRuntimeException("no available backend.");
}
```

这个方法同样位于 DorisWriter 类中，首先从 backends 属性中获取元素。基于上文的分析，我们知道，backends 属性的值为记录了 Backends 信息的 List<BackendV2.BackendRowV2>对象。对于 List 集合中的每个元素，调用 toBackendString()方法，将其返回值赋给字符串对象 res，再通过 tryHTTPConnection 测试连接，如下。

```
public boolean tryHttpConnection(String backend) {
    try {
        backend = "http://" + backend;
        URL url = new URL(backend);
        HttpURLConnection co = (HttpURLConnection) url.openConnection();
        co.setConnectTimeout(60000);
        co.connect();
        co.disconnect();
        return true;
    } catch (Exception ex) {
        LOG.warn("Failed to connect to backend:{}", backend, ex);
        pos++;
        return false;
    }
}
```

显然 res 就是 Backend 的 URI。

如果连接测试没有异常，就将 res 返回，用于数据装载。

(9) 进入 toBackendString 方法

该方法位于 BackendV2 类中，该类如下。

```

13 usages
@JsonIgnoreProperties(ignoreUnknown = true)
public class BackendV2 {

    2 usages
    @JsonProperty(value = "backends")
    private List<BackendRowV2> backends;

    public List<BackendRowV2> getBackends() {
        return backends;
    }

    public void setBackends(List<BackendRowV2> backends) { this.backends = backends; }

    12 usages
    public static class BackendRowV2 {
        @JsonProperty("ip")
        public String ip;
        @JsonProperty("http_port")
        public int httpPort;
        @JsonProperty("is_alive")
        public boolean isAlive;

        public String getIp() { return ip; }

        public void setIp(String ip) { this.ip = ip; }

        public int getHttpPort() { return httpPort; }

        public void setHttpPort(int httpPort) { this.httpPort = httpPort; }

        public boolean isAlive() { return isAlive; }

        public void setAlive(boolean alive) { isAlive = alive; }

        1 usage
        public String toBackendString(){
            return ip + ":" + httpPort;
        }
    }
}

```

显然，该类的 ip 属性记录的是 Backend 的内网 IP，httpPort 记录的是 Backend 的 HTTP 协议端口，此处为 8040。

如果能修改源码将此时的 IP 替换为主机名，然后在本地将主机名映射为外网 IP，就可以建立连接。

3) 覆盖源码的类

(1) 知识储备

Java 的类加载器可以分为四种：根类加载器、扩展类加载器、应用类加载器（系统类加载器）和用户自定义类加载器。通常我们只用前三种、

ClassPath 是一个环境变量，用于指示 JVM 如何搜索字节码文件，如果 JVM 在某个路径下找到了对应的 class 文件，就不再往后继续搜索。根类加载器和扩展类加载器并不依赖于 ClassPath，而是从固定的路径加载核心类库。应用类加载器会从 ClassPath 搜索需要的字节码文件，通过 IDEA 运行程序时，项目编译输出目录中的字节码文件和 pom 文件中引入的依赖都会出现在 ClassPath 中，这两部分字节码文件都由应用类加载器加载。

(2) 查看 ClassPath

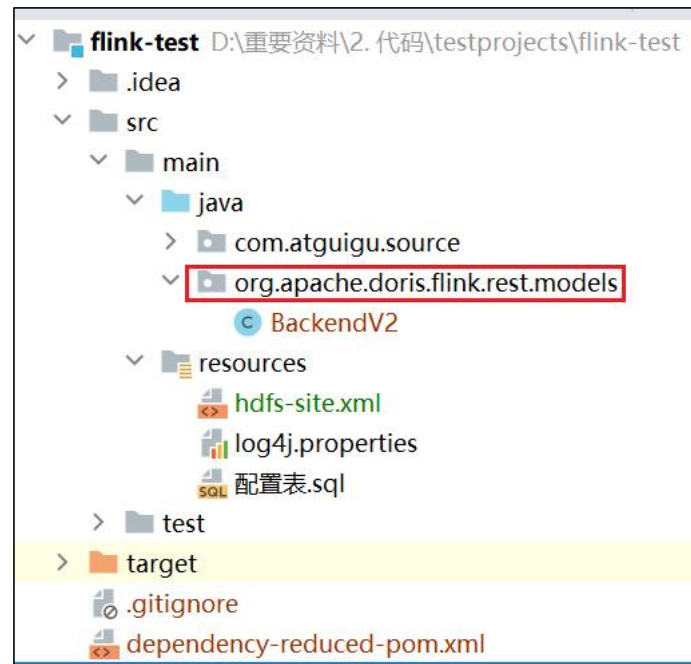
运行测试程序，获取日志第一行，部分内容如下。

可以发现，classpath 最前面的内容为 JRE 中保存的 Java 源码相关的 Jar 包，紧随其后的是我们在本项目中编写的代码编译后的字节码，最后是在 pom 文件中引入的依赖 jar 包。

(3) 编写同名类

上文提到，JVM 需要加载一个核心类库之外的类时，应用类加载器会按照 ClassPath 指定的顺序从前往后扫描，如果有同名类，那么排在最前面的会被加载。通过观察 IDEA 生成的 ClassPath，我们发现自己编写的项目编译输出目录会排在所有依赖之前，所以，只要在自己的项目中创建一个全类名相同的类，就可以把 Doris 的 BackendV2 覆盖掉。在不重新编译源码的前提下完成对源码的修改。

在 IDEA 中创建包，如下



在 `org.apache.doris.flink.rest.models` 包下创建 `BackendV2.java`，将源码中同名类的内容复制制到该文件中，添加 `convertToHostname` 方法，实现 IP 和主机名的映射，如下。

```
package org.apache.doris.flink.rest.models; // Licensed to the Apache
Software Foundation (ASF) under one
// or more contributor license agreements. See the NOTICE file
// distributed with this work for additional information
// regarding copyright ownership. The ASF licenses this file
// to you under the Apache License, Version 2.0 (the
// "License"); you may not use this file except in compliance
// with the License. You may obtain a copy of the License at
//
// http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing,
// software distributed under the License is distributed on an
// "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
// KIND, either express or implied. See the License for the
// specific language governing permissions and limitations
// under the License.

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import com.fasterxml.jackson.annotation.JsonProperty;

import java.util.List;
import java.util.Objects;

/**
 * Be response model
 */
@JsonIgnoreProperties(ignoreUnknown = true)
public class BackendV2 {

    @JsonProperty(value = "backends")
```



```
private List<BackendRowV2> backends;

public List<BackendRowV2> getBackends() {
    return backends;
}

public void setBackends(List<BackendRowV2> backends) {
    this.backends = backends;
}

public static class BackendRowV2 {
    @JsonProperty("ip")
    public String ip;
    @JsonProperty("http_port")
    public int httpPort;
    @JsonProperty("is_alive")
    public boolean isAlive;

    public String getIp() {
        return ip;
    }

    public void setIp(String ip) {
        this.ip = ip;
    }

    public int getHttpPort() {
        return httpPort;
    }

    public void setHttpPort(int httpPort) {
        this.httpPort = httpPort;
    }

    public boolean isAlive() {
        return isAlive;
    }

    public void setAlive(boolean alive) {
        isAlive = alive;
    }

    public String convertToHostname(String ip) {
        this.ip = ip;
        if (Objects.equals(ip, "10.206.16.6")) {
            return "hadoop102";
        } else if (Objects.equals(ip, "10.206.16.17")) {
            return "hadoop103";
        } else if (Objects.equals(ip, "10.206.0.16")) {
            return "hadoop104";
        } else {
            return null;
        }
    }

    public String toBackendString() {
        return convertToHostname(ip) + ":" + httpPort;
    }
}
```

```
}  
  
}  
  
}
```

(4) 执行测试程序

```
at org.apache.flink.runtime.taskmanager.Task.run(Task.java:562) <1 internal line>  
Caused by: org.apache.doris.shaded.com.fasterxml.jackson.databind.exc.UnrecognizedPropertyException: Unrecognized field "http_port" (class org  
at [Source: (String) "{"backends":[{"ip":"10.206.16.17","http_port":8040,"is_alive":true}, {"ip":"10.206.0.16","http_port":8040,"is_alive":true  
at org.apache.doris.shaded.com.fasterxml.jackson.databind.exc.UnrecognizedPropertyException.from(UnrecognizedPropertyException.java:61)  
at org.apache.doris.shaded.com.fasterxml.jackson.databind.DeserializationContext.handleUnknownProperty(DeserializationContext.java:1127)  
at org.apache.doris.shaded.com.fasterxml.jackson.databind.deser.std.StdDeserializer.handleUnknownProperty(StdDeserializer.java:2023)  
at org.apache.doris.shaded.com.fasterxml.jackson.databind.deser.BeanDeserializerBase.handleUnknownProperty(BeanDeserializerBase.java:1760)  
at org.apache.doris.shaded.com.fasterxml.jackson.databind.deser.BeanDeserializerBase.handleUnknownVanilla(BeanDeserializerBase.java:1678)  
at org.apache.doris.shaded.com.fasterxml.jackson.databind.deser.BeanDeserializer.vanillaDeserialize(BeanDeserializer.java:319)  
at org.apache.doris.shaded.com.fasterxml.jackson.databind.deser.BeanDeserializer.deserialize(BeanDeserializer.java:176)  
at org.apache.doris.shaded.com.fasterxml.jackson.databind.deser.std.CollectionDeserializer._deserializeFromArray(CollectionDeserializer.java:244)  
at org.apache.doris.shaded.com.fasterxml.jackson.databind.deser.std.CollectionDeserializer.deserialize(CollectionDeserializer.java:28)  
at org.apache.doris.shaded.com.fasterxml.jackson.databind.deser.impl.MethodProperty.deserializeAndSet(MethodProperty.java:129)  
at org.apache.doris.shaded.com.fasterxml.jackson.databind.deser.BeanDeserializer.vanillaDeserialize(BeanDeserializer.java:313)  
at org.apache.doris.shaded.com.fasterxml.jackson.databind.deser.BeanDeserializer.deserialize(BeanDeserializer.java:176)  
at org.apache.doris.shaded.com.fasterxml.jackson.databind.deser.DefaultDeserializationContext.readRootValue(DefaultDeserializationContext.  
at org.apache.doris.shaded.com.fasterxml.jackson.databind.ObjectMapper._readMapAndClose(ObjectMapper.java:4674)  
at org.apache.doris.shaded.com.fasterxml.jackson.databind.ObjectMapper.readValue(ObjectMapper.java:3629)  
at org.apache.doris.shaded.com.fasterxml.jackson.databind.ObjectMapper.readValue(ObjectMapper.java:3597)  
at org.apache.doris.flink.rest.RestService.parseBackendV2(RestService.java:380)
```

抛出异常，分析发现，JsonProperty 注解没有生效，该注解用于建立 JSON 字符串和实体类中字段名称之间的映射。查看导入的全类名，与源码完全一致，为什么会出问题呢？查看 flink-doris-connector 源码的 pom 文件，看到如下内容。

```
<shadedPattern>org.apache.doris.shaded.io.netty</shadedPattern>  
</relocation>  
<relocation>  
  <pattern>com.fasterxml.jackson</pattern>  
  <shadedPattern>org.apache.doris.shaded.com.fasterxml.jackson</shadedPattern>  
</relocation>  
<relocation>  
  <pattern>org.apache.commons.codec</pattern>  
  <shadedPattern>org.apache.doris.shaded.org.apache.commons.codec</shadedPattern>  
</relocation>  
<relocation>  
  <pattern>com.google.flatbuffers</pattern>  
  <shadedPattern>org.apache.doris.shaded.com.google.flatbuffers</shadedPattern>  
</relocation>
```

此处为了避免依赖冲突，引入了 shaded 插件，对所有全类名以 com.fasterxml.jackson 开头的类做了修改，都加了 org.apache.doris.shaded. 前缀。因此，虽然源码中 JsonProperty 的全类名是 com.fasterxml.jackson.annotation.JsonProperty，但编译后的 jar 包该类的全类名已经变为了 org.apache.doris.shaded.com.fasterxml.jackson.annotation.JsonProperty，要解决上述问题，只要和 jar 包保持一致，在依赖全类名之前补充前缀即可，如下。

```
import  
org.apache.doris.shaded.com.fasterxml.jackson.annotation.JsonIgnoreProperties;  
import  
org.apache.doris.shaded.com.fasterxml.jackson.annotation.JsonProperty;
```

(5) 测试

要写入数据，需要

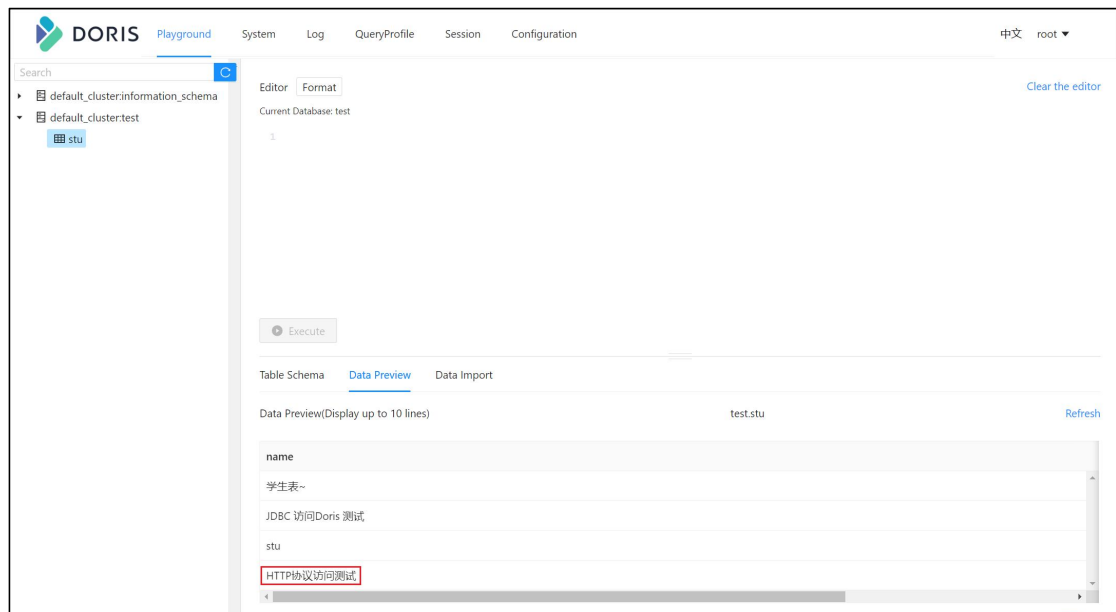
启动 DorisTest，启动命令行生产者，发送消息。

```
[atguigu@hadoop102 ~]$ kafka-console-producer.sh --bootstrap-server hadoop102:9092 --topic test
> JDBC 访问Doris 测试
> 学生表~
> stu
> HTTP协议访问测试
>
```

观察 IDEA 日志，如下。

```
2023-09-19 16:10:59 INFO (org.apache.doris.flink.sink.writer.DorisStreamLoad:stopLoad) - stream load stopped for test_0_b33562ad-9dcf-4ce2-b2e1-1ba6bcde59ee"
2023-09-19 16:10:59 INFO (org.apache.doris.flink.sink.writer.DorisStreamLoad:handlePreCommitResponse) - load Result {
  "TxnId": 7,
  "Label": "test_0_b33562ad-9dcf-4ce2-b2e1-1ba6bcde59ee",
  "TwoPhaseCommit": "false",
  "Status": "Success",
  "Message": "OK",
  "NumberTotalRows": 1,
  "NumberLoadedRows": 1,
  "NumberFilteredRows": 0,
  "NumberUnselectedRows": 0,
  "LoadBytes": 34,
  "LoadTimeMs": 3869,
  "BeginTxnTimeMs": 7,
  "StreamLoadPutTimeMs": 41,
  "ReadDataTimeMs": 0,
  "WriteDataTimeMs": 3789,
  "CommitAndPublishTimeMs": 30
}
```

查看数据



测试成功

(6) 通过转发避免开放端口

① 修改 DorisUtil 中的 FeNodes 为本地 IP 端口组合

```
...
return DorisSink.<String>builder()
    .setDorisReadOptions(DorisReadOptions.builder().build())
    .setDorisOptions(DorisOptions.builder() // 设置 doris 的连接参数
数
```

```
.setFenodes("localhost:8030")
.setTableIdentifier(table)
.setUsername("root")
...
```

② 修改 BackendV2 的 toBackendString 方法返回值为本地 IP 端口组合

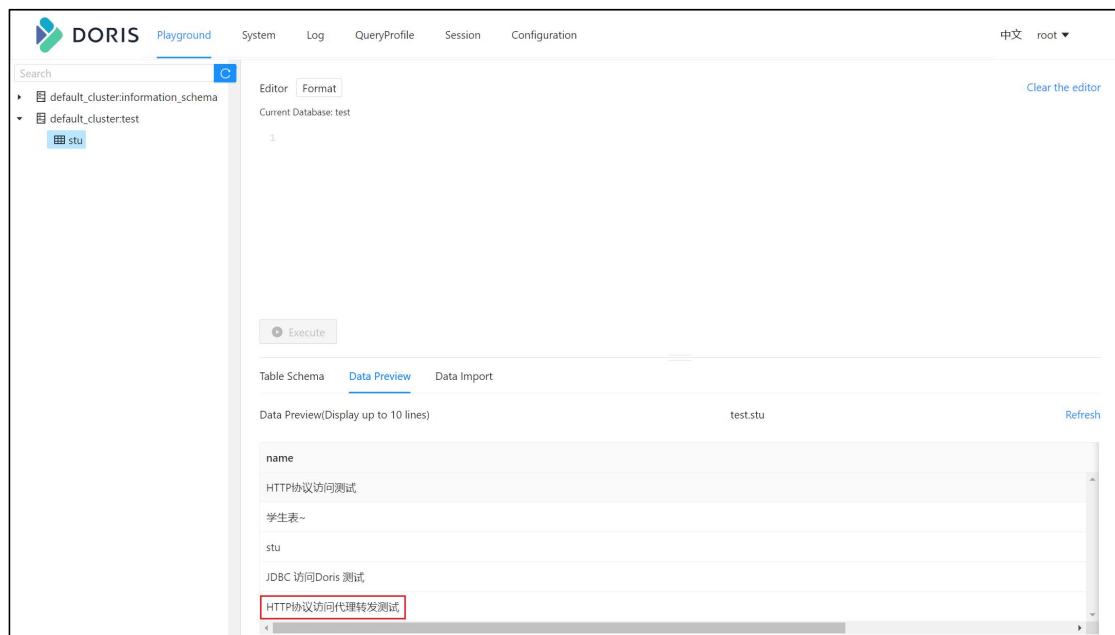
```
public String convertToHostname(String ip) {
    this.ip = ip;
    if (Objects.equals(ip, "10.206.16.6")) {
        return "localhost:8040";
    } else if (Objects.equals(ip, "10.206.16.17")) {
        return "localhost:8041";
    } else if (Objects.equals(ip, "10.206.0.16")) {
        return "localhost:8042";
    } else {
        return null;
    }
}

public String toBackendString() {
    return convertToHostname(ip);
}
```

③ 配置隧道


```
2023-09-19 16:23:21 INFO (org.apache.doris.flink.sink.writer.DorisStreamLoad:handlePreCommitResponse) - load Result {
  "TxnId": 8,
  "Label": "test_0_a6514887-ab92-4142-980c-d12712be0d93",
  "TwoPhaseCommit": "false",
  "Status": "Success",
  "Message": "OK",
  "NumberTotalRows": 1,
  "NumberLoadedRows": 1,
  "NumberFilteredRows": 0,
  "NumberUnselectedRows": 0,
  "LoadBytes": 46,
  "LoadTimeMs": 3595,
  "BeginTxnTimeMs": 0,
  "StreamLoadPutTimeMs": 1,
  "ReadDataTimeMs": 0,
  "WriteDataTimeMs": 3571,
  "CommitAndPublishTimeMs": 22
}
```

查看数据



The screenshot shows the Doris Playground interface. On the left, a sidebar displays the database structure: 'default_cluster:information_schema' and 'default_cluster:test' with a table 'stu'. The main area is titled 'Data Preview' and shows the contents of the 'stu' table. The table has two columns: 'name' and 'stu'. The data rows are: 'HTTP协议访问测试', '学生表-', 'stu', 'JDBC 访问Doris 测试', and 'HTTP协议访问代理转发测试'. The last row is highlighted with a red box. The interface includes tabs for 'Table Schema', 'Data Preview', and 'Data Import', and a 'Refresh' button.

测试成功。